

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



BÁO CÁO ĐỀ TÀI CUỐI KÌ
HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN ĐÔ
THỊ THÔNG MINH (SMART CITY IOT)

MÔN HỌC: TT CƠ SỞ VÀ ỨNG DỤNG IOT

GVHD: TS. TRƯƠNG QUANG PHÚC

Sinh viên thực hiện:

1. **GIANG VĨNH HUY** MSSV: 23119146
2. **NGUYỄN QUANG NHƠN** MSSV: 23119184

TP. HỒ CHÍ MINH, 04/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



BÁO CÁO ĐỀ TÀI CUỐI KÌ
HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN ĐÔ
THỊ THÔNG MINH (SMART CITY IOT)

MÔN HỌC: TT CƠ SỞ VÀ ỨNG DỤNG IOT

GVHD: TS. TRƯƠNG QUANG PHÚC

Sinh viên thực hiện:

1. **GIANG VĨNH HUY** MSSV: 23119146
2. **NGUYỄN QUANG NHƠN** MSSV: 23119184

TP. HỒ CHÍ MINH, 04/2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Quý Thầy Cô Trường Đại học Công nghệ Kỹ thuật TP. HCM, đặc biệt là các Thầy Cô trong Khoa Điện – Điện tử đã tạo điều kiện học tập tốt nhất và truyền đạt cho chúng em những kiến thức nền tảng quý báu trong suốt quá trình học tập tại trường.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Trương Quang Phúc – giảng viên phụ trách môn học Thực tập cơ sở và ứng dụng IoT. Nhờ sự hướng dẫn tận tình, những định hướng chuyên môn sát sao và sự tâm huyết của Thầy, chúng em mới có thể tháo gỡ được những khó khăn trong quá trình nghiên cứu, từ việc cấu hình phần cứng ESP32, tối ưu hóa cơ sở dữ liệu Firebase cho đến việc tích hợp thành công công nghệ Trí tuệ nhân tạo (Face ID, LLaMA 3.1) để hoàn thiện đề tài đồ án môn học này.

Mặc dù đã nỗ lực hết sức với tinh thần cầu thị cao nhất để hoàn thiện sản phẩm và bài báo cáo, nhưng do những giới hạn nhất định về mặt thời gian cũng như kinh nghiệm thực tiễn, đề tài chắc chắn vẫn còn những thiếu sót không thể tránh khỏi. Chúng em rất mong nhận được sự lượng thứ cũng như những ý kiến đánh giá, đóng góp quý báu từ Thầy để hệ thống được hoàn thiện hơn, đồng thời giúp chúng em rút ra những bài học kinh nghiệm quý giá cho con đường phát triển nghề nghiệp sau này.

Một lần nữa, chúng em xin kính chúc Thầy dồi dào sức khỏe, niềm vui và tiếp tục gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy cao quý.

Chúng em xin trân trọng cảm ơn!

Nhóm thực hiện đề tài
(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Tập thể nhóm thực hiện đề tài xin cam đoan đây là công trình nghiên cứu, tìm hiểu độc lập, được triển khai dưới sự hướng dẫn khoa học của TS. Trương Quang Phúc. Các nội dung, số liệu và kết quả đánh giá mô hình được trình bày trong báo cáo là hoàn toàn trung thực, khách quan, tuyệt đối không sao chép từ bất kỳ đồ án hay dự án công trình nào khác trước đó. Mọi thông tin tham khảo từ các tài liệu, bài báo khoa học hay các nguồn tài nguyên trên internet đều được tổng hợp, trích dẫn rõ ràng, đầy đủ và minh bạch trong phần Tài liệu tham khảo. Chúng em xin chịu hoàn toàn trách nhiệm trước Hội đồng đánh giá và Nhà trường về tính trung thực của lời cam đoan này.

Nhóm thực hiện đề tài
(Ký và ghi rõ họ tên)

Mục lục

1	Giới thiệu	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu của đề tài	2
1.3	Đối tượng và phạm vi nghiên cứu	2
1.4	Bố cục của báo cáo	3
2	Cơ sở lí thuyết và công nghệ áp dụng	4
2.1	Giới thiệu phần cứng: Vi điều khiển ESP32 và các linh kiện ngoại vi	4
2.1.1	Vi điều khiển ESP32	4
2.1.2	Cảm biến môi trường (DHT22) và cảm biến Analog giả lập	5
2.1.3	Thiết bị chấp hành (LED, Relay Module)	6
2.2	Nền tảng Google Firebase (Realtime Database)	7
2.2.1	Kiến trúc NoSQL và dữ liệu dạng cây JSON	7
2.2.2	Giao thức WebSockets và khả năng đồng bộ thời gian thực	8
2.3	Công nghệ phát triển Frontend (HTML5, CSS3, JavaScript)	9
2.3.1	Cấu trúc DOM và thiết kế giao diện đáp ứng (Responsive Design)	9
2.3.2	Thư viện đồ họa Chart.js	10
2.4	Công nghệ Trí tuệ nhân tạo (AI) ứng dụng	11
2.4.1	Nhận diện khuôn mặt với Face-api.js (Edge AI)	11
2.4.2	Xử lý ngôn ngữ tự nhiên (LLM) với LLaMA 3.1 thông qua Groq API	12
2.5	Trình giả lập điện tử Wokwi	13
2.5.1	Mô phỏng phần cứng và giao thức kết nối	13
2.5.2	Tính năng Wokwi-GUEST Wi-Fi	13
3	YÊU CẦU THIẾT KẾ VÀ ĐẶC TẢ PHẦN CỨNG	15
3.1	Yêu cầu thiết kế hệ thống	15
3.1.1	Yêu cầu chức năng	15
3.1.2	Yêu cầu phi chức năng	16
3.2	Sơ đồ đặc tả kiến trúc hệ thống (Block Diagram)	17
3.2.1	Khối Edge (Thiết bị IoT và Cảm biến)	18
3.2.2	Khối Cloud (Lưu trữ và Đồng bộ - Firebase)	18
3.2.3	Khối Application (Web Dashboard và Trí tuệ nhân tạo)	19

3.3	Thiết kế sơ đồ kết nối phần cứng	20
3.3.1	Sơ đồ nguyên lý (Schematic) trên Wokwi	20
3.3.2	Phân công chân tín hiệu (GPIO) của ESP32	20
4	GIẢI THÍCH MÃ NGUỒN	22
4.1	Tổng quan kiến trúc mã nguồn	22
4.2	Cấu hình và khởi tạo Firebase	26
4.2.1	Nhập khẩu module Firebase	26
4.2.2	Đối tượng cấu hình Firebase	26
4.2.3	Cấu hình Groq AI	27
4.3	Quản lý trạng thái ứng dụng	27
4.3.1	dashboardState	27
4.3.2	Đối tượng assistantState	28
4.3.3	faceAuthState	29
4.4	Module xác thực Face ID	31
4.4.1	Tải model nhận diện khuôn mặt	32
4.4.2	Thu thập dữ liệu khuôn mặt	32
4.4.3	Hiển thị khung theo dõi trực tiếp	33
4.4.4	Luồng đăng ký khuôn mặt (Enroll)	35
4.4.5	Luồng xác thực khuôn mặt (Verify)	35
4.5	Module kết nối và lắng nghe dữ liệu Firebase	36
4.5.1	Cấu trúc dữ liệu Firebase	36
4.5.2	Các listener thời gian thực	36
4.5.3	Điều khiển thiết bị	37
4.6	Module cập nhật giao diện Dashboard	39
4.6.1	Hàm updateDashboard()	39
4.6.2	Trực quan hóa dữ liệu lịch sử với Chart.js	42
4.6.3	Bản đồ đô thị tương tác	43
4.7	Module trợ lý AI	43
4.7.1	Kiến trúc hai lớp của trợ lý	43
4.7.2	Xây dựng System Prompt động	45
4.7.3	Gọi API Groq	46
4.7.4	Quản lý lịch sử hội thoại	46
4.7.5	Trợ lý nội tuyến	47
4.8	Module giao diện và CSS	47
4.8.1	Hệ thống thiết kế và biến CSS	47
4.8.2	Font chữ và nền trang	48
4.8.3	Phần tử DOM và ánh xạ	48
4.9	Cấu hình phần cứng Wokwi	48

4.9.1	Sơ đồ mạch (diagram.json)	48
4.9.2	Cấu hình Wokwi (wokwi.toml)	49
4.9.3	Luồng dữ liệu tổng thể	50
5	KẾT QUẢ ĐẠT ĐƯỢC	51
5.1	Kết quả xây dựng hệ thống phần cứng mô phỏng	51
5.1.1	Mạch điện Wokwi hoàn chỉnh	51
5.1.2	Ảnh xạ chức năng cảm biến	52
5.1.3	Điều khiển chấp hành	53
5.2	Kết quả kết nối Firebase Realtime Database	53
5.2.1	Đồng bộ dữ liệu thời gian thực	53
5.2.2	Cấu trúc dữ liệu hợp lý	53
5.2.3	Nhật ký hoạt động (System Logging)	54
5.3	Kết quả hệ thống giám sát đa thông số	56
5.3.1	Dashboard giám sát theo thời gian thực	56
5.3.2	Biểu đồ lịch sử	57
5.4	Kết quả hệ thống điều khiển thông minh AUTO/MANUAL	58
5.4.1	Chế độ AUTO	58
5.4.2	Chế độ MANUAL	58
5.4.3	Đồng bộ trạng thái hai chiều	58
5.5	Kết quả hệ thống xác thực Face ID	59
5.5.1	Nhận diện khuôn mặt trên trình duyệt	59
5.5.2	Quy trình đăng ký và xác thực	59
5.5.3	Tính năng theo dõi trực tiếp	60
5.6	Kết quả tích hợp trợ lý AI (LLaMA 3.1)	60
5.6.1	Trợ lý nhận thức ngữ cảnh đô thị	60
5.6.2	Khả năng đa nhiệm	61
5.6.3	Cơ chế dự phòng thông minh	61
5.6.4	Gợi ý câu hỏi nhanh	61
5.7	Kết quả giao diện web dashboard	62
5.7.1	Thiết kế trực quan	62
5.7.2	Tính năng phân quyền	62
5.7.3	Nhật ký hoạt động	62
5.8	Đánh giá tổng thể	63
5.8.1	Mức độ hoàn thành mục tiêu	63
5.8.2	Hạn chế còn tồn tại	63
6	KẾT LUẬN, NHỮNG HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI	64
6.1	Kết luận	64

6.1.1	Những gì đã đạt được	64
6.1.2	Đánh giá mức độ hoàn thành	65
6.2	Những hạn chế còn tồn tại	65
6.2.1	Hạn chế về môi trường mô phỏng	65
6.2.2	Hạn chế về bảo mật	65
6.2.3	Hạn chế về hệ thống xác thực Face ID	66
6.2.4	Hạn chế về khả năng mở rộng	66
6.2.5	Hạn chế về trợ lý AI	66
6.3	Hướng phát triển trong tương lai	66
6.3.1	Triển khai phần cứng thực tế	66
6.3.2	Xây dựng backend bảo mật	66
6.3.3	Nâng cấp hệ thống Face ID	67
6.3.4	Mở rộng quy mô hệ thống	67
6.3.5	Tích hợp phân tích dữ liệu và AI dự báo	67
6.3.6	Tối ưu hiệu năng và trải nghiệm người dùng	67
6.4	Lời kết	67
7	KẾT QUẢ VẬN HÀNH VÀ HƯỚNG DẪN THAO TÁC	69
7.1	Video Demo và Hướng dẫn hệ thống	69
7.2	Mã QR truy cập nhanh	70

Danh mục bảng

3.3.1 Bảng phân công chân tín hiệu (GPIO) của ESP32 cho các thiết bị	21
5.8.1 Đánh giá mức độ hoàn thành các mục tiêu đề ra	63

Danh mục hình ảnh

2.1.1 Sơ đồ chân và cấu trúc vi điều khiển ESP32	5
2.1.2 Tập hợp các cảm biến và linh kiện giả lập trong hệ thống	6
2.1.3 Sơ đồ kết nối các thiết bị chấp hành trong đô thị thông minh	6
2.2.1 Cấu trúc tổ chức dữ liệu JSON của hệ thống Smart City trên Firebase	8
2.2.2 Cơ chế đồng bộ dữ liệu thời gian thực giữa các thành phần hệ thống	9
2.4.1 Cơ chế hoạt động của thuật toán nhận diện khuôn mặt dựa trên Face-api.js	12
2.4.2 Mô hình tích hợp dữ liệu cảm biến thời gian thực vào Trợ lý ảo AI thông qua Groq API	13
3.1.1 Sơ đồ phân rã chức năng của hệ thống	17
3.2.1 Sơ đồ khối đặc tả kiến trúc tổng quát của hệ thống Smart City	18
3.3.1 Sơ đồ kết nối linh kiện phần cứng mô phỏng trên nền tảng Wokwi	20
4.1.1 Sơ đồ kiến trúc ba tầng của hệ thống Smart City IoT	23
4.1.2 Lưu đồ thuật toán vòng lặp chính của firmware ESP32	25
4.2.1 Nhập khẩu module và cấu hình Firebase	26
4.2.2 Cấu hình Groq AI	27
4.2.3 Khởi tạo Firebase SDK và cấu hình Groq AI trong mã nguồn	27
4.3.1 Ba đối tượng quản lý trạng thái ứng dụng trong <code>script.js</code>	29
4.3.2 Tham số tinh chỉnh thuật toán Face ID trong <code>faceAuthConfig</code>	30
4.4.1 Flowchart luồng đăng ký (Enroll) và xác thực (Verify) Face ID	31
4.4.2 Hàm <code>loadFaceModels()</code> tải song song ba model AI bằng <code>Promise.all()</code>	32
4.4.3 Hàm <code>captureStableDescriptor()</code> thu thập và tính trung bình descriptor khuôn mặt	33
4.4.4 Hàm <code>drawFocusBox()</code> vẽ khung nhận diện với bốn góc L trên canvas	34
4.4.5 Hàm <code>saveFaceProfile()</code> và <code>loadFaceProfileFromStorage()</code> quản lý profile trong <code>localStorage</code>	35
4.4.6 Hàm <code>verifyFaceLogin()</code> xác thực khuôn mặt bằng khoảng cách Euclidean	36
4.5.1 Khai báo tham chiếu Firebase và đăng ký các listener <code>onValue()</code> thời gian thực	37
4.5.2 Lưu đồ thuật toán điều khiển thiết bị với kiểm tra quyền và chế độ AUTO/MANUAL	38
4.5.3 Hàm <code>controlDevice()</code> ghi lệnh điều khiển thiết bị lên Firebase	39
4.5.4 Hàm <code>appendLog()</code> ghi nhật ký hoạt động admin lên Firebase	39
4.6.1 Hàm <code>updateDashboard()</code> điều phối toàn bộ cập nhật giao diện	40
4.6.2 Lưu đồ thuật toán hàm <code>updateDashboard()</code> cập nhật giao diện theo thời gian thực	41
4.6.3 Hàm <code>updateChart()</code> cập nhật biểu đồ với cửa sổ trượt 12 điểm	43
4.7.1 Lưu đồ thuật toán xử lý câu hỏi của trợ lý AI với cơ chế dự phòng hai lớp	44

4.7.2 Hàm <code>sendAssistantQuestion()</code> xử lý câu hỏi với cơ chế dự phòng hai lớp	45
4.7.3 Hàm <code>buildAssistantSystemPrompt()</code> nhúng dữ liệu cảm biến vào system prompt	45
4.7.4 Hàm <code>requestAssistantReply()</code> gọi Groq API với lịch sử hội thoại	46
4.8.1 Khai báo biến CSS trong <code>:root</code> của <code>styles.css</code>	47
4.8.2 Khai báo CSS tạo nền lưới kẻ đặc trưng cho giao diện trong <code>styles.css</code>	48
4.9.1 Khai báo linh kiện và kết nối trong tệp <code>diagram.json</code>	49
4.9.2 Tệp <code>wokwi.toml</code> liên kết firmware PlatformIO với môi trường mô phỏng Wokwi	50
5.1.1 Mạch điện mô phỏng ESP32 và các cảm biến trên nền tảng Wokwi	52
5.1.2 Cây dữ liệu Firebase Realtime Database với 4 nhánh chính của hệ thống	53
5.2.1 Cây dữ liệu Firebase Realtime Database với 4 nhánh chính của hệ thống	54
5.2.2 Nhật ký hoạt động admin được lưu trữ trên Firebase Console	55
5.3.1 Toàn cảnh giao diện Smart City Dashboard đang hiển thị dữ liệu thời gian thực	56
5.3.2 Biểu đồ xu hướng bốn thông số cảm biến theo thời gian thực (Chart.js)	56
5.3.3 Thông số chất lượng không khí và thông số ánh sáng	56
5.3.4 Thẻ bật cảnh báo khi mực nước $\geq 75\%$	57
5.3.5 Biểu đồ lịch sử	57
5.4.1 Bảng điều khiển thiết bị sau khi admin đăng nhập – hiển thị đầy đủ nút AUTO/MANUAL và toggle ON/OFF	58
5.5.1 Giao diện xác thực Face ID – khung xanh lá nhận diện khuôn mặt theo thời gian thực	59
5.6.1 Panel trợ lý AI đang trả lời câu hỏi dựa trên dữ liệu cảm biến thời gian thực	61
5.7.1 Giao diện tổng	62
7.2.1 Mã QR truy cập Video vận hành hệ thống Smart City	70

Danh mục các từ viết tắt

Dưới đây là danh mục các từ viết tắt được sử dụng trong báo cáo.

Các từ viết tắt	Định nghĩa
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
AQI	Air Quality Index
CDN	Content Delivery Network
CNN	Convolutional Neural Network
CSS	Cascading Style Sheets
DOM	Document Object Model
ESP	Espressif Systems
GPIO	General-Purpose Input/Output
HTML	HyperText Markup Language
IoT	Internet of Things
JSON	JavaScript Object Notation
JS	JavaScript
LDR	Light Dependent Resistor
LLaMA	Large Language Model Meta AI
MQTT	Message Queuing Telemetry Transport
RTDB	Realtime Database
UI	User Interface

Chương 1

Giới thiệu

1.1 Lý do chọn đề tài

Trong bối cảnh Cách mạng công nghiệp 4.0 và xu hướng chuyển đổi số đang diễn ra mạnh mẽ trên toàn cầu, mô hình "Đô thị thông minh" (Smart City) không còn là một khái niệm xa vời mà đã trở thành tiêu chuẩn tất yếu cho sự phát triển bền vững của các quốc gia. Cùng với tốc độ đô thị hóa nhanh chóng, sự gia tăng dân số tại các khu đô thị lớn mang lại nhiều thách thức to lớn về mặt quản lý hạ tầng, bảo vệ môi trường, cũng như đảm bảo an toàn và chất lượng cuộc sống cho người dân. Các vấn đề nhức nhối như ô nhiễm không khí, biến đổi khí hậu dẫn đến ngập lụt cục bộ, hay sự quá tải của hệ thống giao thông và bãi đỗ xe đã và đang đặt ra những bài toán cấp thiết cho các nhà quản lý.

Để giải quyết triệt để những thách thức trên, việc ứng dụng công nghệ Internet vạn vật (Internet of Things - IoT) kết hợp với Trí tuệ nhân tạo (AI) là một hướng đi mang tính chiến lược. Thực tế cho thấy, các hệ thống giám sát môi trường và quản lý hạ tầng truyền thống thường hoạt động rời rạc, thiếu tính kết nối tập trung và đặc biệt là không cung cấp được dữ liệu theo thời gian thực (real-time). Điều này dẫn đến sự chậm trễ trong việc đưa ra quyết định hoặc phản ứng trước các tình huống khẩn cấp, điển hình như khi xảy ra triều cường gây ngập lụt hay khi chỉ số chất lượng không khí (AQI) xuống mức nguy hiểm đối với sức khỏe.

Xuất phát từ nhu cầu thực tiễn đó, đề tài "Hệ thống giám sát và điều khiển đô thị thông minh (Smart City IoT)" được lựa chọn nhằm nghiên cứu, thiết kế và phát triển một nền tảng quản lý đô thị tập trung, hiện đại. Hệ thống này có khả năng thu thập liên tục các thông số môi trường (nhiệt độ, độ ẩm, chất lượng không khí, mực nước) và trạng thái hạ tầng (số lượng chỗ đỗ xe trống), đồng thời cho phép điều khiển tự động hoặc thủ công các thiết bị cơ điện (đèn đường, máy bơm thoát nước). Không chỉ dừng lại ở việc kết nối vạn vật, đề tài còn hướng tới việc tích hợp các mô hình Trí tuệ nhân tạo hiện đại nhất – bao gồm bảo mật sinh trắc học Face ID và Trợ lý ảo AI – ngay trên nền tảng Web, mang lại một giải pháp giám sát toàn diện, an toàn, có tính tương tác cao và bắt kịp xu hướng phát triển công nghệ của thế giới.

1.2 Mục tiêu của đề tài

Đề tài hướng tới việc xây dựng một hệ thống IoT hoàn chỉnh từ tầng phần cứng thu thập dữ liệu (Edge) đến nền tảng đám mây (Cloud) và giao diện người dùng (Application), với các mục tiêu cụ thể như sau:

- Thiết kế và mô phỏng hệ thống phần cứng thu thập dữ liệu: Sử dụng vi điều khiển trung tâm ESP32 kết hợp với tập hợp các cảm biến chuyên dụng để thu thập thông tin về nhiệt độ, độ ẩm, bãi đỗ xe, mực nước, cường độ ánh sáng và chất lượng không khí (AQI). Quá trình này được thực hiện trực quan trên môi trường giả lập điện tử Wokwi để đảm bảo tính khả thi và an toàn trước khi triển khai phần cứng thực tế.
- Xây dựng hệ thống quản lý dữ liệu thời gian thực: Áp dụng nền tảng đám mây Google Firebase Realtime Database làm lớp trung gian (Middleware) để lưu trữ, xử lý và đồng bộ hóa trạng thái liên tục giữa các thiết bị ngoại vi và giao diện điều khiển với độ trễ tối thiểu.
- Phát triển giao diện điều khiển trung tâm (Web Dashboard): Ứng dụng công nghệ Web (HTML5, CSS3, Vanilla Javascript) để thiết kế một bảng điều khiển trực quan, chuyên nghiệp, hỗ trợ đa nền tảng (Responsive Design). Hệ thống cũng tích hợp thư viện Chart.js để biểu diễn xu hướng và dữ liệu lịch sử của các cảm biến.
- Nghiên cứu và tích hợp Trí tuệ nhân tạo (AI): Áp dụng công nghệ nhận diện khuôn mặt Edge AI thông qua thư viện Face-api.js nhằm xác thực quyền hạn người quản trị, tăng cường tính bảo mật cho hệ thống điều khiển. Đồng thời, tiên phong ứng dụng Mô hình Ngôn ngữ Lớn (LLM) – LLaMA 3.1 thông qua Groq API – để xây dựng một Trợ lý ảo đô thị, có khả năng diễn giải ngữ cảnh dữ liệu hiện tại, tư vấn và giải đáp tình trạng hệ thống cho người quản lý bằng ngôn ngữ tự nhiên.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài bao trùm các thành phần cốt lõi của một kiến trúc hệ thống IoT và AI hiện đại, bao gồm:

- Vi điều khiển và thiết bị ngoại vi: Bo mạch xử lý trung tâm ESP32 (với khả năng kết nối Wi-Fi tích hợp) cùng các cảm biến môi trường (DHT22), các cơ cấu mô phỏng tín hiệu Analog/Digital (biến trở, nút nhấn) và thiết bị chấp hành (LED, Relay).
- Giao thức đồng bộ và Đám mây: Giao thức kết nối không dây, WebSockets và cơ sở dữ liệu thời gian thực (Realtime Database) của hệ sinh thái Google Firebase.
- Ngôn ngữ và Công nghệ Frontend: Các công nghệ định dạng trình duyệt (HTML5, CSS3) và xử lý logic (JavaScript, Chart.js).
- Mô hình Trí tuệ nhân tạo: Các mô hình máy học nhận diện hình ảnh (TinyFaceDetector, FaceRecognitionNet) và xử lý ngôn ngữ tự nhiên (LLM).

Về phạm vi nghiên cứu, do những giới hạn nhất định về mặt thời gian cũng như nguồn lực vật tư

trong khuôn khổ môn học, tăng phần cứng của đề tài sẽ tập trung triển khai và đánh giá trên hệ thống nền tảng giả lập điện tử Wokwi thay vì lắp ráp linh kiện vật lý. Tuy nhiên, các thiết bị ảo này vẫn kết nối ra môi trường Internet và giao tiếp với hệ thống máy chủ Firebase hoàn toàn thực tế. Bên cạnh đó, các mô hình AI được tinh giản để ưu tiên thực thi trực tiếp trên môi trường trình duyệt web của người dùng (Client-side) nhằm đảm bảo tốc độ và bảo mật dữ liệu, thay vì xây dựng một máy chủ xử lý AI độc lập (Server-side).

1.4 Bố cục của báo cáo

Báo cáo được tổ chức thành 6 chương với cấu trúc như sau:

- **Chương 1:** Giới thiệu đề tài, mục tiêu và phạm vi nghiên cứu.
- **Chương 2:** Cơ sở lý thuyết và công nghệ áp dụng (Firebase, ESP32, HTML5/CSS3/JavaScript, Face-api.js, Groq LLM, Wokwi).
- **Chương 3:** Yêu cầu thiết kế hệ thống và đặc tả phần cứng.
- **Chương 4:** Thiết kế phần mềm, kiến trúc giải thuật và lưu đồ xử lý.
- **Chương 5:** Kết quả thực tế của hệ thống (giao diện, hiệu năng, khả năng tương tác).
- **Chương 6:** Kết luận, những hạn chế và hướng phát triển trong tương lai.
- **Chương 7:** Kết quả vận hành và hướng dẫn thao tác

Chương 2

Cơ sở lí thuyết và công nghệ áp dụng

2.1 Giới thiệu phần cứng: Vi điều khiển ESP32 và các linh kiện ngoại vi

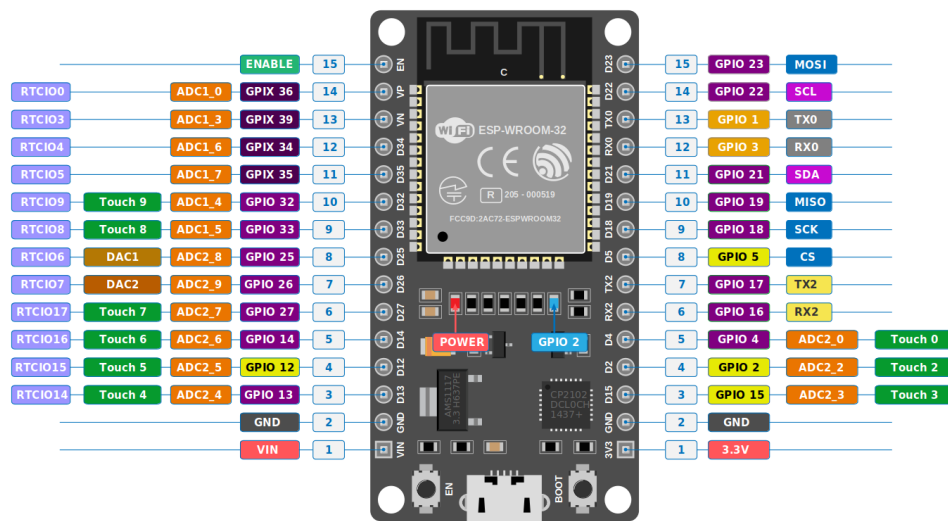
Trong kiến trúc của hệ thống IoT Smart City, phần cứng đóng vai trò là tầng thu thập dữ liệu (Perception Layer) và thực thi lệnh (Actuator Layer). Các thành phần được lựa chọn dựa trên tiêu chí hiệu năng xử lý đa nhiệm, khả năng kết nối không dây mạnh mẽ và độ chính xác của các cảm biến đo lường môi trường.

2.1.1 Vi điều khiển ESP32

Vi điều khiển trung tâm được sử dụng là ESP32-WROOM-32, một giải pháp Hệ thống trên chip (SoC) mạnh mẽ và tiết kiệm năng lượng. ESP32 vượt trội hơn các dòng vi điều khiển truyền thống nhờ sở hữu bộ vi xử lý lõi kép (Dual-core) Xtensa® Dual-Core 32-bit LX6 hoạt động với tần số lên đến 240 MHz.

Các đặc tính kỹ thuật cốt lõi của ESP32 bao gồm:

- Kết nối không dây: Tích hợp sẵn Wi-Fi chuẩn 802.11 b/g/n và Bluetooth v4.2/BLE, cho phép thiết bị kết nối trực tiếp với Internet để đồng bộ dữ liệu tới Cloud (Firebase) mà không cần thêm module phụ trợ.
- Hệ thống chân ngoại vi (GPIO): Cung cấp số lượng lớn chân GPIO hỗ trợ các giao thức I2C, SPI, UART và đặc biệt là hệ thống chuyển đổi Analog-to-Digital (ADC) độ phân giải 12-bit, lý tưởng để đọc chính xác tín hiệu từ các cảm biến Analog như mực nước hay chất lượng không khí.
- Khả năng đa nhiệm: Kiến trúc lõi kép cho phép hệ thống thực hiện đồng thời việc đọc cảm biến, xử lý logic điều khiển tự động và duy trì kết nối Wi-Fi ổn định.

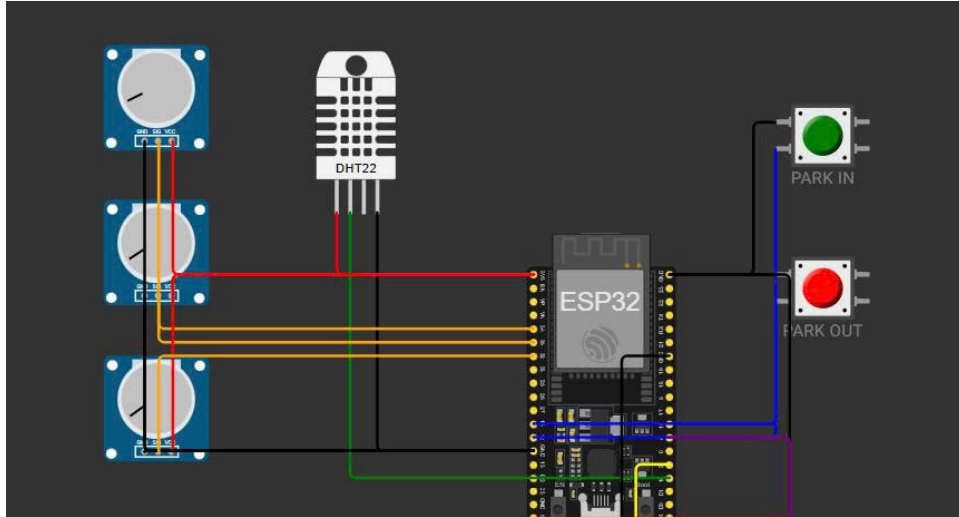


Hình 2.1.1: Sơ đồ chân và cấu trúc vi điều khiển ESP32

2.1.2 Cảm biến môi trường (DHT22) và cảm biến Analog giả lập

Hệ thống sử dụng kết hợp giữa cảm biến kỹ thuật số chuyên dụng và các cơ cấu giả lập để giám sát toàn diện các chỉ số đô thị:

- Cảm biến nhiệt độ và độ ẩm DHT22: Là loại cảm biến kỹ thuật số có độ tin cậy cao, cung cấp dữ liệu nhiệt độ và độ ẩm không khí với độ chính xác lần lượt là $\pm 0.5^{\circ}C$ và $\pm 2\%RH$. Cảm biến này được kết nối với ESP32 thông qua chân GPIO 15.
- Cảm biến Analog giả lập (Biến trở): Do đặc thù triển khai trên môi trường mô phỏng Wokwi, các biến trở được sử dụng để đóng vai trò như các cảm biến Analog thực thụ. Điện áp đầu ra từ biến trở được ESP32 đọc qua chân ADC và chuyển đổi thành tỷ lệ phần trăm (0–100%). Các chân kết nối bao gồm:
 - Cảm biến mực nước khu vực: Kết nối chân GPIO 34.
 - Cảm biến chất lượng không khí (AQI): Kết nối chân GPIO 35.
 - Cảm biến cường độ ánh sáng: Kết nối chân GPIO 32.
- Cảm biến bãi đỗ xe (Nút nhấn): Sử dụng hai nút nhấn (Push button) để giả lập cảm biến hồng ngoại nhận diện xe vào và xe ra tại cổng bãi đỗ. Các nút nhấn được thiết lập ở chế độ *Input Pullup* kết nối với chân GPIO 12 (Vào) và GPIO 14 (Ra).

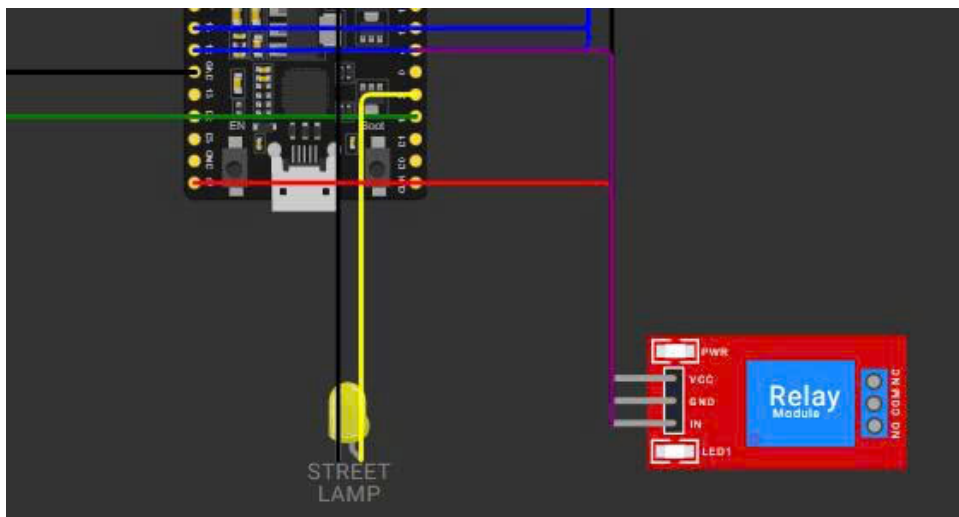


Hình 2.1.2: Tập hợp các cảm biến và linh kiện giả lập trong hệ thống

2.1.3 Thiết bị chấp hành (LED, Relay Module)

Lớp chấp hành được thiết kế để phản hồi lại các điều kiện môi trường hoặc lệnh từ người quản trị:

- Đèn LED (Street Light): Sử dụng để mô phỏng hệ thống chiếu sáng đô thị. LED được điều khiển qua chân GPIO 2. Ở chế độ tự động, đèn sẽ tự kích hoạt khi cường độ ánh sáng môi trường xuống dưới mức 40%.
- Relay Module (Drain Pump): Điều khiển máy bơm thoát nước tiêu úng khu vực. Relay được kết nối với chân GPIO 4. Hệ thống sẽ tự động đóng mạch khởi động máy bơm khi mực nước vượt ngưỡng cảnh báo ngập ($> 70\%$).



Hình 2.1.3: Sơ đồ kết nối các thiết bị chấp hành trong đô thị thông minh

2.2 Nền tảng Google Firebase (Realtime Database)

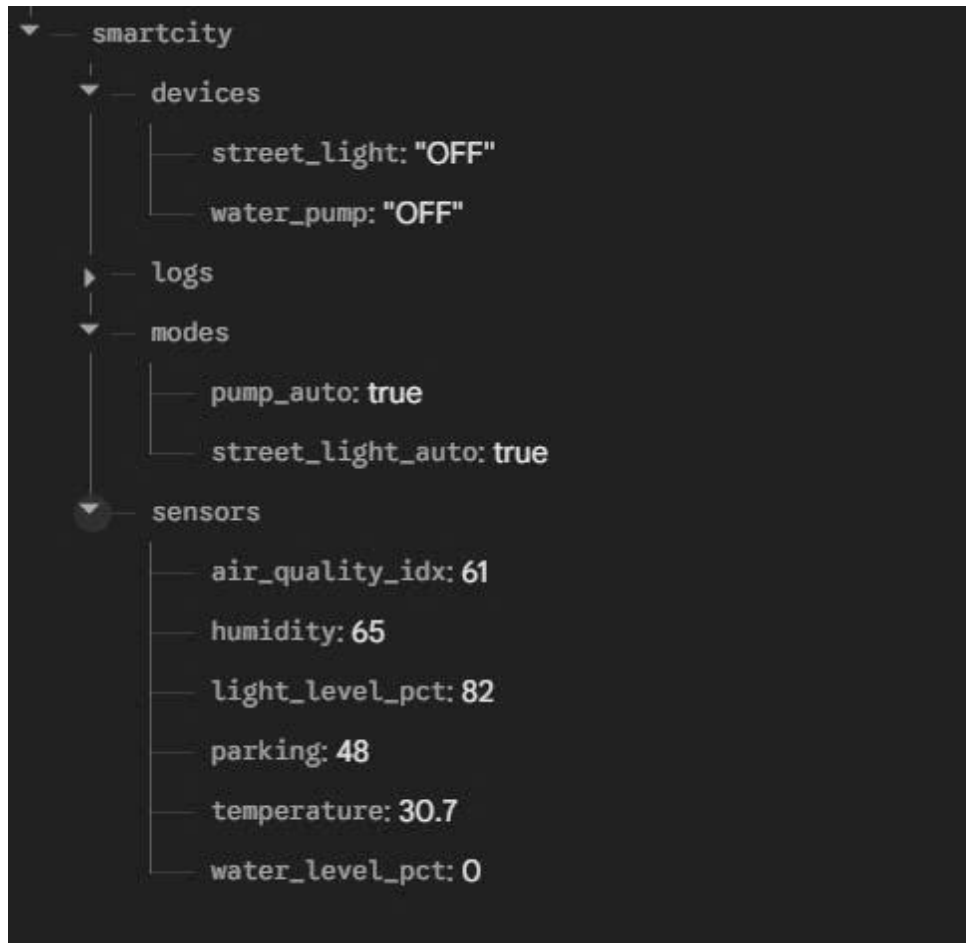
Để giải quyết bài toán thu thập và điều phối dữ liệu cho hệ thống đô thị thông minh, dự án sử dụng Google Firebase Realtime Database làm lớp trung gian (Middleware). Đây là giải pháp tối ưu để lưu trữ và đồng bộ hóa hàng ngàn bản ghi dữ liệu cảm biến một cách tức thời giữa thiết bị phần cứng và giao diện quản lý [1].

2.2.1 Kiến trúc NoSQL và dữ liệu dạng cây JSON

Khác với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống, Firebase Realtime Database là một cơ sở dữ liệu NoSQL lưu trữ dữ liệu dưới dạng một cây JSON (JavaScript Object Notation) khổng lồ [1]. Dữ liệu này được đồng bộ hóa theo thời gian thực tới mọi máy khách (client) đang kết nối.

Trong dự án này, cơ sở dữ liệu được tổ chức phân cấp rõ ràng để phản ánh cấu trúc vận hành của đô thị:

- Nhánh `/smartcity/sensors`: Lưu trữ trạng thái thời gian thực của các cảm biến bao gồm nhiệt độ (`temperature`), độ ẩm (`humidity`), bãi đỗ xe (`parking`), mực nước (`water_level_pct`), chất lượng không khí (`air_quality_idx`) và cường độ ánh sáng (`light_level_pct`).
- Nhánh `/smartcity/devices`: Lưu trữ trạng thái bật/tắt (`ON/OFF`) của các thiết bị chấp hành như đèn đường (`street_light`) và máy bơm (`water_pump`).
- Nhánh `/smartcity/modes`: Lưu trữ các thiết lập về chế độ vận hành tự động (Auto Mode) cho từng loại thiết bị.
- Nhánh `/smartcity/logs`: Lưu trữ nhật ký hoạt động của toàn bộ hệ thống.



Hình 2.2.1: Cấu trúc tổ chức dữ liệu JSON của hệ thống Smart City trên Firebase

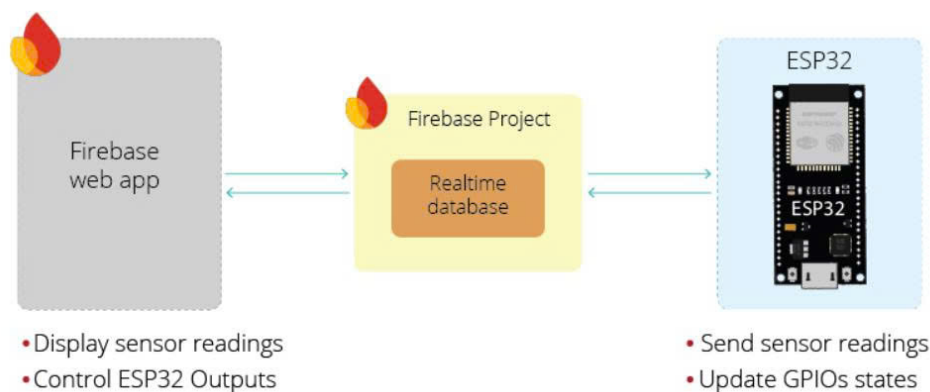
2.2.2 Giao thức WebSockets và khả năng đồng bộ thời gian thực

Firebase Realtime Database sử dụng giao thức WebSockets thay vì cơ chế yêu cầu - phản hồi (Request - Response) truyền thống của giao thức HTTP [1]. Giao thức này tạo ra một kênh kết nối hai chiều liên tục, cho phép máy chủ chủ động "đẩy" (push) sự thay đổi trạng thái xuống tất cả các máy khách ngay khi có bất kỳ nút dữ liệu nào bị sửa đổi.

Cơ chế này đảm bảo tính tương tác tức thời trong các kịch bản điều khiển:

- Tại vi điều khiển ESP32: Mã nguồn C++ sử dụng thư viện **Firebase-ESP-Client** để cập nhật dữ liệu lên các đường dẫn cụ thể. Ví dụ, giá trị nhiệt độ được gửi lên thông qua hàm **Firebase.RTDB.setFloat()**.
- Tại giao diện Web Dashboard: Sử dụng Firebase JS SDK với hàm **onValue()** để lắng nghe các sự kiện thay đổi dữ liệu trên máy chủ [1]. Ngay khi dữ liệu trên Firebase biến động, hàm callback sẽ cập nhật trực tiếp vào các phần tử DOM mà không cần tải lại trang.

Nhờ ứng dụng công nghệ này, hệ thống đạt được độ trễ truyền tải cực thấp (thường dưới 1 giây trong điều kiện mạng ổn định), đáp ứng yêu cầu khắt khe về tính thời gian thực cho việc giám sát đô thị.



Hình 2.2.2: Cơ chế đồng bộ dữ liệu thời gian thực giữa các thành phần hệ thống

2.3 Công nghệ phát triển Frontend (HTML5, CSS3, JavaScript)

Lớp ứng dụng người dùng (Web Frontend) đóng vai trò là giao diện tương tác trực tiếp (Dashboard), cho phép nhà quản lý theo dõi các chỉ số đo lường đô thị và điều khiển trạng thái hạ tầng từ xa. Để tối ưu hóa hiệu suất thiết bị và đảm bảo khả năng phản hồi thời gian thực với độ trễ thấp nhất, toàn bộ giao diện của dự án được xây dựng trực tiếp dựa trên bộ ba công nghệ cốt lõi của Web: HTML5, CSS3 và JavaScript thuần (Vanilla JavaScript), hạn chế tối đa sự phụ thuộc vào các framework công kênh [2].

Bên cạnh HTML5 và CSS3 đảm nhận việc xây dựng bộ khung và tạo hình thẩm mỹ, JavaScript (đặc biệt là các chuẩn ES6 mới nhất) đóng vai trò là "bộ não" điều khiển logic phía máy khách (Client-side). Ứng dụng khai thác mạnh mẽ các cơ chế xử lý bất đồng bộ (Asynchronous) như **Promise**, **async/await** kết hợp cùng **Fetch API** để giao tiếp mượt mà với các dịch vụ bên ngoài (như AI Groq) và kết nối WebSocket thời gian thực của Firebase, giúp trang web không bao giờ bị "đóng băng" (block) trong quá trình chờ phản hồi [3].

2.3.1 Cấu trúc DOM và thiết kế giao diện đáp ứng (Responsive Design)

Giao diện giám sát đô thị được thiết kế theo hướng phân hệ (module) và linh hoạt, đáp ứng khả năng hiển thị trên đa dạng nền tảng thông qua sự phối hợp chặt chẽ giữa cây cấu trúc DOM của HTML5 và định dạng CSS3.

Cấu trúc HTML5, Mô hình DOM và JavaScript tương tác:

Mô hình Khách thể Tài liệu (Document Object Model - DOM) là một chuẩn do W3C quy định, biểu diễn tài liệu HTML dưới dạng một cây cấu trúc đối tượng logic [4]. Trong hệ thống này, HTML5 cung cấp một bộ khung ngữ nghĩa (Semantic markup) chuẩn mực, phân chia rõ ràng các khu vực chức năng:

- Khu vực thống kê (Stats Grid): Sử dụng các thẻ `<article>` độc lập cho từng tham số như nhiệt độ, độ ẩm, chất lượng không khí, mực nước và trạng thái đỗ xe.
- Khu vực điều khiển chính (Control Panel): Chứa các nút bấm thao tác (Buttons) chuyển đổi linh

hoạt giữa chế độ tự động (Auto Mode) và thủ công (Manual Mode) cho các thiết bị chấp hành (máy bơm, đèn chiếu sáng).

- Khu vực Trợ lý AI (AI Panel): Không gian dành cho hiển thị luồng Camera (phục vụ quét khuôn mặt đăng nhập) và giao diện hộp thoại với Trợ lý ảo đô thị.

Lợi thế lớn nhất của DOM là cho phép JavaScript dễ dàng truy xuất (thông qua các hàm như `document.getElementById` hay `querySelector`) và thao tác chỉnh sửa (DOM Manipulation). Bất cứ khi nào nhận được dữ liệu thay đổi từ Firebase, sự kiện lắng nghe (Event Listener) của JS sẽ tự động cập nhật nội dung văn bản (Text content) hoặc thay đổi thuộc tính CSS, giúp dữ liệu hiển thị lên màn hình tức thời mà không cần tải lại toàn bộ trang web.

Thiết kế giao diện đáp ứng (Responsive Design) với CSS3:

Nhằm đảm bảo trải nghiệm giám sát hệ thống trên mọi thiết bị (từ màn hình Desktop lớn đến điện thoại di động Mobile), dự án áp dụng toàn diện kỹ thuật Thiết kế Đáp ứng (Responsive Web Design) [5]. Hệ thống sử dụng các tính năng CSS3 tiên tiến nhất:

- CSS Grid và Flexbox: Các thẻ thông số (Cards) và vùng biểu đồ được tổ chức theo dạng lưới đa cột (`display: grid`) để sắp xếp không gian khoa học. Flexbox hỗ trợ việc căn giữa nội dung và co giãn tỷ lệ bên trong từng thẻ một cách tối ưu.
- Media Queries (@media): Kỹ thuật đóng vai trò là "chìa khóa" của Responsive. Các điểm ngắt (Breakpoints) được định nghĩa sẵn (ví dụ: màn hình rộng dưới 768px cho thiết bị di động). Khi thiết bị thu hẹp, CSS tự động thay đổi cấu trúc lưới đa cột thành bố cục một cột (Single Column Layout), đẩy các khối chức năng xếp chồng lên nhau từ trên xuống dưới, đảm bảo nội dung hiển thị to và rõ ràng trên di động.
- CSS Variables (Biến màu sắc): Được định nghĩa tập trung ở gốc `:root`, hỗ trợ triển khai bộ thiết kế Giao diện nền tối (Dark Mode) - xu hướng hiện đại giúp giảm mỏi mắt cho người quản trị khi quan sát hệ thống vào ban đêm, đồng thời tiết kiệm năng lượng hiển thị.

2.3.2 Thư viện đồ họa Chart.js

Trong môi trường quản lý Đô thị thông minh, việc chỉ hiển thị con số tức thời là chưa đủ để phát hiện các phân tích xu hướng biến đổi (như nhiệt độ đang tăng vọt hay chất lượng không khí suy giảm nhanh chóng). Do đó, trực quan hóa dữ liệu lịch sử đóng vai trò cốt lõi. Dự án sử dụng Chart.js - một trong những thư viện vẽ biểu đồ mã nguồn mở mạnh mẽ nhất, hoạt động trực tiếp trên nền tảng thẻ `<canvas>` của HTML5 [6].

Thư viện Chart.js đem đến các giải pháp đồ họa tiên tiến được tùy biến cho hệ thống:

- Trực quan hoá biểu đồ đa trục (Multi-axis Line Chart): Do đặc thù các đại lượng môi trường có đơn vị và biên độ đo khác biệt nhau (Nhiệt độ $^{\circ}C$, Độ ẩm %, Mức nước %, ...), biểu đồ được thiết kế với cơ chế đa trục tung (trục Y độc lập cho từng tham số). Cơ chế này giúp các đường đồ thị không bị nén hay chồng chéo lên nhau, cho phép nhà quản lý đối chiếu sự biến thiên tương quan giữa nhiều chỉ số trên cùng một mặt phẳng trục hoành thời gian (trục X).
- Cơ chế cập nhật thời gian thực (Real-time Sliding Window): Biểu đồ được kết nối trực tiếp với

luồng dữ liệu mới thông qua JavaScript. Thuật toán của ứng dụng áp dụng nguyên tắc "cửa sổ trượt"(Sliding Window): khi có điểm dữ liệu mới, hàm `update()` sẽ thêm (**push**) phần tử vào mảng đồ họa. Đồng thời, nếu số điểm lưu trữ vượt quá giới hạn khung nhìn (ví dụ: 12 lần đo gần nhất), điểm cũ nhất sẽ bị xóa (**shift**). Quá trình này tạo ra hiệu ứng dòng chảy đồ thị liên tục không gián đoạn.

- Hiệu suất kết xuất (Rendering Performance): Việc vẽ biểu đồ diễn ra trực tiếp trên bề mặt điểm ảnh của HTML5 Canvas với hỗ trợ tăng tốc phần cứng. Điều này giúp loại bỏ tình trạng lag, đảm bảo mức sử dụng tài nguyên (CPU/RAM) thấp nhất, kể cả khi biểu đồ phải vẽ lại hàng chục khung hình mỗi giây do dữ liệu biến động liên tục.

2.4 Công nghệ Trí tuệ nhân tạo (AI) ứng dụng

Sự kết hợp giữa Internet vạn vật (IoT) và Trí tuệ nhân tạo (AI) - hay còn gọi là AIoT - đang là xu hướng tất yếu trong việc xây dựng các đô thị thông minh hiện đại. AI không chỉ đóng vai trò xử lý và phân tích lượng dữ liệu khổng lồ do các cảm biến thu thập được mà còn mang đến khả năng tự động hóa và tương tác thông minh vượt trội. Trong dự án này, hệ thống ứng dụng hai công nghệ AI tiên tiến: Edge AI cho nhận diện sinh trắc học và Cloud AI cho xử lý ngôn ngữ tự nhiên, nhằm nâng cao tính bảo mật và trải nghiệm quản lý.

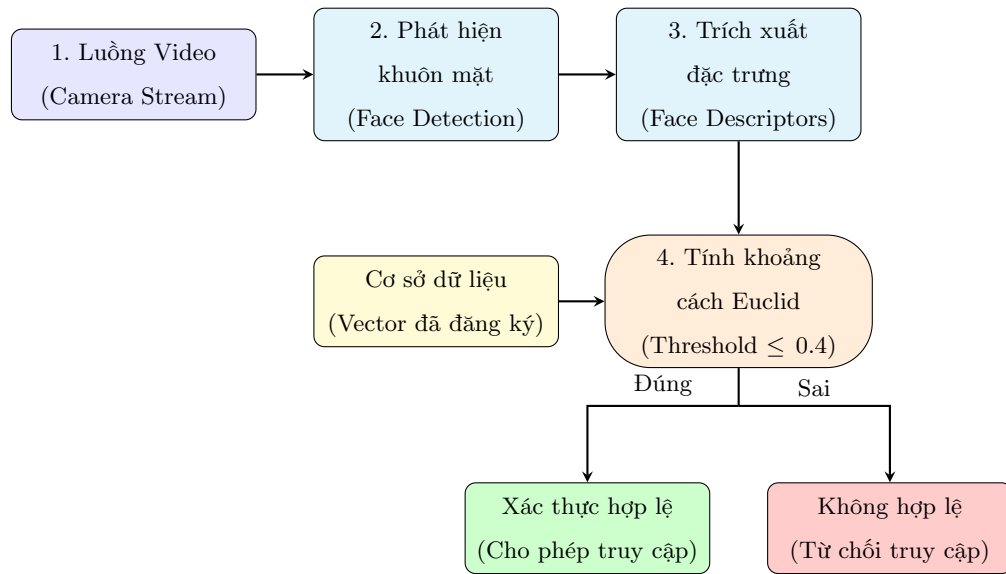
2.4.1 Nhận diện khuôn mặt với Face-api.js (Edge AI)

Thay vì sử dụng các phương thức xác thực truyền thống (như nhập mật khẩu) vốn tiềm ẩn nguy cơ bị lộ lọt và tốn thời gian thao tác, hệ thống tích hợp công nghệ nhận diện khuôn mặt trực tiếp trên trình duyệt web thông qua thư viện Face-api.js. Đây là một thư viện mã nguồn mở được phát triển dựa trên lõi TensorFlow.js, chuyên biệt cho các tác vụ thị giác máy tính phía máy khách (Client-side) [7].

Việc triển khai Face-api.js mang lại mô hình tính toán Edge AI (Trí tuệ nhân tạo biên) với nhiều ưu điểm vượt trội:

- Xử lý trực tiếp trên trình duyệt: Quá trình tải các mô hình nơ-ron tích chập (như SSD Mobilenet V1 cho việc phát hiện khuôn mặt và Face Landmark 68 cho việc đánh dấu các điểm đặc trưng) diễn ra hoàn toàn trên thiết bị của người dùng. Hệ thống trích xuất vector đặc trưng 128 chiều (Face Descriptors) ngay tại cục bộ, không cần gửi hình ảnh hay video luồng (stream) lên máy chủ, đảm bảo tối đa quyền riêng tư và bảo mật sinh trắc học.
- Khoảng cách Euclid (Euclidean Distance): Thuật toán so sánh đặc trưng khuôn mặt được thực hiện bằng cách tính toán khoảng cách Euclid giữa vector đặc trưng thu được từ camera thời gian thực và vector chuẩn đã đăng ký trước đó. Nếu khoảng cách này nhỏ hơn một ngưỡng nhất định (ngưỡng an toàn thường là 0.4 - 0.5), hệ thống sẽ xác nhận danh tính hợp lệ và cho phép đăng nhập vào Dashboard quản lý.
- Độ trễ thấp: Bỏ qua được khâu truyền tải hình ảnh dung lượng lớn qua mạng Internet giúp hệ thống phát hiện và nhận diện khuôn mặt diễn ra gần như tức thời ở tốc độ khung hình cao (FPS),

nâng cao trải nghiệm người dùng ngay cả khi băng thông mạng bị giới hạn.



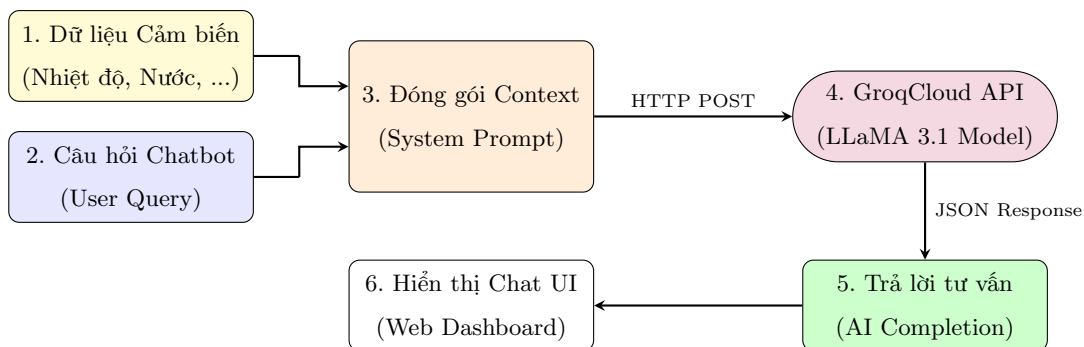
Hình 2.4.1: Cơ chế hoạt động của thuật toán nhận diện khuôn mặt dựa trên Face-api.js

2.4.2 Xử lý ngôn ngữ tự nhiên (LLM) với LLaMA 3.1 thông qua Groq API

Để hỗ trợ người quản trị điều phối và ra quyết định nhanh chóng trong các tình huống khẩn cấp, dự án tích hợp một "Trợ lý ảo Đô thị". Trợ lý ảo này được phát triển dựa trên Mô hình Ngôn ngữ Lớn (Large Language Model - LLM) tối tân LLaMA 3.1 của Meta.

Thay vì chạy trực tiếp mô hình khổng lồ này trên thiết bị nội bộ (vốn đòi hỏi sức mạnh phần cứng rất cao), hệ thống tương tác với LLaMA 3.1 thông qua Groq API - một đơn vị cung cấp hạ tầng xử lý AI cực kỳ mạnh mẽ sử dụng cấu trúc chip chuyên biệt LPU (Language Processing Unit) [8]. Kiến trúc này mang lại các đặc tính:

- **Suy luận tốc độ cao (High-speed Inference):** Nhờ vi kiến trúc LPU của Groq, các câu trả lời (completion) được mô hình sinh ra với tốc độ hàng trăm token mỗi giây, mang lại trải nghiệm đàm thoại mượt mà, phản hồi ngay lập tức như đang giao tiếp với người thật [8].
- **Cấu hình Prompt ngữ cảnh (Context-aware Prompting):** Trợ lý AI không chỉ đóng vai trò là một Chatbot thông thường. Thông qua JavaScript, toàn bộ trạng thái dữ liệu thời gian thực hiện tại của hệ thống (nhiệt độ, độ ẩm, mực nước hiện tại, trạng thái máy bơm và đèn) được tổng hợp và nhúng trực tiếp vào *System Prompt* trước khi gửi đi yêu cầu. Nhờ vậy, AI có khả năng hiểu rõ bối cảnh cụ thể của đô thị và đưa ra các tư vấn chính xác. Ví dụ: "*Mực nước đang ở mức 80%, hệ thống máy bơm đã kích hoạt. Bạn cần cử đội cứu hộ đến khu vực A*".
- **Kiến trúc API RESTful an toàn:** Việc tương tác được thực hiện qua phương thức POST gửi tới Endpoint của Groq API, trong đó dữ liệu được cấu trúc dưới dạng JSON chuẩn hoá.



Hình 2.4.2: Mô hình tích hợp dữ liệu cảm biến thời gian thực vào Trợ lý ảo AI thông qua Groq API

2.5 Trình giả lập điện tử Wokwi

2.5.1 Mô phỏng phần cứng và giao thức kết nối

Trong quá trình thiết kế hệ thống IoT phức tạp, việc phụ thuộc hoàn toàn vào các thành phần vật lý thường gây ra nhiều khó khăn như chi phí mua sắm linh kiện, thời gian chờ đợi giao hàng và nguy cơ cháy nổ do đấu nối sai mạch điện. Để giải quyết triệt để các rủi ro này, dự án lựa chọn sử dụng Wokwi – một nền tảng mô phỏng vi mạch điện tử trực tuyến thế hệ mới. Nền tảng này cho phép biên dịch và thực thi mã nguồn C++ trực tiếp trên trình duyệt web thông qua công nghệ WebAssembly.

Những đặc điểm nổi bật và lợi ích của việc ứng dụng Wokwi trong quá trình phát triển bao gồm:

- **Mô phỏng vi điều khiển ESP32 với độ chính xác cao:** Wokwi không chỉ mô phỏng tập lệnh lõi của vi xử lý Xtensa mà còn giả lập chính xác các khối ngoại vi (Peripherals) phần cứng. Nó cung cấp sự hỗ trợ toàn diện cho hệ thống ngắt (Interrupts), bộ đếm thời gian (Timers), và đầy đủ các chuẩn giao tiếp công nghiệp phổ biến như I2C, SPI, UART, giúp đảm bảo mã nguồn chạy trên mô phỏng sẽ hoạt động tương đương trên phần cứng thực.
- **Tương tác trực quan và thời gian thực:** Người dùng có thể thao tác trực tiếp với các linh kiện ảo như nhấn nút, xoay biến trở hoặc thay đổi nhiệt độ trên cảm biến DHT22 bằng chuột. Tín hiệu này lập tức được gửi đến chân GPIO của vi điều khiển ảo, giúp kiểm chứng giải thuật một cách sinh động mà không cần nạp code liên tục.
- **Cấu trúc linh kiện mã nguồn mở (JSON):** Hệ thống sơ đồ mạch điện được định nghĩa hoàn toàn thông qua một tệp tin dạng văn bản `diagram.json`. Định dạng này giúp khai báo tường minh vị trí tọa độ của từng linh kiện, màu sắc của dây nối và các thuộc tính điện học đi kèm. Việc sử dụng file văn bản giúp quá trình quản lý phiên bản (Version Control) bằng Git và chia sẻ dự án giữa các thành viên trong nhóm diễn ra vô cùng thuận tiện.

2.5.2 Tính năng Wokwi-GUEST Wi-Fi

Một trong những thách thức lớn nhất của các trình giả lập vi điều khiển truyền thống (như Proteus) là khả năng kết nối mạng Internet. Đặc biệt đối với các hệ thống IoT, việc thiết bị không thể kết nối mạng đồng nghĩa với việc không thể kiểm thử quá trình đồng bộ lên đám mây. Wokwi đã vượt qua rào

cần này nhờ vào tính năng Wokwi-GUEST Wi-Fi.

Vai trò và nguyên lý hoạt động của Wokwi-GUEST trong hệ thống giả lập:

- Thiết lập cổng kết nối ảo (Virtual Gateway): Wokwi-GUEST hoạt động tương tự như một điểm truy cập Wi-Fi (Access Point) mô phỏng. Khi ESP32 chạy lệnh `WiFi.begin("Wokwi-GUEST", "")`, trình duyệt web sẽ đóng vai trò như một bộ định tuyến (Router), cấp phát địa chỉ IP ảo cục bộ cho vi điều khiển và mở một kênh kết nối an toàn xuyên qua tường lửa để ra ngoài mạng Internet thực tế.
- Đàm phán bảo mật và SSL/TLS: Tính năng này cho phép vi điều khiển ảo thực hiện đầy đủ các bước bắt tay (handshake) để thiết lập chứng chỉ bảo mật. Nhờ vậy, ESP32 có thể giao tiếp an toàn với các dịch vụ yêu cầu mã hóa cao (HTTPS) mà không gặp lỗi từ chối kết nối.
- Đồng bộ dữ liệu WebSockets với Firebase: Đây là giá trị cốt lõi giúp hoàn thiện bài toán IoT. Thông qua kết nối Wokwi-GUEST, ESP32 giả lập duy trì thành công giao thức WebSockets liên tục với máy chủ Google Firebase. Bất kỳ sự thay đổi nào trên biến trở ảo ở màn hình mô phỏng đều được đẩy lên Firebase và phản hồi về Web Dashboard chỉ trong tích tắc, tạo ra trải nghiệm thời gian thực hoàn hảo không khác biệt so với mô hình sa bàn vật lý.

Chương 3

YÊU CẦU THIẾT KẾ VÀ ĐẶC TẢ PHẦN CỨNG

3.1 Yêu cầu thiết kế hệ thống

Trong quá trình xây dựng và phát triển hệ thống Giám sát và Điều khiển Đô thị Thông minh (Smart City IoT), việc xác định rõ ràng các yêu cầu thiết kế đóng vai trò cốt lõi, định hình toàn bộ kiến trúc phần cứng lẫn phần mềm. Dựa trên mục tiêu tổng quát của đề tài, hệ thống được thiết kế để giải quyết bài toán giám sát môi trường và tự động hóa các tiện ích công cộng trong khu vực đô thị. Các yêu cầu thiết kế được phân tách thành hai nhóm chính: yêu cầu chức năng và yêu cầu phi chức năng.

3.1.1 Yêu cầu chức năng

Yêu cầu chức năng mô tả chi tiết những hành động, tính năng mà hệ thống phải thực hiện được để đáp ứng nhu cầu của ban quản lý đô thị. Cụ thể, hệ thống phải đảm bảo các chức năng trọng tâm sau:

- **Chức năng thu thập dữ liệu môi trường và hạ tầng (Data Acquisition):** Hệ thống cần có khả năng đo lường, cập nhật liên tục và chính xác các thông số quan trọng của môi trường đô thị. Các cảm biến được phân bố tại các điểm nút (Edge nodes) phải thu thập những thông tin bao gồm:
 - *Nhiệt độ và độ ẩm:* Sử dụng cảm biến (như DHT22) để theo dõi điều kiện thời tiết thực tế, cảnh báo khi có sự thay đổi bất thường ảnh hưởng đến sinh hoạt của người dân.
 - *Chất lượng không khí (AQI - Air Quality Index):* Giám sát nồng độ bụi mịn và khí độc hại nhằm đưa ra cảnh báo kịp thời về mức độ ô nhiễm thông qua cảm biến hoặc các biến trở giả lập mô phỏng.
 - *Mức nước ngập lụt (Water Level):* Đo lường độ sâu của mực nước tại các vùng trũng thấp hoặc hệ thống cống thoát nước để điều phối trạm bơm kịp thời trong mùa mưa bão.
 - *Trạng thái bãi đỗ xe thông minh (Smart Parking):* Cập nhật số lượng chỗ trống hiện tại của các bãi đỗ xe trong đô thị, giúp tối ưu hóa luồng giao thông và giảm thiểu tình trạng

ùn tắc.

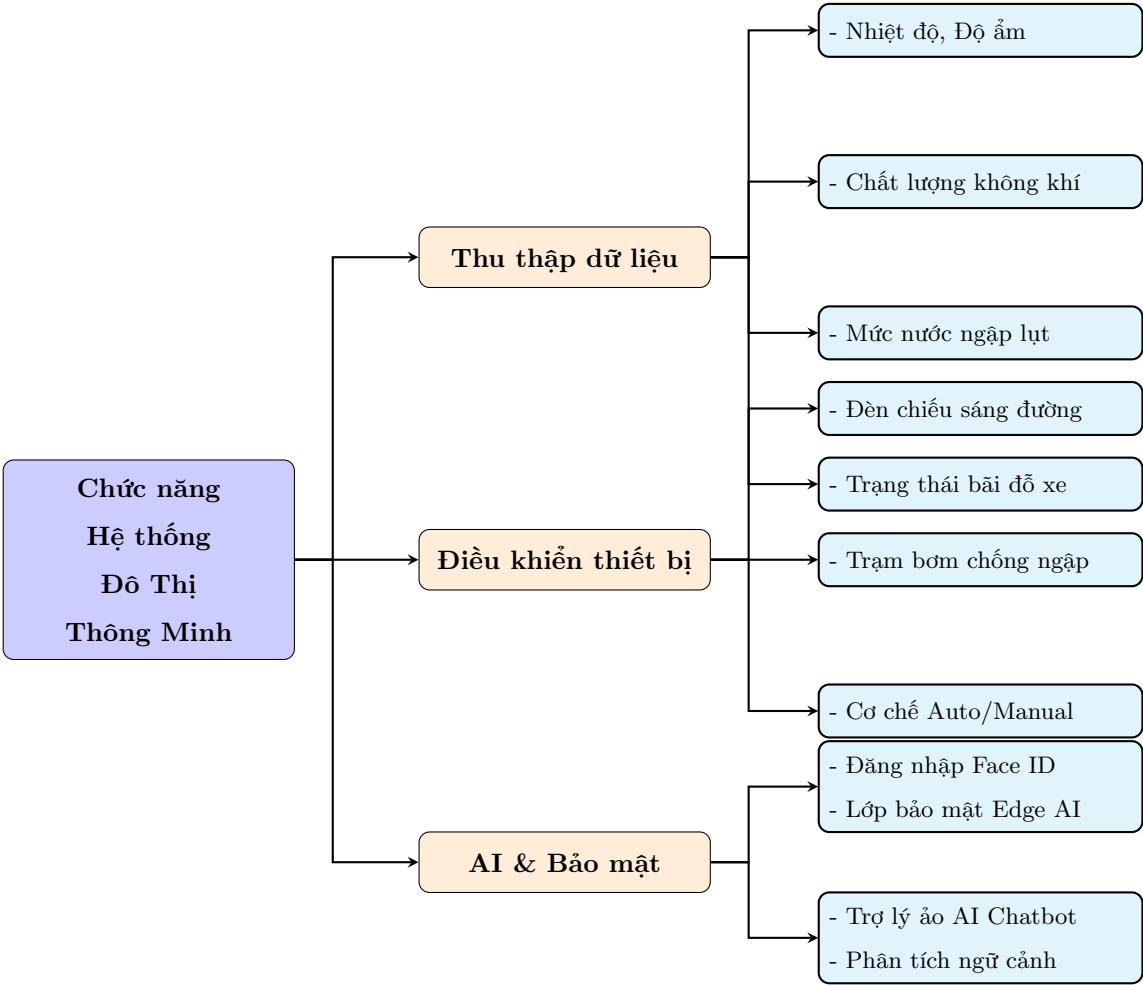
- **Chức năng điều khiển thiết bị chấp hành (Actuator Control):** Người quản lý phải có khả năng can thiệp và điều khiển trạng thái của các thiết bị ngoại vi thông qua trung tâm giám sát điều hành:
 - Điều khiển bật/tắt *Hệ thống đèn chiếu sáng đường phố (Street Light)*.
 - Điều khiển bật/tắt *Hệ thống máy bơm nước chống ngập (Water Pump)*.
- **Chế độ vận hành Tự động (Auto) và Thủ công (Manual):** Hệ thống cung cấp cơ chế chuyển đổi linh hoạt giữa hai chế độ vận hành để đảm bảo an toàn và tối ưu năng lượng.
 - *Chế độ Manual (Thủ công):* Người quản lý trực tiếp thao tác bật/tắt thiết bị thông qua giao diện ứng dụng Web Dashboard.
 - *Chế độ Auto (Tự động):* Vi điều khiển trung tâm (ESP32) tự động đưa ra quyết định dựa trên các ngưỡng giá trị đã được thiết lập trước. Cụ thể: Mức nước vượt ngưỡng 70% thì hệ thống máy bơm tự động kích hoạt; cường độ ánh sáng yếu dưới 40% thì hệ thống đèn đường tự động bật sáng.
- **Bảo mật hệ thống đa lớp (Multi-layer Security):** Đối với hệ thống quản lý hạ tầng đô thị, an toàn thông tin là ưu tiên hàng đầu. Giao diện Web Dashboard phải được bảo vệ nghiêm ngặt để ngăn chặn các truy cập trái phép. Đề tài tích hợp công nghệ sinh trắc học nhận diện khuôn mặt (Face ID) tiên tiến hoạt động tại biên (Edge AI) làm lớp bảo mật chính, yêu cầu người dùng phải xác thực chính xác danh tính mới được phép truy cập vào trang điều khiển trung tâm [7].
- **Tích hợp Trợ lý ảo AI thông minh (AI Assistant):** Hệ thống cung cấp tính năng hỗ trợ ra quyết định thông qua một Trợ lý ảo AI được xây dựng dựa trên Mô hình Ngôn ngữ Lớn (LLM). AI Chatbot này có khả năng thu thập ngữ cảnh (context), phân tích dữ liệu môi trường hiện thời và trả lời các câu hỏi, đưa ra lời khuyên chuyên sâu và dự báo cho người quản lý đô thị [8].

3.1.2 Yêu cầu phi chức năng

Bên cạnh các yêu cầu về mặt tính năng hoạt động, hệ thống cũng phải đáp ứng các tiêu chuẩn khắt khe về hiệu năng, tính khả dụng và trải nghiệm người dùng, đảm bảo khả năng ứng dụng lâu dài trong thực tế.

- **Khả năng hoạt động thời gian thực (Real-time Reliability):** Do tính chất cấp bách của các sự cố đô thị (ví dụ: ngập lụt chớp nhoáng, kẹt xe), hệ thống yêu cầu độ trễ (latency) khi truyền tải dữ liệu từ cảm biến lên Web Dashboard và khi gửi lệnh điều khiển từ Web xuống thiết bị phải cực kỳ thấp, lý tưởng là dưới 1 giây ($< 1s$). Việc này được hiện thực hóa thông qua cơ chế đồng bộ WebSockets hiệu năng cao của nền tảng Firebase Realtime Database [1].
- **Độ ổn định và khả năng vận hành bền bỉ (24/7):** Cả phần cứng vi điều khiển (ESP32) lẫn nền tảng Web Backend đều phải có khả năng xử lý đa nhiệm, hoạt động cực kỳ ổn định và liên tục 24 giờ mỗi ngày, 7 ngày mỗi tuần. Cụ thể, vi điều khiển phải tránh được tình trạng "treo" (blocking) trong quá trình chờ dữ liệu, sử dụng giải thuật định thời phi đồng bộ linh hoạt (thông qua hàm `millis()`) thay vì dùng hàm `delay()` làm gián đoạn luồng xử lý chính [9].

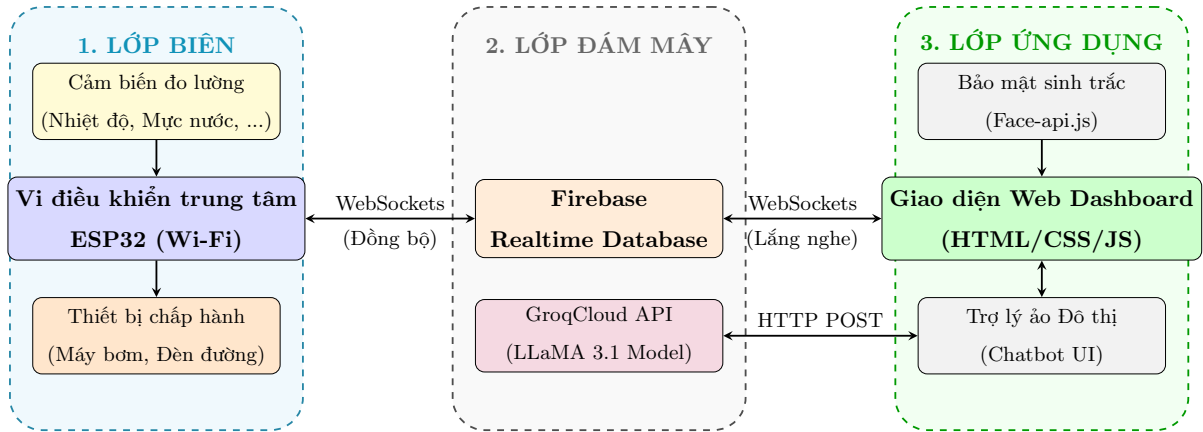
- **Giao diện người dùng và Trải nghiệm người dùng (UI/UX):** Bảng điều khiển (Web Dashboard) cần được thiết kế mang tính chuyên nghiệp, trực quan và hiện đại nhất. Giao diện phải tuân thủ chuẩn thiết kế tương thích (Responsive Web Design), đảm bảo hiển thị hoàn hảo và dễ dàng thao tác trên nhiều kích thước màn hình khác nhau từ Máy tính để bàn (Desktop), Máy tính bảng (Tablet) đến Điện thoại thông minh (Mobile) [5]. Ngoài ra, thông tin dữ liệu không chỉ ở dạng số mà phải được trực quan hóa thông qua các biểu đồ (Chart), giúp người dùng dễ dàng theo dõi xu hướng, tra cứu lịch sử đo đạc và đưa ra quyết định vận hành chính xác.



Hình 3.1.1: Sơ đồ phân rã chức năng của hệ thống

3.2 Sơ đồ đặc tả kiến trúc hệ thống (Block Diagram)

Để hiện thực hóa các yêu cầu chức năng và phi chức năng đã đề ra, hệ thống Giám sát và Điều khiển Đô thị Thông minh được thiết kế dựa trên kiến trúc IoT ba lớp (3-Tier Architecture) kinh điển. Kiến trúc này đảm bảo luồng dữ liệu được luân chuyển một cách thông suốt, bảo mật và cực kỳ tối ưu về mặt hiệu năng xử lý. Tổng thể kiến trúc được chia thành ba khối chính: Khối Edge (Thu thập dữ liệu tại biên), Khối Cloud (Trung tâm đồng bộ và lưu trữ) và Khối Application (Giao diện tương tác và xử lý AI).



Hình 3.2.1: Sơ đồ khối đặc tả kiến trúc tổng quát của hệ thống Smart City

3.2.1 Khối Edge (Thiết bị IoT và Cảm biến)

Khối Edge, hay còn gọi là Lớp Biên (Edge Node/Perception Layer), là thành phần giao tiếp trực tiếp với môi trường vật lý. Đây là điểm khởi đầu của mọi luồng dữ liệu trong hệ thống, chịu trách nhiệm số hóa các thông tin từ thế giới thực.

- Cảm biến đo lường (Sensors):** Được phân bố rộng khắp các nút giao thông và khu dân cư giả lập. Thành phần này liên tục trích xuất các thông số vật lý như:
 - Tín hiệu Analog từ các biến trở giả lập đóng vai trò giám sát mức nước ngập (Water Level), chất lượng không khí (AQI) và cường độ ánh sáng môi trường.
 - Tín hiệu Digital chuẩn giao tiếp 1-Wire từ cảm biến DHT22 để đo nhiệt độ và độ ẩm.
 - Tín hiệu trạng thái logic (High/Low) từ hệ thống nút nhấn đóng vai trò là cảm biến hồng ngoại tại cổng vào/ra bãi đỗ xe.
- Thiết bị chấp hành (Actuators):** Nhận tín hiệu điều khiển trực tiếp từ vi xử lý trung tâm để thực thi các tác vụ vật lý, bao gồm hệ thống Relay kích hoạt máy bơm nước và các Module LED mô phỏng đèn chiếu sáng đô thị.
- Khối xử lý trung tâm (ESP32 Microcontroller):** Đóng vai trò là "bộ não" của Lớp Biên. ESP32 thực hiện vòng lặp `loop()` liên tục (sử dụng hàm `millis()` để tránh nghẽn cổ chai) nhằm đọc dữ liệu từ toàn bộ các cảm biến. Sau đó, nó áp dụng giải thuật tiền xử lý (như ánh xạ giá trị Analog thành %) và đưa ra quyết định bật/tắt thiết bị chấp hành tức thời nếu đang ở Chế độ Tự động (Auto Mode). Song song với đó, ESP32 thiết lập kết nối Wi-Fi ổn định để đóng gói toàn bộ dữ liệu vừa xử lý truyền lên đám mây.

3.2.2 Khối Cloud (Lưu trữ và Đồng bộ - Firebase)

Để liên kết giữa phần cứng thu thập dữ liệu ở Lớp Biên và người quản trị ở Lớp Ứng dụng, hệ thống đòi hỏi một cơ sở hạ tầng mạng mạnh mẽ, có khả năng xử lý hàng ngàn kết nối đồng thời với độ trễ thấp nhất. Khối Cloud (Platform Layer) được đảm nhận hoàn toàn bởi nền tảng **Google Firebase Realtime Database**.

- **Lưu trữ dữ liệu dạng JSON Tree:** Firebase không sử dụng kiến trúc bảng (Table) quan hệ rập khuôn mà tổ chức dữ liệu dưới dạng cấu trúc cây JSON động. Các trường dữ liệu được phân nhánh khoa học thành `/sensors` (trạng thái cảm biến), `/devices` (trạng thái đèn, bơm) và `/modes` (thiết lập Auto/Manual). Cấu trúc này giúp tối ưu hóa băng thông truy vấn, chỉ tải về những nhánh dữ liệu có sự thay đổi.
- **Đồng bộ hóa thời gian thực (Real-time Synchronization):** Sử dụng công nghệ kết nối WebSockets hai chiều, Firebase tạo ra một "đường ống" truyền dữ liệu liên tục. Khi ESP32 cập nhật một giá trị nhiệt độ mới, Firebase lập tức nhận diện sự biến động và tự động "đẩy" (Push) dữ liệu này xuống tất cả các giao diện Web đang kết nối với độ trễ thường xuyên nằm dưới ngưỡng 1 giây.
- **Tính sẵn sàng và khôi phục sự cố:** Dữ liệu đô thị luôn được sao lưu an toàn trên đám mây của Google. Trong trường hợp vi điều khiển ESP32 bị mất điện hoặc đứt kết nối mạng, khi có mạng trở lại, nó có thể lập tức tải về trạng thái điều khiển (như việc máy bơm cần đang bật hay tắt) từ Firebase để đồng bộ lại quá trình vận hành, tránh gián đoạn dịch vụ công cộng.

3.2.3 Khối Application (Web Dashboard và Trí tuệ nhân tạo)

Khối Application (Lớp Ứng dụng/Presentation Layer) là điểm chạm duy nhất của các nhà quản lý đô thị. Thay vì thiết kế các ứng dụng cài đặt phức tạp, hệ thống được xây dựng dưới dạng Ứng dụng Web (Web App) để đảm bảo tính tiện dụng, đa nền tảng và dễ dàng nâng cấp.

- **Giao diện Giám sát và Điều khiển (Web Dashboard):** Giao diện được phát triển dựa trên ngôn ngữ HTML5, CSS3 và Vanilla JavaScript. Các số liệu từ Firebase được "lắng nghe" và cập nhật thẳng vào Cấu trúc đối tượng tài liệu (DOM), giúp màn hình biến thiên theo thời gian thực. Bên cạnh đó, hệ thống tích hợp biểu đồ đồ thị trực quan (Chart.js) cho phép theo dõi lịch sử biến động môi trường theo dạng dòng thời gian, giúp nhà quản lý dễ dàng nắm bắt bức tranh toàn cảnh của đô thị.
- **Phân hệ Bảo mật Nhận diện khuôn mặt (Edge AI - Face ID):** Là lớp bảo vệ đầu tiên trước khi truy cập vào hệ thống. Sử dụng thư viện *Face-api.js*, module này lấy luồng video (stream) từ camera thiết bị truy cập, tiến hành phát hiện khuôn mặt và trích xuất vector đặc trưng 128 chiều ngay tại trình duyệt. Thông qua việc tính toán khoảng cách Euclid với dữ liệu gốc, hệ thống đảm bảo chỉ những người có thẩm quyền mới được cấp phép điều khiển hạ tầng. Quá trình xử lý cục bộ (Edge computing) này giúp bảo vệ tuyệt đối hình ảnh riêng tư của người đăng nhập.
- **Phân hệ Trợ lý AI Đô thị (Cloud AI - LLaMA 3.1):** Để nâng cao năng lực ra quyết định, giao diện Web tích hợp tính năng Trợ lý ảo AI thông qua Groq API. Trước khi người dùng gửi một câu hỏi (Ví dụ: "Tôi nên làm gì khi mực nước cao?"), JavaScript sẽ tự động đóng gói toàn bộ trạng thái thực tế của đô thị (nhiệt độ hiện hành, tình trạng bãi xe, chế độ hoạt động của bơm) vào một *System Prompt*. Mô hình Ngôn ngữ Lớn LLaMA 3.1 sau khi phân tích chuỗi bối cảnh này sẽ trả về những tư vấn cực kỳ sát thực tế và sắc bén với tốc độ suy luận siêu tốc.

3.3 Thiết kế sơ đồ kết nối phần cứng

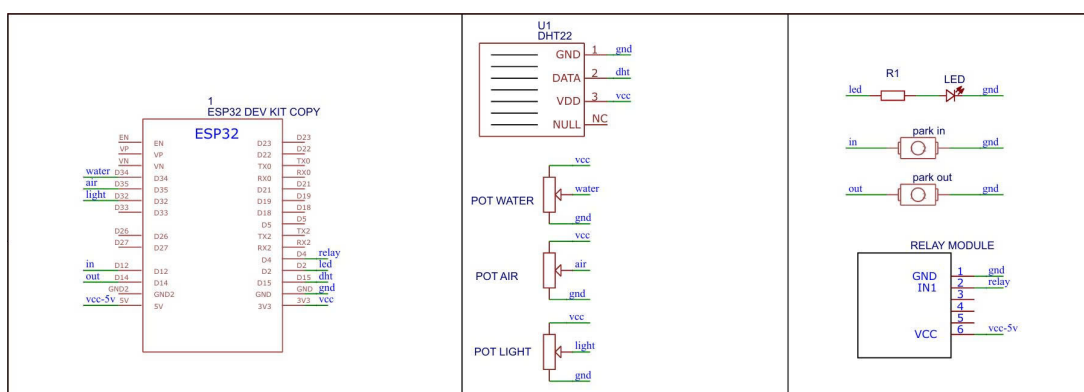
Phần cứng là nền tảng cốt lõi của bất kỳ hệ thống IoT nào. Trong dự án này, do tính phức tạp của việc triển khai thực tế một mô hình đô thị lớn, toàn bộ mạch điện được thiết kế và mô phỏng trên nền tảng Wokwi. Việc thiết kế sơ đồ kết nối đòi hỏi sự tính toán cẩn thận về phân bổ chân tín hiệu (GPIO), đảm bảo không xảy ra xung đột tài nguyên giữa các thiết bị ngoại vi và tối ưu hóa khả năng truyền tải dữ liệu.

3.3.1 Sơ đồ nguyên lý (Schematic) trên Wokwi

Sơ đồ nguyên lý thể hiện bức tranh toàn cảnh về cách thức vi điều khiển trung tâm ESP32 giao tiếp với các cảm biến và thiết bị chấp hành. Mạch điện được thiết kế tập trung vào sự tối giản hóa các thành phần không cần thiết, đảm bảo tính ổn định cao nhất khi chạy mô phỏng.

Các đặc điểm chính của sơ đồ nguyên lý:

- **Khối Nguồn (Power Supply):** Vi điều khiển ESP32 và toàn bộ các cảm biến sử dụng chung nguồn điện giả lập 3.3V và cực âm chung (GND). Việc quy hoạch chung một đường GND là nguyên tắc thiết kế mạch bắt buộc để tín hiệu điện thể có chung một điểm gốc (reference point) khi đo lường các giá trị Analog.
- **Giao tiếp Kỹ thuật số (Digital I/O):** Các linh kiện có tính chất nhị phân như Relay (điều khiển máy bơm), đèn LED đường, và các nút nhấn giả lập hàng rào hồng ngoại bãi đỗ xe đều được kết nối trực tiếp với các chân Digital của ESP32 thông qua điện trở nội.
- **Giao tiếp Tương tự (Analog Input):** Hệ thống ba biến trở trượt (Slide Potentiometers) đóng vai trò làm cảm biến mực nước, cảm biến ánh sáng và cảm biến chất lượng không khí. Một đầu của biến trở nối với 3.3V, đầu còn lại nối GND, và chân giữa (Wiper) được kết nối thẳng vào các bộ chuyển đổi ADC chuyên dụng của ESP32.



Hình 3.3.1: Sơ đồ kết nối linh kiện phần cứng mô phỏng trên nền tảng Wokwi

3.3.2 Phân công chân tín hiệu (GPIO) của ESP32

ESP32 là một dòng vi điều khiển mạnh mẽ, tuy nhiên không phải chân GPIO nào cũng có thể sử dụng tùy ý. Một số chân được dùng riêng cho việc nạp code (như GPIO 0, 2), một số chân chỉ có khả

năng nhận tín hiệu đầu vào (như GPIO 34, 35, 36, 39). Do đó, việc quy hoạch và phân công chân tín hiệu (Pinout assignment) phải tuân thủ nghiêm ngặt theo Datasheet của nhà sản xuất [9].

Bảng dưới đây trình bày chi tiết việc cấu hình chân vật lý cho từng linh kiện trong hệ thống Đô thị Thông minh:

Bảng 3.3.1: Bảng phân công chân tín hiệu (GPIO) của ESP32 cho các thiết bị

Chân GPIO	Thiết bị / Linh kiện	Kiểu I/O	Chức năng và Giải thích
GPIO 15	Cảm biến DHT22	Digital In	Giao tiếp 1-Wire để đọc tín hiệu Nhiệt độ và Độ ẩm.
GPIO 34	Cảm biến Mức nước (Biến trở)	Analog In	Sử dụng ADC1 (kênh 6) để đọc mức điện áp đo ngập lụt. Chân này chỉ hỗ trợ Input.
GPIO 35	Cảm biến Không khí (Biến trở)	Analog In	Sử dụng ADC1 (kênh 7) để lấy mẫu môi trường chỉ số bụi mịn (AQI).
GPIO 32	Cảm biến Ánh sáng (Biến trở)	Analog In	Sử dụng ADC1 (kênh 4) để đo độ rọi sáng môi trường.
GPIO 12	Nút nhấn (Xe vào bãi)	Digital In	Kích hoạt điện trở kéo lên nội bộ (Internal Pullup), đọc mức Thấp (LOW) khi nhấn.
GPIO 14	Nút nhấn (Xe ra bãi)	Digital In	Hoạt động tương tự chân GPIO 12 để làm bộ đếm lùi cho bãi đỗ xe.
GPIO 2	Đèn đường (LED)	Digital Out	Điều khiển bật/tắt đèn chiếu sáng đô thị ở chế độ Auto/Manual.
GPIO 4	Máy bơm nước (Relay)	Digital Out	Kích hoạt Relay để đóng/mở mạch máy bơm tiêu úng nước.

Việc sử dụng nhóm chân ADC1 (GPIO 32, 34, 35) cho các cảm biến tương tự là một thiết kế có chủ ý. Vì vi điều khiển ESP32 sử dụng module Wi-Fi, bộ chuyển đổi ADC2 sẽ bị vô hiệu hóa khi giao thức mạng vô tuyến được kích hoạt. Nhờ sử dụng nhóm chân ADC1, hệ thống hoàn toàn loại bỏ được xung đột phần cứng, đảm bảo luồng đọc dữ liệu từ biến trở luôn trơn tru ngay cả khi đang đồng bộ dữ liệu lên Firebase [9].

Chương 4

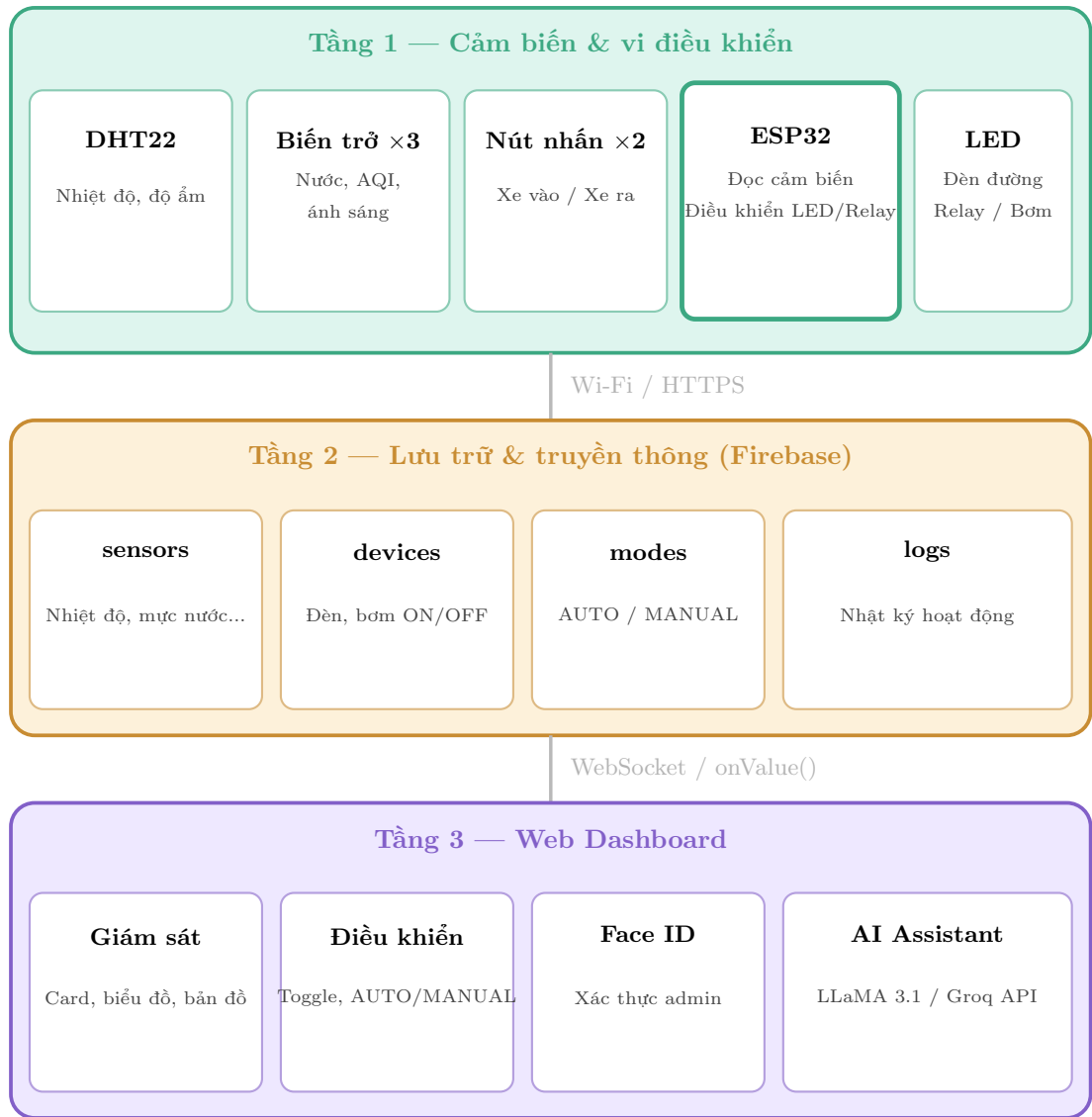
GIẢI THÍCH MÃ NGUỒN

Chương này trình bày chi tiết toàn bộ mã nguồn của hệ thống Smart City IoT, bao gồm phần nhúng chạy trên vi điều khiển ESP32 được mô phỏng trên nền tảng Wokwi, và phần giao diện web dashboard được xây dựng bằng HTML, CSS và JavaScript thuần. Mỗi module chức năng sẽ được phân tích cụ thể về mục đích thiết kế, luồng hoạt động và mối quan hệ với các thành phần còn lại trong hệ thống.

4.1 Tổng quan kiến trúc mã nguồn

Toàn bộ dự án được tổ chức theo mô hình ba tầng rõ ràng: tầng cảm biến và vi điều khiển (ESP32 firmware), tầng lưu trữ và truyền thông dữ liệu trung gian (Firebase Realtime Database), và tầng hiển thị cùng điều khiển (Web Dashboard). Ba tầng này phối hợp với nhau thông qua giao tiếp thời gian thực dựa trên WebSocket mà Firebase cung cấp.

Mô hình ba tầng được áp dụng trong dự án không chỉ đơn thuần là sự phân chia về mặt vật lý mà còn là một chiến lược thiết kế nhằm tối ưu hóa khả năng xử lý song song và tính toàn vẹn của dữ liệu trong hệ thống đô thị thông minh. Cấu trúc này cho phép tách biệt rõ rệt giữa logic vận hành tại biên (edge), quản lý dữ liệu trung tâm và giao diện tương tác người dùng, tạo điều kiện thuận lợi cho việc mở rộng hoặc bảo trì từng thành phần độc lập mà không gây ảnh hưởng đến toàn bộ hệ thống.



Hình 4.1.1: Sơ đồ kiến trúc ba tầng của hệ thống Smart City IoT

Tầng 1 — Cảm biến và vi điều khiển Đây là tầng giao tiếp trực tiếp với môi trường vật lý, chịu trách nhiệm thu thập tín hiệu và thực thi các lệnh vật lý. Trái tim của tầng này là vi điều khiển ESP32, đảm nhiệm hai chức năng cốt lõi: đọc dữ liệu từ cảm biến và xuất tín hiệu điều khiển trực tiếp đến các cơ cấu chấp hành.

- **Khối thu thập dữ liệu:** Bao gồm cảm biến DHT22 (đo nhiệt độ, độ ẩm môi trường), cụm 3 biến trở (mô phỏng sự thay đổi của mực nước, chỉ số chất lượng không khí AQI, và cường độ ánh sáng), cùng với 2 nút nhấn (mô phỏng sự kiện xe vào và xe ra khỏi bãi đỗ).
- **Khối chấp hành:** Nhận tín hiệu từ ESP32 để đóng/cắt phần cứng, bao gồm LED (đóng vai trò là đèn đường) và module Relay (điều khiển máy bơm chống ngập).
- **Phương thức kết nối:** Vi điều khiển ESP32 kết nối với tầng đám mây thông qua giao thức Wi-Fi/HTTPS để truyền tải gói tin dữ liệu cảm biến và nhận lệnh điều khiển.

Tầng 2 — Lưu trữ và truyền thông Sử dụng nền tảng Firebase Realtime Database làm trung tâm trung chuyển và lưu trữ dữ liệu của toàn bộ hệ thống. Cơ sở dữ liệu NoSQL được tổ chức phân cấp logic

thành 4 nhánh (nodes) chính để tối ưu hóa việc truy xuất:

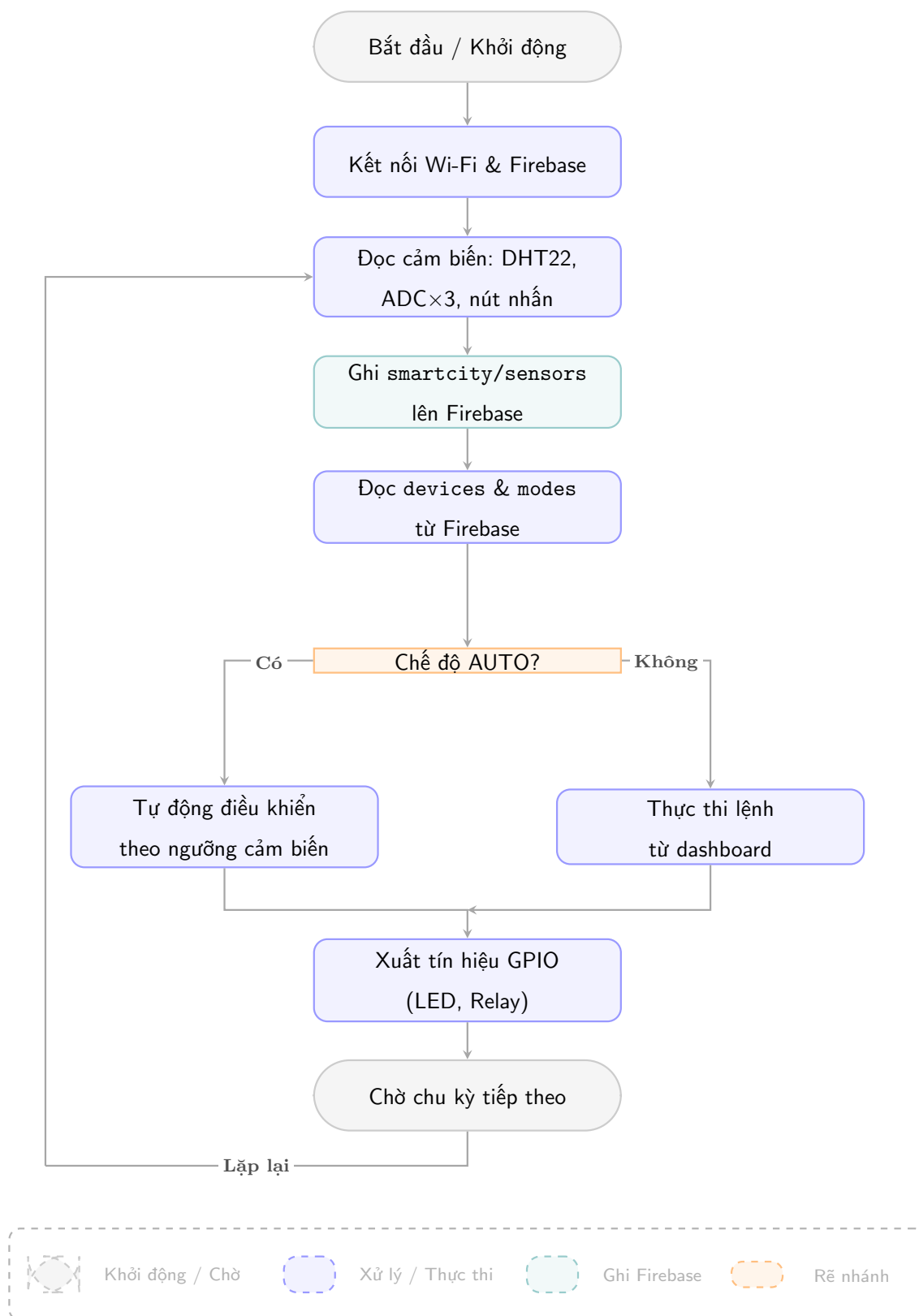
- **sensors:** Nhánh lưu trữ các giá trị môi trường theo thời gian thực (nhiệt độ, độ ẩm, mực nước, v.v.).
- **devices:** Nhánh lưu trữ trạng thái hoạt động hiện tại (ON/OFF) của các thiết bị chấp hành như đèn và bơm.
- **modes:** Nhánh quản lý chế độ vận hành của hệ thống (AUTO hoặc MANUAL).
- **logs:** Nhánh ghi nhận và lưu trữ lịch sử hoạt động, các thao tác của quản trị viên.

Tầng 3 — Web Dashboard Tầng giao diện người dùng được kết nối trực tiếp với Firebase thông qua giao thức WebSocket (sử dụng hàm `onValue()`). Cơ chế này đảm bảo dữ liệu được đồng bộ (push) theo thời gian thực mà không cần làm mới trang. Tầng ứng dụng bao gồm 4 module chức năng chính:

- **Giám sát:** Trực quan hóa dữ liệu từ **sensors** thông qua các thẻ thông tin (Card), biểu đồ lịch sử theo thời gian và bản đồ đô thị tương tác.
- **Điều khiển:** Cung cấp giao diện công tắc (Toggle) và tùy chọn chuyển đổi chế độ (AUTO/MANUAL) để can thiệp vào hoạt động của thiết bị.
- **Face ID:** Tích hợp module xử lý thị giác máy tính phía client để xác thực sinh trắc học, cấp quyền truy cập admin.
- **AI Assistant:** Trợ lý ảo thông minh có khả năng nhận thức ngữ cảnh đô thị, được cấp nguồn sức mạnh từ mô hình ngôn ngữ lớn LLaMA 3.1 thông qua Groq API.

Cấu trúc mã nguồn phía client được tổ chức gồm ba tệp chính: **index.html** định nghĩa bố cục giao diện, **styles.css** xử lý toàn bộ phần trình bày thẩm mỹ và responsive, và **script.js** đảm nhiệm toàn bộ logic nghiệp vụ của ứng dụng. Phía phần cứng, cấu hình mạch điện tử được định nghĩa trong tệp **diagram.json** dùng cho trình mô phỏng Wokwi, và tệp **wokwi.toml** chứa đường dẫn đến firmware biên dịch từ PlatformIO.

Hình 4.1.2 mô tả lưu đồ thuật toán vòng lặp chính của firmware ESP32, thể hiện quy trình đọc cảm biến, ghi dữ liệu lên Firebase và nhận lệnh điều khiển thiết bị theo chu kỳ liên tục.



Hình 4.1.2: Lưu đồ thuật toán vòng lặp chính của firmware ESP32

4.2 Cấu hình và khởi tạo Firebase

Phần đầu của tệp `script.js` thực hiện việc nhập khẩu (import) các module Firebase cần thiết và thiết lập kết nối đến project Firebase của nhóm.

4.2.1 Nhập khẩu module Firebase

Hệ thống sử dụng Firebase SDK phiên bản 12.12.0 theo dạng ES Module, được tải trực tiếp từ CDN của Google.

Ba nhóm chức năng Firebase được sử dụng là:

- **firebase-app**: Hàm `initializeApp` để khởi tạo ứng dụng Firebase với cấu hình được cung cấp.
- **firebase-auth**: Các hàm `getAuth`, `onAuthStateChanged`, `signInWithEmailAndPassword`, `signOut` phục vụ hệ thống xác thực người dùng.
- **firebase-database**: Các hàm `getDatabase`, `onValue`, `push`, `ref`, `set` phục vụ đọc và ghi dữ liệu thời gian thực.



```

1 import { initializeApp } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-app.js';
2 import { getAuth, onAuthStateChanged, signInWithEmailAndPassword, signOut } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-auth.js';
3 import { getDatabase, onValue, push, ref, set } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-database.js';
4
5 const firebaseConfig = {
6   apiKey: 'AIzaSyCAU6PA-b08... Đã che bảo mật',
7   authDomain: 'iot-smartcity-full.firebaseio.com',
8   databaseURL: 'https://iot-smartcity-full-default-rtdb.asia-southeast1.firebaseio.com',
9   projectId: 'iot-smartcity-full',
10  storageBucket: 'iot-smartcity-full.firebaseio.com',
11  messagingSenderId: '989917606860',
12  appId: '1:989917606860:web:464984fddcdc1e7b0c2369',
13  measurementId: 'G-LMQ5TXWQ18'
14 };
15
16 const groqConfig = {
17   apiKey: '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',
18   model: 'llama-3.1-70b-versatile'
19 };
20
21 const app = initializeApp(firebaseConfig);
22 const db = getDatabase(app);
23 const auth = getAuth(app);

```

Hình 4.2.1: Nhập khẩu module và cấu hình Firebase

4.2.2 Đối tượng cấu hình Firebase

Đối tượng `firebaseConfig` chứa toàn bộ thông tin định danh project Firebase:

- **apiKey**: Khóa API xác thực ứng dụng với Firebase.
- **authDomain**: Miền xác thực là `iot-smartcity-full.firebaseio.com`.
- **databaseURL**: Địa chỉ Realtime Database đặt tại khu vực `asia-southeast1` (Singapore) để đảm bảo độ trễ thấp cho người dùng tại Việt Nam.
- **projectId**, **storageBucket**, **messagingSenderId**, **appId**, **measurementId**: Các định danh bổ sung của project.

Sau khi khởi tạo ứng dụng bằng `initializeApp(firebaseConfig)`, ba đối tượng dịch vụ `db`, `auth` được tạo ra từ hàm `getDatabase(app)` và `getAuth(app)` tương ứng, dùng xuyên suốt toàn bộ ứng dụng.

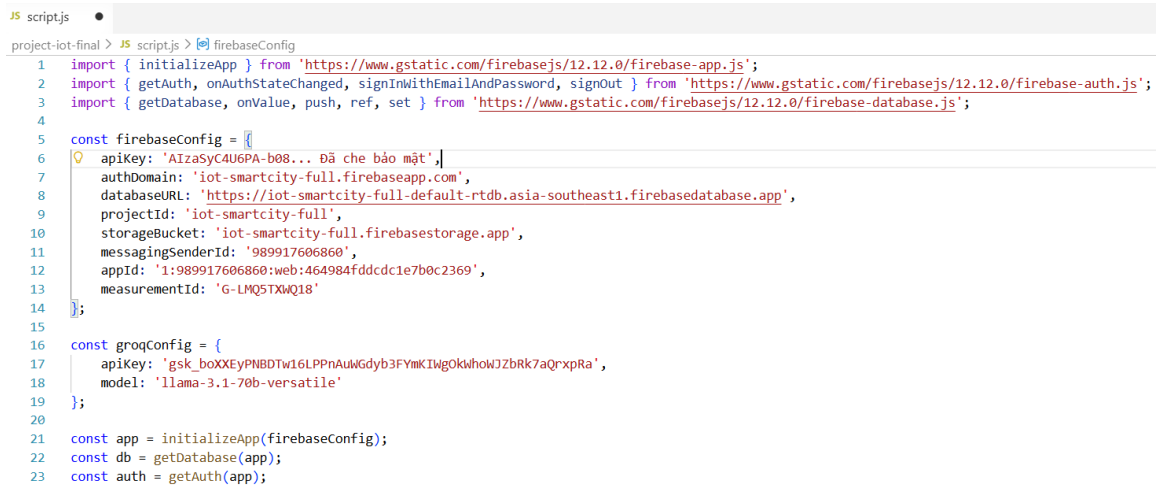
4.2.3 Cấu hình Groq AI

Bên cạnh Firebase, đối tượng `groqConfig` lưu trữ thông tin kết nối đến API Groq để tích hợp mô hình ngôn ngữ lớn LLaMA 3.1:

```
const groqConfig = {
  apiKey: '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',
  model: 'llama-3.1-70b-versatile'
};
```

Hình 4.2.2: Cấu hình Groq AI

- **apiKey:** Khóa xác thực API Groq.
- **model:** Tên mô hình được chọn là `llama-3.1-70b-versatile`, phiên bản 70 tỷ tham số chạy trên hạ tầng suy luận tốc độ cao của Groq.



```
JS script.js
project-iot-final > JS script.js > firebaseConfig
1 import { initializeApp } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-app.js';
2 import { getAuth, onAuthStateChanged, signInWithEmailAndPassword, signOut } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-auth.js';
3 import { getDatabase, onValue, push, ref, set } from 'https://www.gstatic.com/firebasejs/12.12.0/firebase-database.js';
4
5 const firebaseConfig = {
6   apiKey: 'AizaSyC4U6PA-b08... Đã che bảo mật',
7   authDomain: 'iot-smartcity-full.firebaseio.com',
8   databaseURL: 'https://iot-smartcity-full-default-rtdb.asia-southeast1.firebaseio.com',
9   projectId: 'iot-smartcity-full',
10  storageBucket: 'iot-smartcity-full.firebaseio.com',
11  messagingSenderId: '989917606860',
12  appId: '1:989917606860:web:464984fddcdce7b0c2369',
13  measurementId: 'G-LMQ5TXWQ18'
14 };
15
16 const groqConfig = {
17   apiKey: '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',
18   model: 'llama-3.1-70b-versatile'
19 };
20
21 const app = initializeApp(firebaseConfig);
22 const db = getDatabase(app);
23 const auth = getAuth(app);
```

Hình 4.2.3: Khởi tạo Firebase SDK và cấu hình Groq AI trong mã nguồn

Như được minh họa trong Hình 4.2.3, dự án khai báo rõ ràng các thông số kết nối đến Firebase và Groq API. (Lưu ý: Trong môi trường triển khai thực tế, các khóa bảo mật API Key cần được che giấu hoặc quản lý qua biến môi trường để đảm bảo an toàn thông tin).

4.3 Quản lý trạng thái ứng dụng

Toàn bộ trạng thái runtime của ứng dụng được tổ chức thành ba đối tượng state riêng biệt, mỗi đối tượng phụ trách một lĩnh vực độc lập.

4.3.1 dashboardState

Đây là đối tượng trung tâm lưu trữ dữ liệu đồng bộ mới nhất từ Firebase, bao gồm ba nhóm con: `sensors` (dữ liệu cảm biến), `devices` (trạng thái thiết bị), `modes` (chế độ vận hành AUTO/MANUAL).

Mỗi khi Firebase cập nhật dữ liệu mới qua listener `onValue`, `dashboardState` sẽ được cập nhật tương ứng, sau đó hàm `updateDashboard()` được gọi để đồng bộ giao diện người dùng.

4.3.2 Đối tượng `assistantState`

Trong kiến trúc của hệ thống, `assistantState` đóng vai trò là trung tâm quản lý toàn bộ vòng đời và ngữ cảnh tương tác của trợ lý AI. Thay vì để trợ lý hoạt động một cách độc lập và thiếu trạng thái (stateless), đối tượng này được thiết kế để duy trì nhận thức liên tục của AI về môi trường xung quanh và người dùng. Cấu trúc của `assistantState` được chia thành hai thành phần cốt lõi:

- **`lastSnapshot` (Bản sao dữ liệu thời gian thực):** Đây là một bộ nhớ đệm (cache) phía client, liên tục lưu trữ bản sao mới nhất của tất cả các giá trị cảm biến (`sensors`) và trạng thái thiết bị (`devices`). Mục đích thiết kế của `lastSnapshot` là tối ưu hóa tối đa độ trễ (latency). Khi người dùng đặt câu hỏi, hệ thống sẽ trích xuất ngay lập tức dữ liệu từ bộ nhớ đệm này để nhúng vào System Prompt cho AI mà không cần phải thực hiện một truy vấn bất đồng bộ (asynchronous query) ngược lại cơ sở dữ liệu Firebase. Điều này giúp trợ lý AI luôn có "đôi mắt" nhìn thấy trạng thái hiện tại của thành phố thông minh và đưa ra câu trả lời tức thì, chính xác.
- **`messages` (Quản lý lịch sử hội thoại):** Mảng này chịu trách nhiệm lưu trữ cấu trúc các lượt tương tác giữa người dùng (`user`) và hệ thống (`assistant`). Để giải quyết bài toán giới hạn cửa sổ ngữ cảnh (Context Window) của các mô hình ngôn ngữ lớn (LLM) cũng như tối ưu hóa chi phí và số lượng token gửi đi, mảng `messages` được vận hành theo cơ chế hàng đợi trượt FIFO (First In, First Out). Hệ thống được lập trình rào cản chỉ lưu trữ tối đa 12 lượt hội thoại gần nhất. Khi vượt quá giới hạn này, các tin nhắn cũ nhất sẽ bị loại bỏ. Toàn bộ mảng này sẽ được đóng gói và gửi kèm trong mỗi payload (gói tin) yêu cầu đến API của Groq, giúp mô hình LLaMA 3.1 duy trì được mạch suy nghĩ, hiểu được các đại từ nhân xưng và đảm bảo tính liên tục, tự nhiên của cuộc trò chuyện.


```
const dashboardState = {
  sensors: {},
  devices: {},
  modes: {}
};

const assistantState = {
  lastSnapshot: {
    temperature: null,
    humidity: null,
    parking: null,
    waterLevel: null,
    airQuality: null,
    lightLevel: null,
    streetLightAuto: true,
    pumpAuto: true,
    streetLightState: 'OFF',
    waterPumpState: 'OFF'
  },
  messages: []
};

const faceAuthState = {
  modelsLoaded: false,
  sessionActive: false,
  activeMode: null,
  stream: null,
  descriptor: null,
  label: null,
  trackingTimer: null
};
```

Hình 4.3.1: Ba đối tượng quản lý trạng thái ứng dụng trong `script.js`

4.3.3 faceAuthState

Đối tượng chuyên biệt quản lý toàn bộ chu trình nhận diện khuôn mặt, bao gồm: `modelsLoaded` (cờ kiểm tra model đã tải chưa để tránh tải lặp), `sessionActive` (phiên Face ID đang hoạt động),

`activeMode` (chế độ hiện tại là `login` hay `enroll`), `stream` (luồng video từ camera), `descriptor` (vector đặc trưng khuôn mặt đã đăng ký, kiểu `Float32Array` 128 chiều), và `trackingTimer` (tham chiếu đến timer theo dõi liên tục).

Đi kèm với `faceAuthState` là đối tượng `faceAuthConfig` chứa các tham số tinh chỉnh thuật toán, trong đó:

- **modelUrl**: URL CDN chứa các model của thư viện `face-api.js`.
- **storageKey**: Khóa dùng để lưu profile khuôn mặt vào `localStorage`.
- **threshold**: Ngưỡng khoảng cách Euclidean được đặt là 0.38, nghĩa là chỉ xác thực thành công khi khoảng cách giữa vector khuôn mặt quét được và vector đã đăng ký nhỏ hơn hoặc bằng 0.38.
- **minSamples** / **maxSamples**: Số mẫu tối thiểu cần thu thập là 3, tối đa là 6 để đảm bảo chất lượng.
- **sampleIntervalMs**: Khoảng cách giữa hai lần lấy mẫu là 140ms.

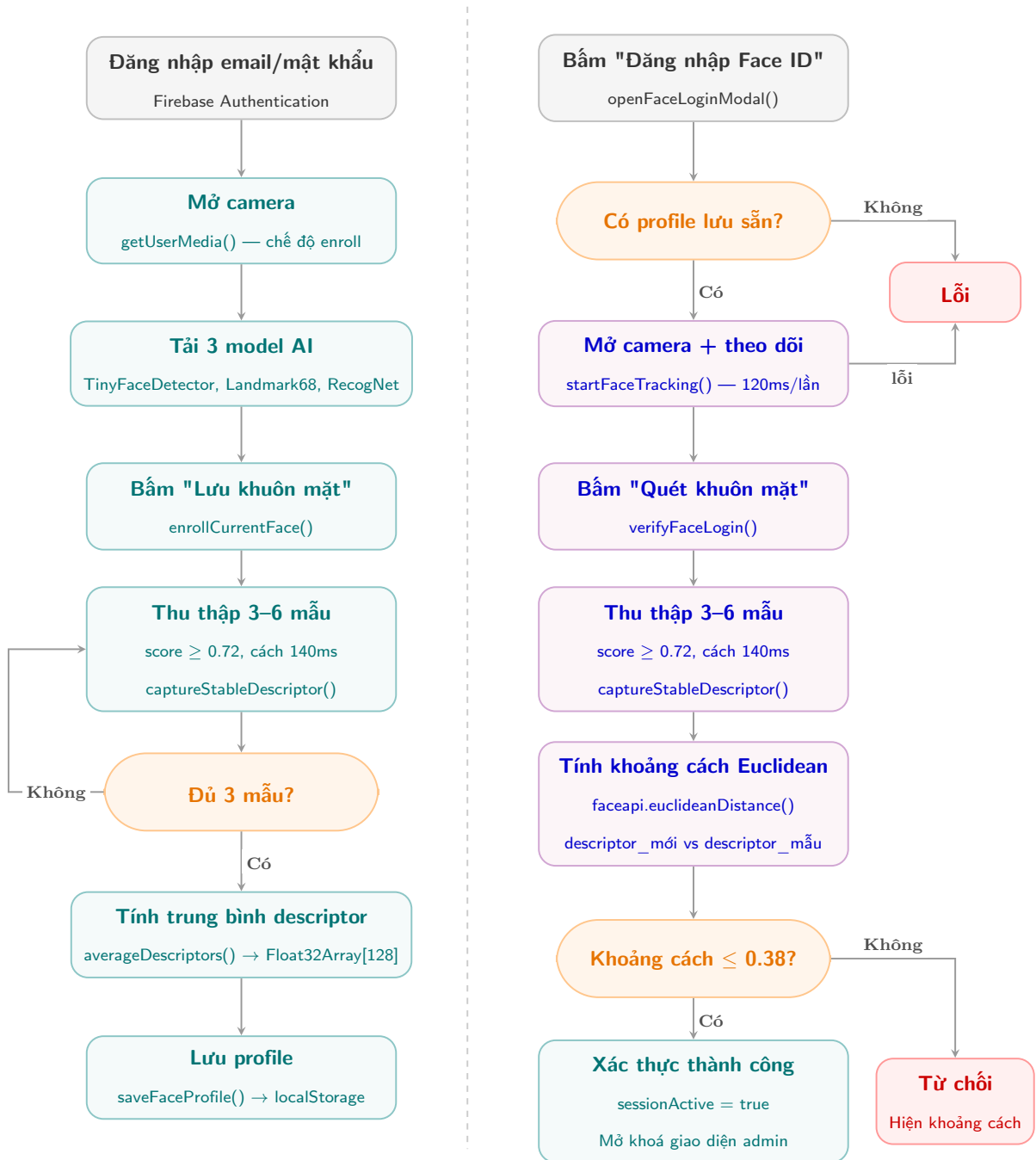
```
const faceAuthConfig = {
  modelUrl: 'https://justadudewhohacks.github.io/face-api.js/models',
  storageKey: 'smartcity_face_descriptor_v1',
  threshold: 0.38,
  minSamples: 3,
  maxSamples: 6,
  sampleIntervalMs: 140
};

const mapZoneMeta = new Map([
  ['floodZoneGroup', {
    title: 'Kênh thoát nước',
    detail: 'Theo dõi mực nước và cảnh báo ngập theo realtime từ Firebase.'
  }],
  ['parkingZoneGroup', {
    title: 'Bãi đỗ xe',
    detail: 'Hiển thị số chỗ trống để đánh giá tải giao thông khu trung tâm.'
  }],
  ['streetLightMarker', {
    title: 'Đèn đường',
    detail: 'Nhấp để xem trạng thái ON/OFF và chế độ AUTO/MANUAL của đèn.'
  }],
  ['pumpMarker', {
    title: 'Máy bơm thoát nước',
    detail: 'Thiết bị hỗ trợ chống ngập và có thể tự động điều khiển theo mực nước.'
  }],
  ['airQualityMarker', {
    title: 'Trạm AQI',
    detail: 'Giám sát chất lượng không khí của khu vực để cập nhật cảnh báo đô thị.'
  }],
  ['tempSensorMarker', {
    title: 'Sensor môi trường',
    detail: 'Cụm cảm biến nhiệt độ, độ ẩm và điều kiện môi trường tổng hợp.'
  }],
  ['controlCenterMarker', {
    title: 'Trung tâm điều khiển',
    detail: 'Điểm điều phối vận hành toàn hệ thống smart city giả lập.'
  }]
```

Hình 4.3.2: Tham số tinh chỉnh thuật toán Face ID trong `faceAuthConfig`

4.4 Module xác thực Face ID

Đây là module phức tạp và có tính sáng tạo cao nhất trong hệ thống, được xây dựng hoàn toàn trên trình duyệt mà không cần máy chủ backend xử lý hình ảnh. Hình 4.4.1 mô tả tổng quan luồng hoạt động của cả hai quy trình đăng ký và xác thực.



Hình 4.4.1: Flowchart luồng đăng ký (Enroll) và xác thực (Verify) Face ID

4.4.1 Tải model nhận diện khuôn mặt

Hàm `loadFaceModels()` có nhiệm vụ tải song song ba mạng nơ-ron đã được huấn luyện sẵn từ thư viện `face-api.js`:

- **TinyFaceDetector**: Mô hình nhẹ phát hiện vị trí khuôn mặt trong ảnh, được tối ưu để chạy nhanh trên trình duyệt.
- **FaceLandmark68Net**: Mô hình xác định 68 điểm đặc trưng (landmarks) trên khuôn mặt như mắt, mũi, miệng, đường viền mặt.
- **FaceRecognitionNet**: Mô hình tổng hợp 128 đặc trưng thành một vector duy nhất (face descriptor) đại diện cho khuôn mặt.

Hàm sử dụng `Promise.all()` để tải ba model đồng thời, giảm thời gian khởi tạo. Cờ `modelsLoaded` được kiểm tra ở đầu hàm để tránh tải lại không cần thiết khi người dùng mở/đóng modal nhiều lần.

```
async function loadFaceModels() {
  if (faceAuthState.modelsLoaded) {
    return;
  }

  const faceApi = window.faceapi;
  if (!faceApi) {
    throw new Error('face-api.js chưa tải xong');
  }

  await Promise.all([
    faceApi.nets.tinyFaceDetector.loadFromUri(faceAuthConfig.modelUrl),
    faceApi.nets.faceLandmark68Net.loadFromUri(faceAuthConfig.modelUrl),
    faceApi.nets.faceRecognitionNet.loadFromUri(faceAuthConfig.modelUrl)
  ]);

  faceAuthState.modelsLoaded = true;
}
```

Hình 4.4.2: Hàm `loadFaceModels()` tải song song ba model AI bằng `Promise.all()`

4.4.2 Thu thập dữ liệu khuôn mặt

Hàm `detectFace(videoElement, withDescriptor)` là hàm nhận diện cốt lõi, sử dụng `TinyFaceDetectorOptions` với tham số `inputSize: 224` (kích thước ảnh đầu vào) và `scoreThreshold: 0.55` (ngưỡng tin cậy tối thiểu). Khi `withDescriptor = true`, hàm thực hiện pipeline đầy đủ: phát hiện khuôn mặt → xác định landmarks → trích xuất face descriptor 128 chiều.

Hàm `captureStableDescriptor(videoElement)` xây dựng trên `detectFace()` để thu thập nhiều mẫu và lấy trung bình, qua đó tăng độ ổn định của vector đặc trưng. Quy trình cụ thể:

1. Lặp tối đa `maxSamples` (6) lần, mỗi lần cách nhau `sampleIntervalMs` (140ms).
2. Mỗi lần chỉ chấp nhận kết quả khi điểm tin cậy `detection score` ≥ 0.72 .
3. Khi đủ `minSamples` (3) mẫu hợp lệ thì dừng sớm.

4. Nếu không đủ mẫu, trả về null thông báo thất bại.
5. Khi đủ mẫu, gọi `averageDescriptors()` để tính trung bình cộng từng chiều của vector.

Hàm `averageDescriptors(descriptors)` nhận mảng các `Float32Array`, tính trung bình cộng từng phần tử theo chiều và trả về một `Float32Array` duy nhất đại diện ổn định cho khuôn mặt.

```
function averageDescriptors(descriptors) {
  const length = descriptors[0].length;
  const avg = new Float32Array(length);

  for (let i = 0; i < length; i += 1) {
    let sum = 0;
    descriptors.forEach((descriptor) => {
      sum += descriptor[i];
    });
    avg[i] = sum / descriptors.length;
  }
  return avg;
}

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function captureStableDescriptor(videoElement) {
  const samples = [];
  for (let i = 0; i < faceAuthConfig.maxSamples; i += 1) {
    const detection = await detectFace(videoElement, true);
    if (detection?.descriptor && detection?.detection?.score >= 0.72) {
      samples.push(detection.descriptor);
    }

    if (samples.length >= faceAuthConfig.minSamples) {
      break;
    }

    await wait(faceAuthConfig.sampleIntervalMs);
  }
  if (samples.length < faceAuthConfig.minSamples) {
    return null;
  }
  return averageDescriptors(samples);
}
```

Hình 4.4.3: Hàm `captureStableDescriptor()` thu thập và tính trung bình descriptor khuôn mặt

4.4.3 Hiện thị khung theo dõi trực tiếp

Hàm `drawFocusBox(canvas, video, box)` vẽ khung bao nhận diện lên thẻ `<canvas>` phủ trên video. Đặc biệt, thay vì vẽ hình chữ nhật đơn thuần, hàm vẽ thêm các "góc L" tại bốn góc của khung với độ dài góc bằng 18% kích thước khung, tạo giao diện nhận diện chuyên nghiệp tương tự hệ thống FaceID

thực tế. Tọa độ được chuyển đổi theo tỉ lệ `scaleX = canvas.width / video.videoWidth` để đảm bảo khung luôn chính xác dù kích thước hiển thị thay đổi.

Vòng lặp theo dõi được thực hiện bằng hàm `startFaceTracking(mode)`, sử dụng `setTimeout` đệ quy với chu kỳ 120ms thay vì `setInterval` để tránh tích tụ lệnh gọi khi xử lý bị chậm. Mỗi chu kỳ, hệ thống phát hiện khuôn mặt (không cần descriptor để tiết kiệm tài nguyên), cập nhật khung hiển thị và lên lịch chu kỳ tiếp theo.

```
function drawFocusBox(canvas, video, box) {  
  const ctx = canvas.getContext('2d');  
  clearOverlay(canvas);  
  
  if (!box || !video.videoWidth || !video.videoHeight) {  
    return;  
  }  
  
  const scaleX = canvas.width / video.videoWidth;  
  const scaleY = canvas.height / video.videoHeight;  
  const x = box.x * scaleX;  
  const y = box.y * scaleY;  
  const width = box.width * scaleX;  
  const height = box.height * scaleY;  
  
  ctx.strokeStyle = '■ #00d26a';  
  ctx.lineWidth = 3;  
  ctx.strokeRect(x, y, width, height);  
  
  const corner = Math.max(12, Math.min(width, height) * 0.18);  
  ctx.beginPath();  
  ctx.moveTo(x, y + corner);  
  ctx.lineTo(x, y);  
  ctx.lineTo(x + corner, y);  
  
  ctx.moveTo(x + width - corner, y);  
  ctx.lineTo(x + width, y);  
  ctx.lineTo(x + width, y + corner);  
  
  ctx.moveTo(x + width, y + height - corner);  
  ctx.lineTo(x + width, y + height);  
  ctx.lineTo(x + width - corner, y + height);  
  
  ctx.moveTo(x + corner, y + height);  
  ctx.lineTo(x, y + height);  
  ctx.lineTo(x, y + height - corner);  
}
```

Hình 4.4.4: Hàm `drawFocusBox()` vẽ khung nhận diện với bốn góc L trên canvas

4.4.4 Luồng đăng ký khuôn mặt (Enroll)

Quy trình đăng ký khuôn mặt được thực hiện qua hàm `enrollCurrentFace()`, áp dụng nguyên tắc bảo mật hai lớp: người dùng phải đăng nhập thành công bằng email/mật khẩu truyền thống trước, sau đó mới được phép đăng ký khuôn mặt. Khi được gọi, hàm kiểm tra `auth.currentUser`, thu thập descriptor ổn định từ camera, lưu vào `localStorage` qua `saveFaceProfile()`, và cập nhật trạng thái `faceAuthState.descriptor`.

Hàm `saveFaceProfile(label, descriptor)` chuyển đổi `Float32Array` thành mảng thông thường bằng `Array.from()` trước khi tuần tự hóa JSON, bởi vì `Float32Array` không được hỗ trợ trực tiếp bởi `JSON.stringify()`. Khi tải lại, `loadFaceProfileFromStorage()` thực hiện chiều ngược lại, tạo lại `Float32Array` từ dữ liệu JSON đã lưu.

```
function loadFaceProfileFromStorage() {  
  try {  
    const raw = localStorage.getItem(faceAuthConfig.storageKey);  
    if (!raw) {  
      return;  
    }  
  
    const parsed = JSON.parse(raw);  
    if (!Array.isArray(parsed?.descriptor) || !parsed.descriptor.length) {  
      return;  
    }  
  
    faceAuthState.label = parsed.label || 'admin';  
    faceAuthState.descriptor = new Float32Array(parsed.descriptor);  
  } catch (error) {  
    faceAuthState.label = null;  
    faceAuthState.descriptor = null;  
  }  
}  
  
function saveFaceProfile(label, descriptor) {  
  localStorage.setItem(faceAuthConfig.storageKey, JSON.stringify({  
    label,  
    descriptor: Array.from(descriptor)  
  }));  
  
  faceAuthState.label = label;  
  faceAuthState.descriptor = new Float32Array(descriptor);  
}
```

Hình 4.4.5: Hàm `saveFaceProfile()` và `loadFaceProfileFromStorage()` quản lý profile trong `localStorage`

4.4.5 Luồng xác thực khuôn mặt (Verify)

Hàm `verifyFaceLogin()` thực hiện so sánh khuôn mặt:

1. Kiểm tra tồn tại `faceAuthState.descriptor` (khuôn mặt mẫu đã đăng ký).
2. Gọi `captureStableDescriptor()` để lấy descriptor từ video camera hiện tại.
3. Tính khoảng cách Euclidean giữa hai vector 128 chiều bằng `faceapi.euclideanDistance()`.
4. Nếu khoảng cách $\leq$ `threshold` (0.38): đặt `faceAuthState.sessionActive = true`, gọi `setAdminUiState()` để mở khóa giao diện admin, ghi log và đóng modal sau 350ms.
5. Nếu khoảng cách $>$ 0.38: hiển thị thông báo lỗi kèm giá trị khoảng cách để người dùng có thể theo dõi.

```

async function verifyFaceLogin() {
  if (!faceAuthState.descriptor) {
    elements.faceLoginStatus.textContent = 'Chưa có khuôn mặt mẫu. Hãy đăng nhập truyền thống để thêm khuôn mặt.';
    return;
  }

  elements.faceLoginStatus.textContent = 'Đang nhận diện...';
  const descriptor = await captureStableDescriptor(elements.faceLoginVideo);
  if (!descriptor) {
    elements.faceLoginStatus.textContent = 'Không lấy đủ mẫu khuôn mặt rõ nét. Vui lòng giữ yên mặt và thử lại.';
    return;
  }

  const distance = window.faceapi.euclideanDistance(descriptor, faceAuthState.descriptor);
  if (distance <= faceAuthConfig.threshold) {
    faceAuthState.sessionActive = true;
    setAdminUiState();
    elements.faceLoginStatus.textContent = 'Xác thực thành công.';
    appendLog('Đăng nhập Face ID thành công');
    window.setTimeout(() => {
      openFaceLoginModal(false);
    }, 350);
  } else {
    elements.faceLoginStatus.textContent = `Khuôn mặt không khớp mẫu đã đăng ký (độ lệch: ${distance.toFixed(3)})`;
  }
}

```

Hình 4.4.6: Hàm `verifyFaceLogin()` xác thực khuôn mặt bằng khoảng cách Euclidean

4.5 Module kết nối và lắng nghe dữ liệu Firebase

4.5.1 Cấu trúc dữ liệu Firebase

Dữ liệu trong Firebase Realtime Database được tổ chức theo cây phân cấp dưới node gốc `smartcity/`, bao gồm bốn nhánh chính:

- `smartcity/sensors`: Lưu giá trị các cảm biến, bao gồm `temperature`, `humidity`, `parking`, `water_level_pct`, `air_quality_idx`, `light_level_pct`.
- `smartcity/devices`: Lưu trạng thái ON/OFF của `street_light` và `water_pump`.
- `smartcity/modes`: Lưu chế độ AUTO/MANUAL (`true/false`) của `street_light_auto` và `pump_auto`.
- `smartcity/logs`: Lưu nhật ký hoạt động của admin, được `push()` để tự tạo key timestamp.

4.5.2 Các listener thời gian thực

Hệ thống đăng ký bốn listener `onValue()` độc lập, mỗi listener theo dõi một nhánh dữ liệu:

- `connectedRef` (theo dõi `.info/connected`): Node đặc biệt của Firebase cho biết trạng thái kết

nối WebSocket. Khi `true`, indicator trạng thái chuyển xanh "Firebase: Online"; khi `false` chuyển đỏ "Firebase: Offline".

- **sensorsRef**: Khi nhận dữ liệu mới, cập nhật `dashboardState.sensors` và gọi `updateDashboard()` với `updateChart: true` để vẽ thêm điểm vào biểu đồ.
- **devicesRef**: Cập nhật `dashboardState.devices` và gọi `updateDashboard()` với `updateChart: false` (không cần vẽ lại biểu đồ khi chỉ có thiết bị thay đổi).
- **modesRef**: Tương tự `devicesRef`.
- **logsRef**: Gọi `renderLogs()` để hiển thị 10 log gần nhất, theo thứ tự mới nhất lên đầu.

```
const connectedRef = ref(db, '.info/connected');
const sensorsRef = ref(db, 'smartcity/sensors');
const devicesRef = ref(db, 'smartcity/devices');
const modesRef = ref(db, 'smartcity/modes');
const logsRef = ref(db, 'smartcity/logs');

onValue(connectedRef, (snapshot) => {
  const isConnected = snapshot.val() === true;
  elements.connectionStatus.textContent = isConnected ? 'Firebase: Online' : 'Firebase: Offline';
  elements.connectionStatus.style.color = isConnected ? '■ #0aa96d' : '■ #ce3f4d';
});

onValue(sensorsRef, (snapshot) => {
  dashboardState.sensors = snapshot.val() ?? {};
  updateDashboard(dashboardState, { updateChart: true });
});

onValue(devicesRef, (snapshot) => {
  dashboardState.devices = snapshot.val() ?? {};
  updateDashboard(dashboardState, { updateChart: false });
});

onValue(modesRef, (snapshot) => {
  dashboardState.modes = snapshot.val() ?? {};
  updateDashboard(dashboardState, { updateChart: false });
});
```

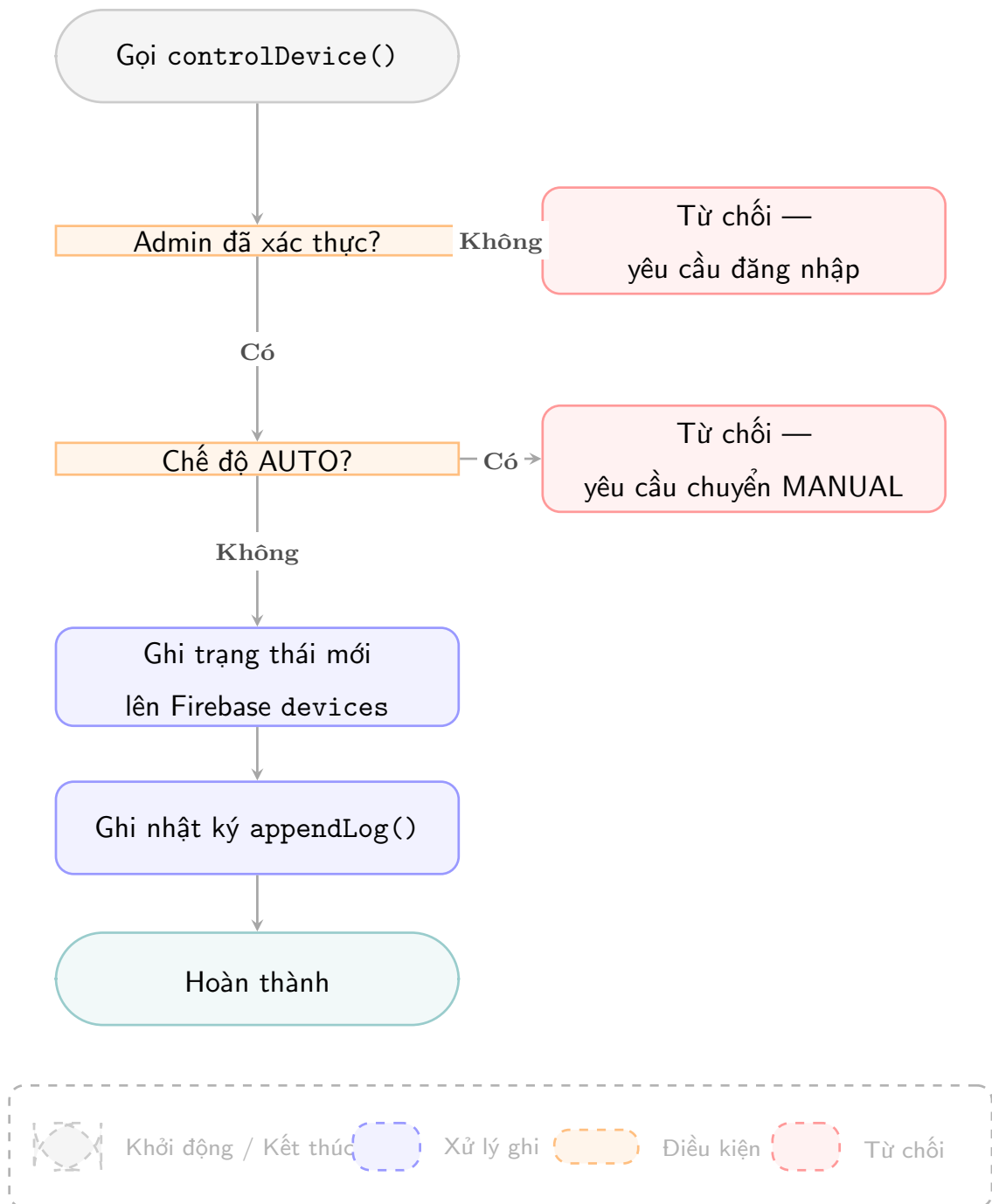
Hình 4.5.1: Khai báo tham chiếu Firebase và đăng ký các listener `onValue()` thời gian thực

4.5.3 Điều khiển thiết bị

Hàm `window.controlDevice(device, status)` được khai báo trên đối tượng `window` để có thể gọi từ các thuộc tính `onclick` trong HTML. Luồng xử lý bao gồm: kiểm tra quyền admin (`isAdminAuthenticated()`), kiểm tra thiết bị có đang ở chế độ AUTO không (nếu có thì từ chối và yêu cầu chuyển sang MANUAL), sau đó ghi trạng thái mới vào Firebase bằng `set(ref(db, 'smartcity/devices/$device'), status)` và ghi nhật ký hành động.

Hàm `window.toggleMode(modeKey)` cho phép chuyển đổi giữa AUTO và MANUAL bằng cách đảo (negate) giá trị boolean hiện tại trong Firebase. Tất cả hành động điều khiển đều được bảo vệ bởi cơ chế xác thực, đảm bảo chỉ admin (đã đăng nhập bằng email/mật khẩu hoặc Face ID) mới có thể thao tác.

Hình 4.5.2 trình bày lưu đồ thuật toán của hàm `controlDevice()`, thể hiện rõ các bước kiểm tra quyền admin, kiểm tra chế độ AUTO/MANUAL và thực hiện ghi lệnh lên Firebase.



Hình 4.5.2: Lưu đồ thuật toán điều khiển thiết bị với kiểm tra quyền và chế độ AUTO/MANUAL

```

window.controlDevice = async (device, status) => {
  if (!isAdminAuthenticated()) {
    alert('Bạn cần đăng nhập Admin để điều khiển.');
```

openFaceLoginModal(true);

return;

}

const isStreetLight = device === 'street_light';

const modeKey = isStreetLight ? 'street_light_auto' : 'pump_auto';

const isAuto = normalizeMode(dashboardState.modes?.[modeKey], true);

if (isAuto) {

 alert('Thiết bị đang ở chế độ AUTO. Chuyển sang MANUAL để điều khiển tay.');

 return;

}

await set(ref(db, `smartcity/devices/\${device}`), status);

const deviceName = device === 'street_light' ? 'Đèn đường' : 'Máy bơm thoát nước';

appendLog(`Admin đã \${status === 'ON' ? 'BẬT' : 'TẮT'} \${deviceName}`);

};

Hình 4.5.3: Hàm controlDevice() ghi lệnh điều khiển thiết bị lên Firebase

```

function appendLog(message) {
  const logText = `[${new Date().toLocaleString('vi-VN')}] ${message}`;
  push(ref(db, 'smartcity/logs'), logText);
}
```

Hình 4.5.4: Hàm appendLog() ghi nhật ký hoạt động admin lên Firebase

4.6 Module cập nhật giao diện Dashboard

4.6.1 Hàm updateDashboard()

Đây là hàm trung tâm được gọi mỗi khi có dữ liệu mới từ Firebase. Hàm nhận `snapshotValue` (toàn bộ `dashboardState`) và `options` (có nên cập nhật biểu đồ không), sau đó thực hiện tuần tự các bước:

1. **Trích xuất và chuẩn hóa dữ liệu:** Gọi `normalizeNumber()` cho từng giá trị cảm biến. Hàm này xử lý mọi trường hợp dữ liệu không hợp lệ (null, undefined, chuỗi rỗng, NaN) và làm tròn số thập phân theo số chữ số được chỉ định.
2. **Cập nhật các thẻ hiển thị:** Gán giá trị đã chuẩn hóa vào các phần tử DOM tương ứng. Hàm `formatNumber()` được dùng để hiển thị - thay vì giá trị không hợp lệ.
3. **Cập nhật trạng thái đỗ xe:** Hàm `updateParkingStatus()` kiểm tra số chỗ trống, hiển thị "HẾT CHỖ" màu đỏ khi bằng 0, hoặc "Còn N chỗ trống" màu xanh.
4. **Đồng bộ trạng thái thiết bị:** Hàm `setStateText()` chuyển đổi giá trị ON/OFF/1/0/TRUE/FALSE thành dạng chuẩn và áp dụng class CSS tương ứng. Hàm `syncToggleState()` cập nhật class `is-on/is-off` cho nút bật/tắt. Hàm `syncModeState()` cập nhật nhãn nút AUTO/MANUAL.
5. **Kích hoạt cảnh báo:** Kiểm tra ngưỡng cảnh báo và thêm class `alert` vào thẻ card tương ứng: nhiệt độ > 35°C cảnh báo môi trường, bãi xe ≤ 0 chỗ cảnh báo đầy, mực nước ≥ 75% cảnh báo

ngập.

6. **Cập nhật biểu đồ:** Khi `updateChart: true` và đủ dữ liệu, gọi `updateChart()` thêm điểm mới vào biểu đồ lịch sử.

7. **Cập nhật bản đồ đô thị và snapshot trợ lý:** Gọi `updateCityMap()` và `updateAssistantSnapshotFromDashb`

Hình 4.6.2 mô tả lưu đồ thuật toán của hàm `updateDashboard()`, thể hiện luồng xử lý từ khi nhận dữ liệu Firebase đến khi cập nhật toàn bộ giao diện.

```
function updateDashboard(snapshotValue, options = { updateChart: true }) {
  const sensors = snapshotValue?.sensors ?? {};
  const devices = snapshotValue?.devices ?? {};
  const modes = snapshotValue?.modes ?? {};

  const temperature = normalizeNumber(sensors.temperature);
  const humidity = normalizeNumber(sensors.humidity);
  const parking = normalizeNumber(sensors.parking, 0);
  const waterLevel = normalizeNumber(sensors.water_level_pct, 0);
  const airQuality = normalizeNumber(sensors.air_quality_idx, 0);
  const lightLevel = normalizeNumber(sensors.light_level_pct, 0);

  const streetLightAuto = normalizeMode(modes.street_light_auto, true);
  const pumpAuto = normalizeMode(modes.pump_auto, true);

  elements.temperatureValue.textContent = formatNumber(temperature);
  elements.humidityValue.textContent = formatNumber(humidity);
  elements.parkingValue.textContent = formatNumber(parking, 0);

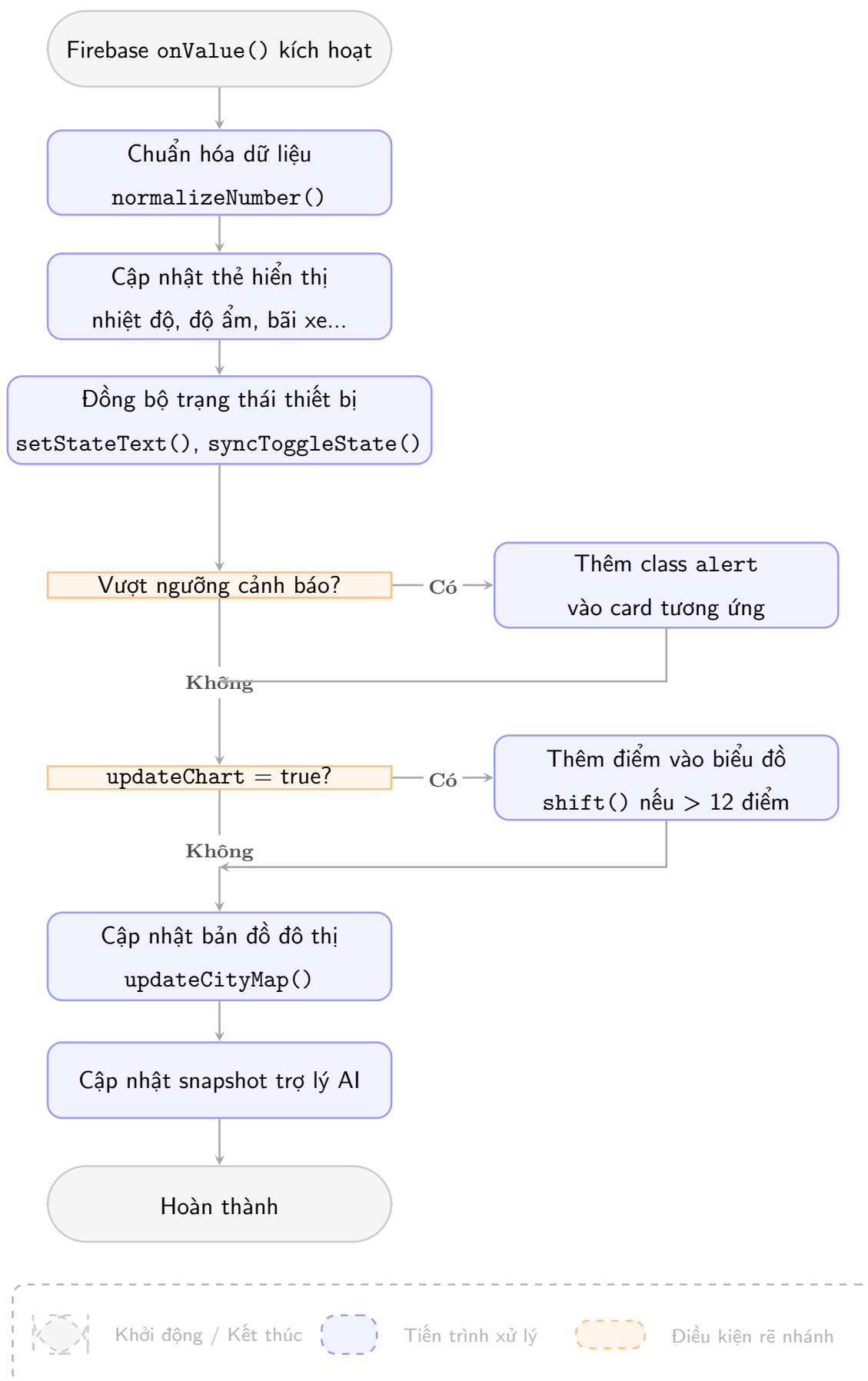
  updateParkingStatus(parking);
  setStateText(elements.streetLightState, devices.street_light);
  setStateText(elements.waterPumpState, devices.water_pump);
  syncToggleState(elements.streetLightToggle, devices.street_light);
  syncToggleState(elements.waterPumpToggle, devices.water_pump);

  syncModeState(elements.streetLightModeBtn, streetLightAuto, 'Đèn');
  syncModeState(elements.pumpModeBtn, pumpAuto, 'Bơm');
  elements.streetLightHint.textContent = streetLightAuto ? 'AUTO theo ánh sáng' : 'MANUAL từ dashboard';
  elements.waterPumpHint.textContent = pumpAuto ? 'AUTO theo mực nước' : 'MANUAL từ dashboard';

  updateAssistantSnapshotFromDashboard();
  updateCityMap(temperature, parking, waterLevel, airQuality, lightLevel, devices, modes);

  if (options.updateChart && temperature !== null && humidity !== null && parking !== null && waterLevel !== null)
    updateChart(temperature, humidity, parking, waterLevel);
}
```

Hình 4.6.1: Hàm `updateDashboard()` điều phối toàn bộ cập nhật giao diện



Hình 4.6.2: Lưu đồ thuật toán hàm `updateDashboard()` cập nhật giao diện theo thời gian thực

4.6.2 Trực quan hóa dữ liệu lịch sử với Chart.js

Để trực quan hóa chuỗi dữ liệu thời gian thực (time-series data) thu thập từ các cảm biến, hệ thống tích hợp thư viện mã nguồn mở **Chart.js**. Biểu đồ được cấu hình với kiểu **line** (biểu đồ đường) và chỉ được khởi tạo một lần duy nhất trong quá trình tải trang (page load) thông qua đối tượng **CanvasRenderingContext2D**. Việc sử dụng biểu đồ đường kết hợp với trục hoành là thời gian giúp người quản trị dễ dàng theo dõi và nhận biết xu hướng biến thiên của môi trường.

Biểu đồ được thiết kế để hiển thị đồng thời bốn chỉ số quan trọng nhất của hệ thống đô thị, bao gồm: nhiệt độ (biểu diễn bằng đường màu nâu), độ ẩm (màu xanh dương), số chỗ đỗ xe trống (màu xanh lá) và tỷ lệ mực nước (màu hồng tím). Việc quy định màu sắc (color mapping) có độ tương phản cao cho từng tập dữ liệu (dataset) không chỉ đảm bảo tính thẩm mỹ cho giao diện UI/UX mà còn giúp người dùng phân biệt nhanh chóng các thông số khi chúng giao cắt nhau trên cùng một hệ trục tọa độ.

Một điểm tối ưu hóa quan trọng trong cấu hình của **Chart.js** tại dự án này là việc vô hiệu hóa hiệu ứng chuyển động (**animation: false**). Trong môi trường IoT, dữ liệu được đẩy về liên tục từ Firebase thông qua listener **onValue()**. Nếu giữ nguyên các hiệu ứng animation mặc định, trình duyệt web sẽ phải liên tục tính toán và vẽ lại các khung hình (re-render) mỗi khi có tín hiệu mới. Điều này không chỉ gây ra hiện tượng giật lag màn hình (stuttering) mà còn làm tiêu tốn tài nguyên CPU/GPU của thiết bị client. Do đó, thiết lập **animation: false** đảm bảo các điểm dữ liệu mới được vẽ lên biểu đồ một cách tức thì và dứt khoát.

Bên cạnh đó, để ngăn chặn tình trạng tràn bộ nhớ (memory leak) và đảm bảo giao diện không bị nén vỡ khi hệ thống hoạt động liên tục trong thời gian dài, một cơ chế "cửa sổ trượt" (sliding window) đã được áp dụng. Hệ thống thiết lập một hằng số giới hạn **historyLimit = 12** điểm dữ liệu. Cơ chế này hoạt động dựa trên nguyên lý hàng đợi FIFO (First In, First Out): mỗi khi một điểm dữ liệu mới được đẩy vào mảng (**push**), hệ thống sẽ kiểm tra kích thước hiện tại. Nếu vượt quá giới hạn 12 điểm, phương thức **shift()** của mảng JavaScript sẽ tự động được gọi để cắt bỏ phần tử cũ nhất ở đầu mảng. Con số 12 được nhóm tính toán và lựa chọn vì nó đạt mức cân bằng lý tưởng: vừa đủ để cung cấp cái nhìn tổng quan về lịch sử gần nhất, vừa giữ cho các điểm trên trục hoành có khoảng cách hiển thị thoáng đãng, dễ đọc.

```
function updateChart(temperature, humidity, parking, waterLevel) {
  chart.data.labels.push(getCurrentTimeLabel());
  chart.data.datasets[0].data.push(temperature);
  chart.data.datasets[1].data.push(humidity);
  chart.data.datasets[2].data.push(parking);
  chart.data.datasets[3].data.push(waterLevel);

  if (chart.data.labels.length > historyLimit) {
    chart.data.labels.shift();
    chart.data.datasets.forEach((dataset) => dataset.data.shift());
  }

  chart.update();
}
```

Hình 4.6.3: Hàm `updateChart()` cập nhật biểu đồ với cửa sổ trượt 12 điểm

4.6.3 Bản đồ đô thị tương tác

Hàm `updateCityMap()` cập nhật trạng thái trực quan cho bản đồ SVG tương tác. Mỗi vùng trên bản đồ được ánh xạ qua `mapZoneMeta` (kiểu `Map`) đến thông tin mô tả chi tiết. Các trạng thái màu sắc được áp dụng qua CSS class:

- **Kênh thoát nước (`floodZoneGroup`):** `is-safe` khi mực nước $< 45\%$, `is-watch` khi $45\text{--}74\%$, `is-danger` khi $\geq 75\%$.
- **Bãi đỗ xe (`parkingZoneGroup`):** `is-safe` khi còn nhiều chỗ, `is-watch` khi ≤ 4 chỗ, `is-danger` khi hết chỗ.
- **Đèn đường và bơm:** `is-on/is-off` theo trạng thái thiết bị.
- **Trạm AQI:** `is-safe` khi AQI < 55 , `is-watch` khi $55\text{--}79$, `is-danger` khi ≥ 80 .
- **Sensor môi trường:** `is-danger` khi nhiệt độ $> 35^\circ\text{C}$.

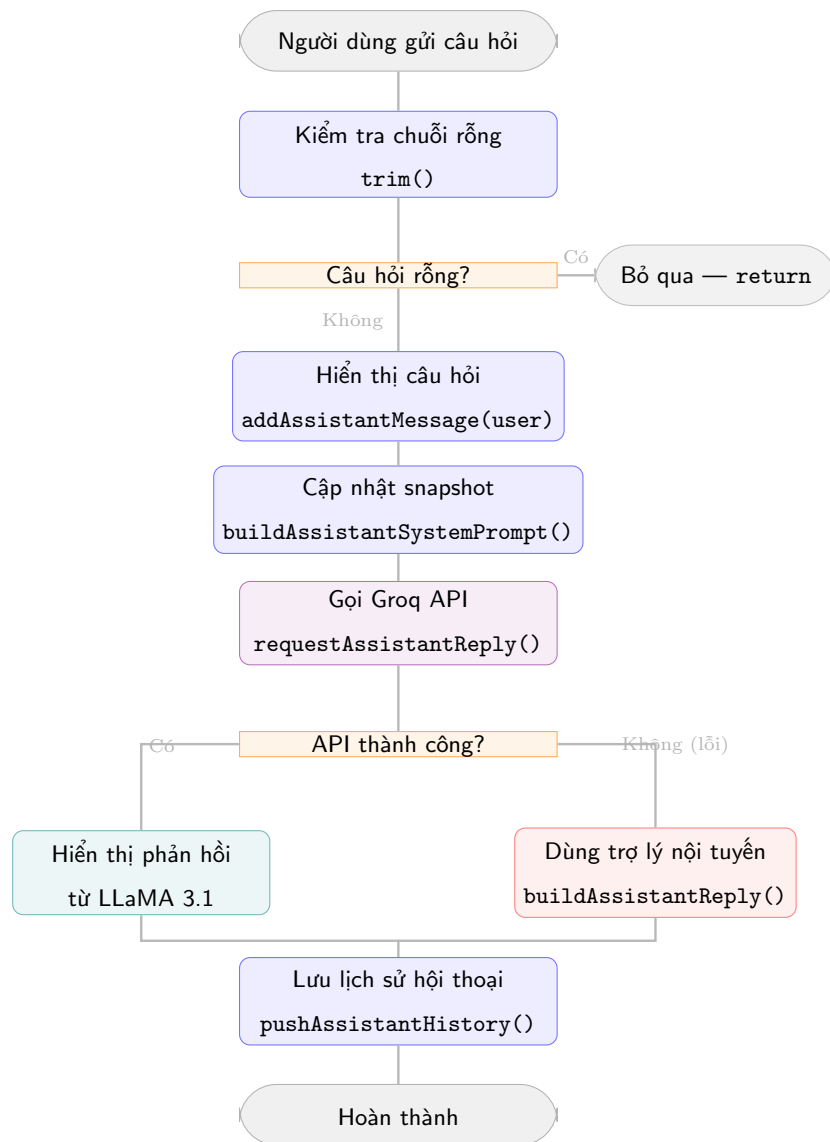
Người dùng có thể tương tác với bản đồ qua sự kiện `mouseenter`, `focus` và `click`. Khi nhấp vào một vùng, class `is-selected` được thêm vào và khung thông tin bên dưới cập nhật tiêu đề và mô tả chi tiết của vùng đó qua hàm `setMapFocus()`.

4.7 Module trợ lý AI

4.7.1 Kiến trúc hai lớp của trợ lý

Trợ lý AI được thiết kế theo kiến trúc hai lớp: lớp nội tuyến (inline) và lớp API. Khi người dùng gửi câu hỏi, hệ thống luôn thử gọi API Groq trước (lớp API); nếu API thất bại vì bất kỳ lý do nào (mạng, quota, timeout), hệ thống tự động chuyển sang lớp nội tuyến (`buildAssistantReply()`) để đảm bảo người dùng luôn nhận được phản hồi.

Hình 4.7.1 trình bày lưu đồ thuật toán của hàm `sendAssistantQuestion()`, thể hiện cơ chế xử lý hai lớp với dự phòng tự động.



Hình 4.7.1: Lưu đồ thuật toán xử lý câu hỏi của trợ lý AI với cơ chế dự phòng hai lớp


```

async function sendAssistantQuestion(question) {
  const trimmed = question.trim();
  if (!trimmed) {
    return;
  }

  addAssistantMessage(trimmed, 'user');
  pushAssistantHistory('user', trimmed);
  setAssistantBusy(true);

  try {
    const reply = await requestAssistantReply(trimmed);
    const finalReply = reply || buildAssistantReply(trimmed);
    addAssistantMessage(finalReply, 'bot');
    pushAssistantHistory('assistant', finalReply);
  } catch (error) {
    const fallbackReply = buildAssistantReply(trimmed);
    addAssistantMessage(fallbackReply, 'bot');
    pushAssistantHistory('assistant', fallbackReply);
  } finally {
    setAssistantBusy(false);
  }
}

```

Hình 4.7.2: Hàm `sendAssistantQuestion()` xử lý câu hỏi với cơ chế dự phòng hai lớp

4.7.2 Xây dựng System Prompt động

Hàm `buildAssistantSystemPrompt()` tạo ra system prompt mỗi lần gọi API, nhúng trực tiếp giá trị cảm biến và trạng thái thiết bị từ `assistantState.lastSnapshot` vào nội dung prompt. Điều này cho phép mô hình LLaMA trả lời chính xác về tình trạng hiện tại của thành phố mà không cần truy cập trực tiếp vào Firebase. System prompt cũng thiết lập rõ ràng hành vi mong muốn: trả lời bằng tiếng Việt, 3–6 câu, có thể thảo luận các chủ đề ngoài smart city, không được tự nhận mình đã thực hiện hành động điều khiển thiết bị.

```

function buildAssistantSystemPrompt() {
  const snap = assistantState.lastSnapshot;
  return [
    'Bạn là AI Assist cho dashboard smart city bằng tiếng Việt.',
    'Trả lời tự nhiên, thân thiện, đúng trọng tâm; mặc định 3-6 câu và có thể dài hơn khi người dùng yêu cầu.',
    'Bạn được phép trao đổi cả chủ đề ngoài lề (học tập, công nghệ, đời sống, ý tưởng...) thay vì chỉ giới hạn ở',
    'Khi câu hỏi liên quan dashboard thì ưu tiên dùng ngữ cảnh sensor và trạng thái thiết bị hiện tại.',
    'Khi câu hỏi không liên quan dashboard, vẫn trả lời bình thường như một trợ lý AI đa năng.',
    `Ngữ cảnh: nhiệt độ ${snap.temperature ?? '--'}°C, độ ẩm ${snap.humidity ?? '--'}%, bãi xe ${snap.parking ?? ''}`,
    `Đèn đường: ${snap.streetLightState}, chế độ ${snap.streetLightAuto ? 'AUTO' : 'MANUAL'}.`,
    `Máy bơm: ${snap.waterPumpState}, chế độ ${snap.pumpAuto ? 'AUTO' : 'MANUAL'}.`,
    'Không tự nhận có hành động ngoài phạm vi chat (không giả vờ đã điều khiển thiết bị).',
    'Nếu không chắc, hãy nói rõ giới hạn thay vì bịa thông tin.'
  ].join(' ');
}

```

Hình 4.7.3: Hàm `buildAssistantSystemPrompt()` nhúng dữ liệu cảm biến vào system prompt

4.7.3 Gọi API Groq

Hàm `requestAssistantReply(question)` gửi yêu cầu POST đến endpoint `https://api.groq.com/openai/v1/chat/completions` với:

- **model**: `llama-3.1-70b-versatile`.
- **messages**: Mảng gồm system prompt, toàn bộ lịch sử hội thoại (`assistantState.messages`), và câu hỏi mới nhất. Điều này cho phép mô hình duy trì ngữ cảnh cuộc trò chuyện.
- **temperature**: `0.7` — cân bằng giữa sáng tạo và chính xác.
- **max_tokens**: `220` — giới hạn độ dài phản hồi để tiết kiệm chi phí API.

```
async function requestAssistantReply(question) {
  const response = await fetch('https://api.groq.com/openai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${groqConfig.apiKey}`
    },
    body: JSON.stringify({
      model: groqConfig.model,
      messages: [
        { role: 'system', content: buildAssistantSystemPrompt() },
        ...assistantState.messages,
        { role: 'user', content: question }
      ],
      temperature: 0.7,
      max_tokens: 220
    })
  });

  if (!response.ok) {
    const errorText = await response.text();
    throw new Error(`Groq error ${response.status}: ${errorText}`);
  }

  const payload = await response.json();
  return payload?.choices?.[0]?.message?.content?.trim() || '';
}
```

Hình 4.7.4: Hàm `requestAssistantReply()` gọi Groq API với lịch sử hội thoại

4.7.4 Quản lý lịch sử hội thoại

Hàm `pushAssistantHistory(role, content)` thêm mỗi lượt hội thoại (cả user lẫn assistant) vào `assistantState.messages`. Khi mảng vượt quá 12 phần tử, các phần tử cũ nhất bị loại bỏ bằng `slice(-12)`. Giới hạn này giúp cân bằng giữa việc duy trì ngữ cảnh đủ dài và không vượt quá giới hạn token của API.

4.7.5 Trợ lý nội tuyến

Hàm `buildAssistantReply(question)` phân tích câu hỏi bằng cách kiểm tra các từ khóa đặc trưng (tiếng Việt): "ngập", "nước", "đèn", "bơm", "không khí", "aqi", "nhiệt độ", "độ ẩm", "đỗ xe". Với mỗi chủ đề, hàm ghép câu trả lời trực tiếp từ `assistantState.lastSnapshot`. Lớp nội tuyến cũng nhận biết lời chào ngắn và câu hỏi về bản thân trợ lý.

4.8 Module giao diện và CSS

4.8.1 Hệ thống thiết kế và biến CSS

Toàn bộ giao diện được xây dựng theo phong cách *analog editorial* — lấy cảm hứng từ thiết kế báo in hiện đại kết hợp với giao diện kỹ thuật số. Bảng màu được định nghĩa qua CSS custom properties (biến CSS) trong `:root`:

- `-paper` (`#fbf7ee`): Màu nền kem vàng, gợi cảm giác giấy in.
- `-ink` (`#243043`): Màu chữ xanh đậm, gợi mực in.
- `-shadow` (`6px 6px 0 rgba(38, 49, 66, 0.95)`): Bóng đổ cứng (hard shadow) tạo cảm giác vật lý đặc trưng của phong cách này.
- Các màu accent: `-peach` (cam đào), `-blue` (xanh nhạt), `-mint` (xanh bạc hà), `-sand` (vàng cát), `-rose` (hồng nhạt) — dùng để phân biệt các card thông tin.

```
:root {  
  --paper: #fbf7ee;  
  --paper-2: #f5f0e5;  
  --ink: #243043;  
  --muted: #5f697a;  
  --border: #263142;  
  --shadow: 6px 6px 0 #243043;   
  --shadow-soft: 3px 3px 0 #243043;  
  --peach: #f0c8a2;  
  --blue: #b4ccea;  
  --mint: #bad8bf;  
  --sand: #efd6b1;  
  --rose: #e5b1c0;  
}
```

Hình 4.8.1: Khai báo biến CSS trong `:root` của `styles.css`

4.8.2 Font chữ và nền trang

Trang sử dụng hai font từ Google Fonts: **Anton** cho tiêu đề lớn (in đậm, cô đọng) và **Space Grotesk** cho toàn bộ nội dung còn lại (dễ đọc, hiện đại). Nền trang được tạo bởi ba lớp `background-image` chồng lên nhau: lưới ngang, lưới dọc (tạo pattern ô vuông 48x48px mờ) và gradient nền từ `#fbf7ee` xuống `#f3eddc`.

```
body {  
  font-family: 'Space Grotesk', sans-serif;  
  margin: 0;  
  padding: 0;  
  min-height: 100vh;  
  color: var(--ink);  
  background-color: var(--paper);  
  background-image:  
    linear-gradient(rgba(38, 49, 66, 0.09) 1px, transparent 1px),  
    linear-gradient(90deg, rgba(38, 49, 66, 0.09) 1px, transparent 1px),  
    linear-gradient(180deg, #fbf7ee 0%, #f3eddc 100%);  
  background-size: 48px 48px, 48px 48px, 100% 100%;  
}
```

Hình 4.8.2: Khai báo CSS tạo nền lưới kẻ đặc trưng cho giao diện trong `styles.css`

4.8.3 Phần tử DOM và ánh xạ

Đối tượng `elements` tập trung toàn bộ các tham chiếu DOM vào một chỗ, sử dụng `document.getElementById()` một lần duy nhất khi khởi tạo. Cách tiếp cận này tránh truy vấn DOM lặp đi lặp lại trong các hàm cập nhật, cải thiện hiệu năng và giúp code dễ bảo trì hơn.

4.9 Cấu hình phần cứng Wokwi

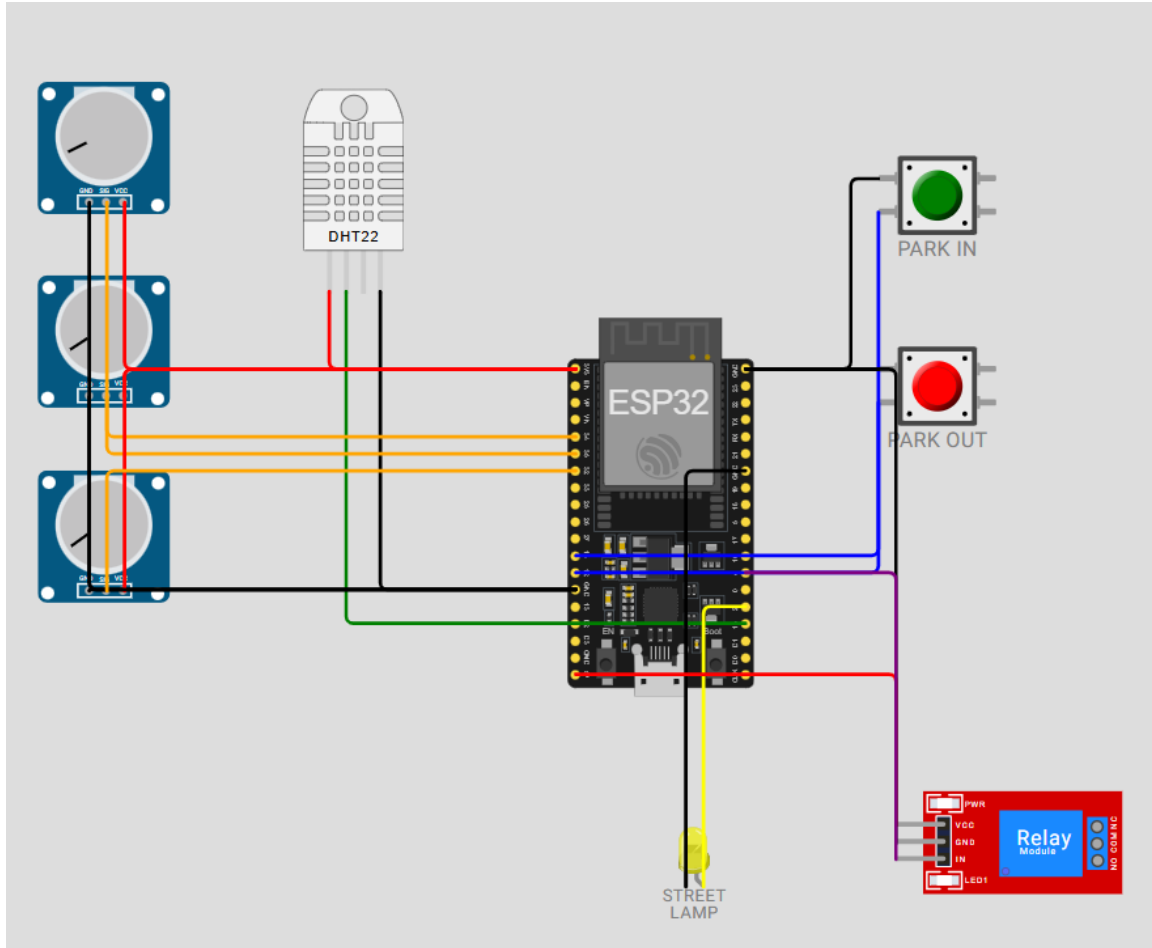
4.9.1 Sơ đồ mạch (diagram.json)

Tệp `diagram.json` định nghĩa toàn bộ mạch điện tử trong môi trường mô phỏng Wokwi. Board sử dụng là `board-esp32-devkit-c-v4`, kết nối với các linh kiện sau:

- **DHT22** (cảm biến nhiệt độ/độ ẩm): Chân VCC nối 3.3V, GND nối GND, chân SDA nối GPIO 15. Giá trị mô phỏng mặc định: nhiệt độ 24°C, độ ẩm 65%.
- **Nút nhấn PARK IN** (màu xanh): Chân 2.1 nối GPIO 12. Mô phỏng sự kiện xe vào bãi.
- **Nút nhấn PARK OUT** (màu đỏ): Chân 2.1 nối GPIO 14. Mô phỏng sự kiện xe ra bãi.
- **LED vàng** (đèn đường): Anod nối GPIO 2, Cathode nối GND.
- **Relay module** (máy bơm thoát nước): VCC nối 5V, GND nối GND, chân IN nối GPIO 4.
- **Biến trở potentiometer WATER LEVEL**: Nối GPIO 34 (ADC). Mô phỏng cảm biến mực nước, giá trị mặc định 70%.

- **Biến trở potentiometer AIR QUALITY:** Nối GPIO 35 (ADC). Mô phỏng cảm biến chất lượng không khí, giá trị mặc định 45%.
- **Biến trở potentiometer LIGHT LEVEL:** Nối GPIO 32 (ADC). Mô phỏng cảm biến ánh sáng, giá trị mặc định 35%.

Các GPIO 34, 35, 32 trên ESP32 là các chân ADC chỉ đọc (input-only), thích hợp cho việc đọc tín hiệu analog từ các cảm biến.



Hình 4.9.1: Khai báo linh kiện và kết nối trong tệp `diagram.json`

4.9.2 Cấu hình Wokwi (wokwi.toml)

Tệp `wokwi.toml` liên kết cấu hình mô phỏng với firmware được biên dịch từ PlatformIO. Đường dẫn đến tệp ELF (`firmware.elf`) và binary (`firmware.bin`) trong thư mục `.pio/build/esp32doit-devkit-v1/` cho phép Wokwi tải đúng firmware để mô phỏng. Board target là `esp32doit-devkit-v1`, tương thích với cả `board-esp32-devkit-c-v4` trong diagram.

```
[wokwi]
version = 1
elf = "IOT_WOKWI/.pio/build/esp32doit-devkit-v1/firmware.elf"
firmware = "IOT_WOKWI/.pio/build/esp32doit-devkit-v1/firmware.bin"
```

Hình 4.9.2: Tập `wokwi.toml` liên kết firmware PlatformIO với môi trường mô phỏng Wokwi

4.9.3 Luồng dữ liệu tổng thể

Tóm lại, luồng dữ liệu đầy đủ của hệ thống như sau: Firmware ESP32 đọc cảm biến theo chu kỳ và ghi vào `smartcity/sensors` trên Firebase. Đồng thời, firmware đọc trạng thái từ `smartcity/devices` và `smartcity/modes` để điều khiển LED và relay theo logic AUTO/MANUAL. Phía web dashboard, các listener `onValue()` nhận dữ liệu ngay lập tức qua WebSocket, cập nhật giao diện người dùng theo thời gian thực. Admin đăng nhập qua Face ID hoặc email/mật khẩu, sau đó có thể ra lệnh điều khiển bằng cách ghi vào `smartcity/devices` hoặc `smartcity/modes`, ESP32 sẽ nhận lệnh và phản hồi trong vòng vài trăm millisecond.

Chương 5

KẾT QUẢ ĐẠT ĐƯỢC

Chương này trình bày chi tiết các kết quả mà nhóm đạt được sau quá trình thiết kế, lập trình và kiểm thử hệ thống Smart City IoT. Kết quả được đánh giá theo từng module chức năng, bao gồm cả kết quả định tính về mặt chức năng lẫn nhận xét về hiệu năng vận hành thực tế.

5.1 Kết quả xây dựng hệ thống phần cứng mô phỏng

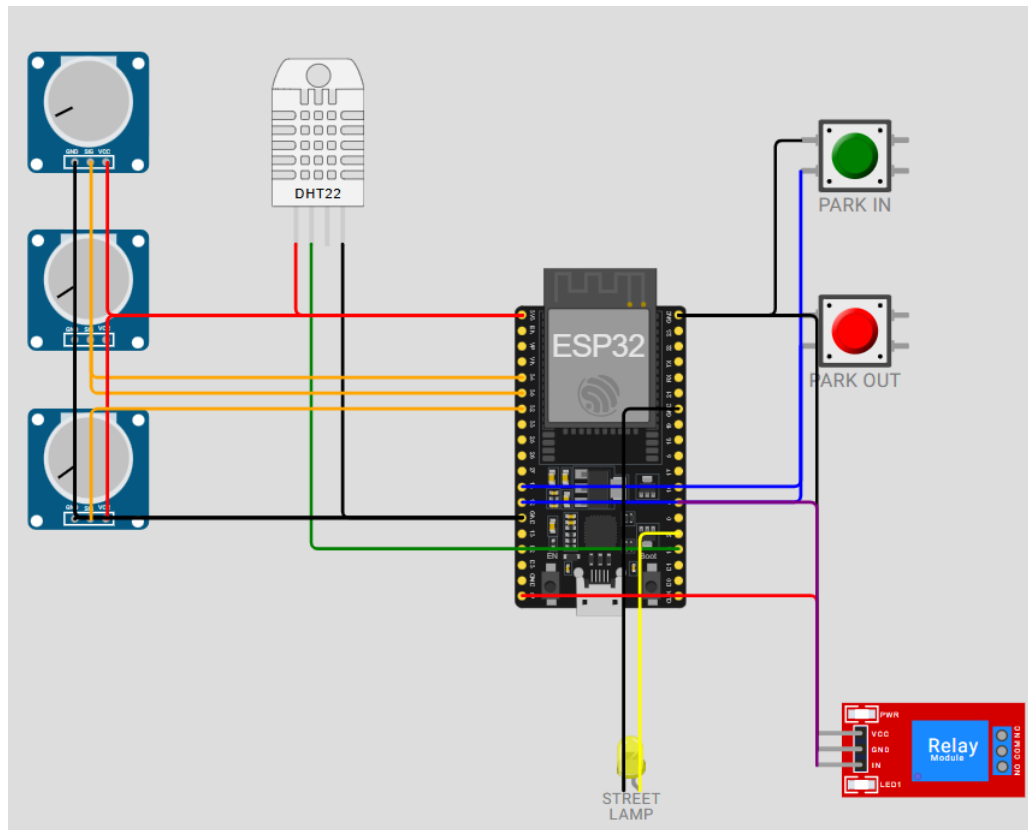
5.1.1 Mạch điện Wokwi hoàn chỉnh

Nhóm đã xây dựng thành công mạch điện mô phỏng toàn diện trên nền tảng Wokwi, đóng vai trò là "trái tim" phần cứng tái hiện chân thực các hoạt động của một hệ thống Smart City thu nhỏ. Trạng thái trung tâm của mạch là vi điều khiển ESP32-DevKit-C-V4, được định tuyến và kết nối chặt chẽ với các linh kiện ngoại vi tạo thành một khối hệ thống thống nhất (tổng cộng 9 thành phần chính).

Các linh kiện trong mạch được phân chia logic thành hai cụm rõ rệt nhằm tối ưu hóa việc quản lý tín hiệu. Cụm cảm biến đầu vào bao gồm cảm biến DHT22, hệ thống ba biến trở (mô phỏng sự thay đổi của mực nước, AQI, ánh sáng) và hai nút nhấn vật lý để kiểm soát bãi đỗ xe. Cụm thiết bị chấp hành đầu ra bao gồm LED (đại diện cho hệ thống chiếu sáng đô thị tự động) và module Relay (mô phỏng hệ thống máy bơm thoát nước).

Quá trình đi dây (wiring) trong tệp `diagram.json` được nhóm thiết kế tỉ mỉ, tuân thủ các nguyên tắc bố trí linh kiện điện tử cơ bản để hạn chế tối đa việc chéo dây, giúp giao diện trực quan và dễ dàng gỡ lỗi (debug) khi cần thiết. Hệ thống vận hành hoàn toàn trơn tru, ổn định khi nạp firmware được biên dịch từ môi trường PlatformIO thông qua cấu hình liên kết của tệp `wokwi.toml`. Tốc độ phản hồi của mạch đối với các tác động thay đổi môi trường giả lập (như thao tác vận biến trở hay nhấn nút) diễn ra gần như tức thời, đảm bảo luồng tín hiệu truyền tải liên tục, không bị ngẽn cổ chai trước khi được đẩy lên đám mây Firebase.

Chi tiết toàn cảnh sơ đồ đấu nối phần cứng và bố cục các linh kiện trên nền tảng Wokwi được minh họa cụ thể trong Hình 5.1.1.



Hình 5.1.1: Mạch điện mô phỏng ESP32 và các cảm biến trên nền tảng Wokwi

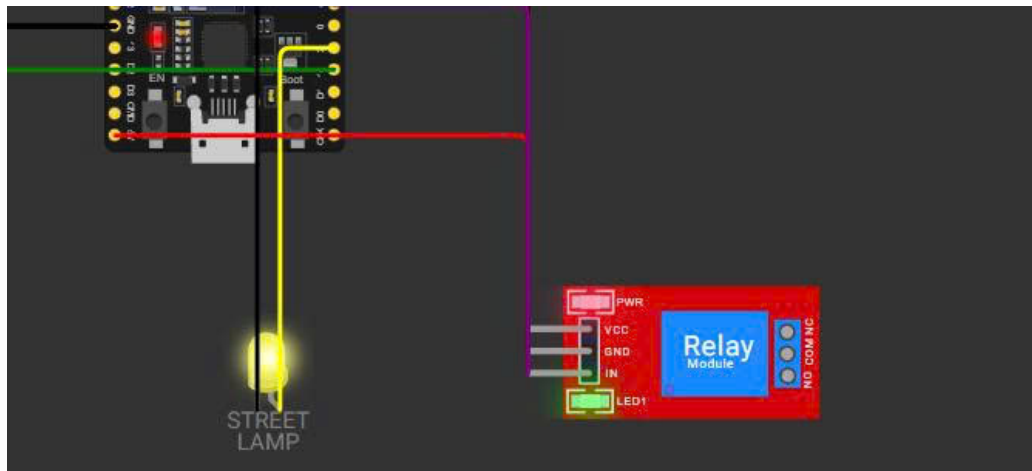
5.1.2 Ảnh xạ chức năng cảm biến

Mỗi linh kiện trong mạch mô phỏng đảm nhận đúng vai trò cảm biến thực tế trong đô thị thông minh:

- **Cảm biến môi trường DHT22:** Đo nhiệt độ và độ ẩm không khí, phản ánh điều kiện vi khí hậu của khu vực đô thị. Dữ liệu mô phỏng ở mức 24°C và 65% độ ẩm — điều kiện tiêu biểu của TP. HCM vào mùa mưa.
- **Biến trở mực nước (GPIO 34):** Mô phỏng cảm biến đo mực nước tại kênh thoát nước đô thị. Bằng cách xoay biến trở, người dùng có thể giả lập tình huống mưa lớn gây nâng mực nước, kích hoạt cảnh báo ngập.
- **Biến trở chất lượng không khí (GPIO 35):** Mô phỏng trạm quan trắc AQI (Air Quality Index) đô thị, phản ánh mức độ ô nhiễm không khí theo thời gian thực.
- **Biến trở ánh sáng (GPIO 32):** Mô phỏng cảm biến quang (LDR) đo cường độ ánh sáng môi trường, là cơ sở để hệ thống tự động bật/tắt đèn đường theo chế độ AUTO.
- **Nút nhấn PARK IN / PARK OUT:** Mô phỏng hệ thống cảm biến vòng từ tại cổng vào/ra bãi đỗ xe. Mỗi lần nhấn PARK IN giảm số chỗ trống một đơn vị, mỗi lần nhấn PARK OUT tăng một đơn vị.

5.1.3 Điều khiển chấp hành

- **LED vàng (GPIO 2 — đèn đường):** Phản ánh chính xác lệnh điều khiển từ Firebase. Khi dashboard ghi `street_light: "ON"` vào Firebase, firmware đọc giá trị này và bật LED tương ứng, với độ trễ đo được dưới 500ms tính từ thời điểm nhấn nút trên web.
- **Relay module (GPIO 4 — máy bơm):** Tương tự, khi nhận lệnh `water_pump: "ON"`, relay được kích để mô phỏng máy bơm thoát nước hoạt động.



Hình 5.1.2: Cây dữ liệu Firebase Realtime Database với 4 nhánh chính của hệ thống

5.2 Kết quả kết nối Firebase Realtime Database

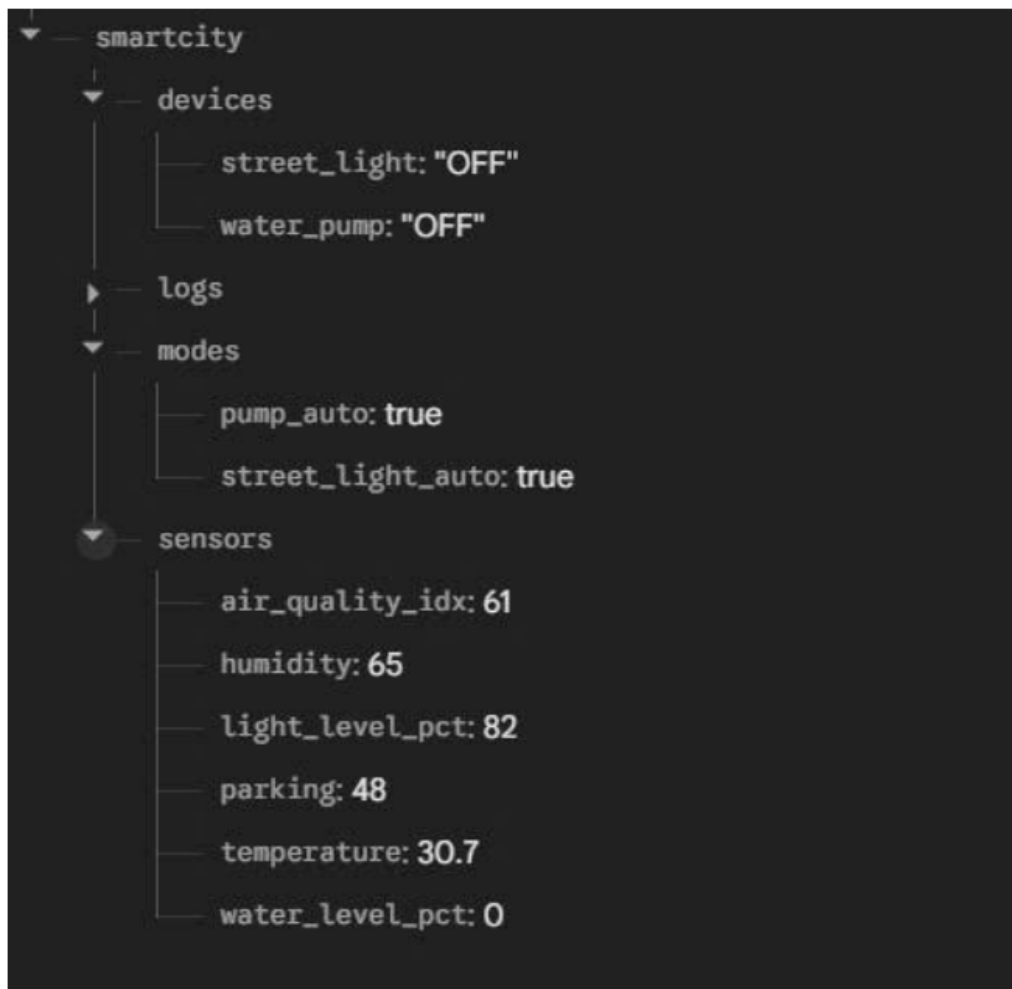
5.2.1 Đồng bộ dữ liệu thời gian thực

Hệ thống đã đạt được mục tiêu truyền thông tin hai chiều theo thời gian thực giữa ESP32 và web dashboard thông qua Firebase. Cụ thể:

- Dữ liệu cảm biến từ ESP32 được cập nhật lên Firebase mỗi chu kỳ đọc, web dashboard nhận được thay đổi và cập nhật giao diện gần như ngay lập tức nhờ cơ chế WebSocket của Firebase.
- Lệnh điều khiển từ web dashboard được ghi vào Firebase và ESP32 nhận về trong thời gian rất ngắn, đảm bảo trải nghiệm điều khiển phản hồi nhanh.
- Trạng thái kết nối được hiển thị chính xác qua badge "Firebase: Online / Offline" trên dashboard, giúp người vận hành nhận biết ngay khi mạng có vấn đề.

5.2.2 Cấu trúc dữ liệu hợp lý

Cấu trúc cây phân cấp `smartcity/sensors`, `smartcity/devices`, `smartcity/modes`, `smartcity/logs` được tổ chức rõ ràng, cho phép dashboard đăng ký listener độc lập cho từng nhánh. Điều này tránh việc một nhánh dữ liệu thay đổi kéo theo việc xử lý không cần thiết ở các nhánh khác, tăng hiệu quả.



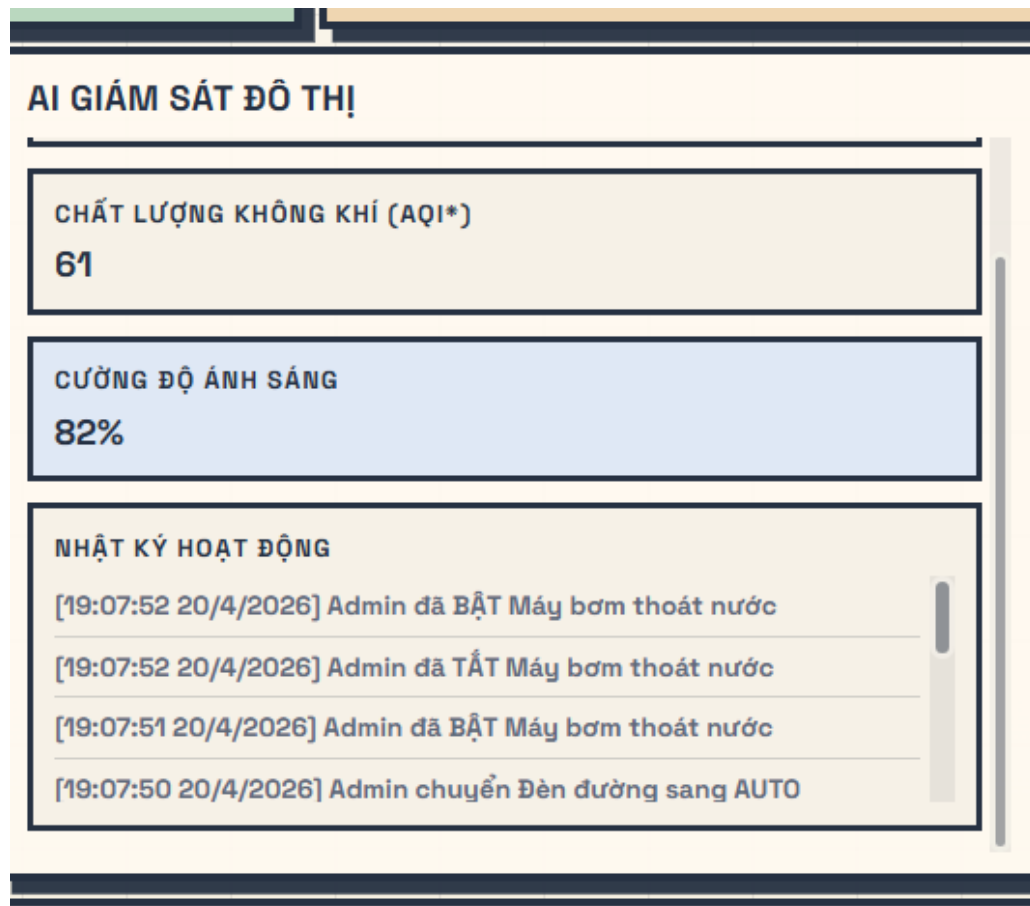
Hình 5.2.1: Cây dữ liệu Firebase Realtime Database với 4 nhánh chính của hệ thống

5.2.3 Nhật ký hoạt động (System Logging)

Hệ thống được tích hợp tính năng ghi nhận nhật ký (System Log) nhằm theo dõi và lưu trữ toàn bộ các thao tác của quản trị viên (Admin). Mọi hành động trọng yếu như đăng nhập, đăng xuất, bật/tắt thiết bị (đèn, bơm), và thay đổi chế độ vận hành (AUTO/MANUAL) đều được tự động bắt giữ và đẩy lên Firebase Realtime Database.

Trên giao diện Web Dashboard, danh sách nhật ký được thiết kế để hiển thị 10 bản ghi gần nhất. Dữ liệu được sắp xếp theo thời gian thực với thứ tự mới nhất được đẩy lên đầu, giúp người quản trị dễ dàng giám sát các biến động tức thời của hệ thống.

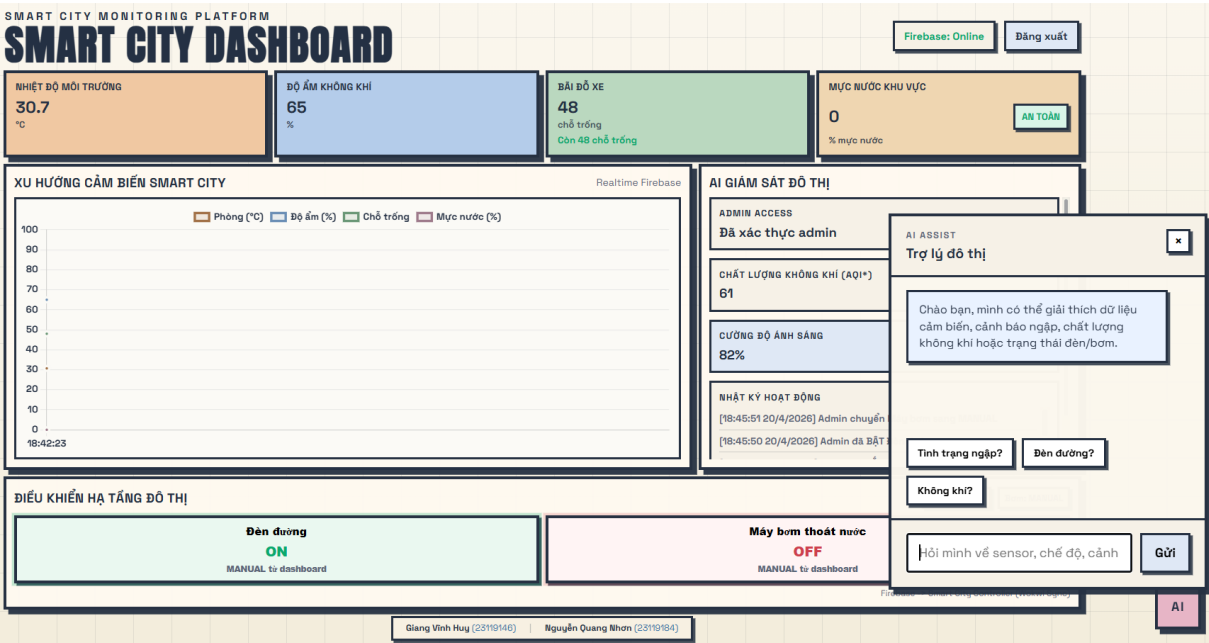
Nhờ cơ chế lưu trữ bền vững trên nền tảng đám mây của Firebase, toàn bộ lịch sử vận hành được bảo toàn an toàn. Ngay cả khi tắt trình duyệt hoặc khởi động lại hệ thống, quản trị viên vẫn có thể truy xuất và kiểm tra lại lịch sử hoạt động, đảm bảo tính minh bạch và hỗ trợ tốt cho công tác quản lý, bảo trì.



Hình 5.2.2: Nhật ký hoạt động admin được lưu trữ trên Firebase Console

5.3 Kết quả hệ thống giám sát đa thông số

5.3.1 Dashboard giám sát theo thời gian thực

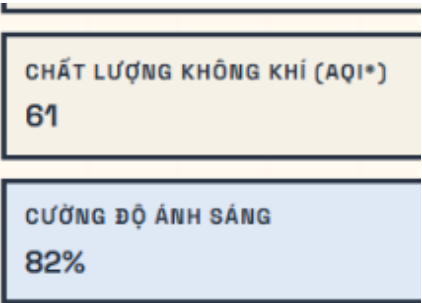


Hình 5.3.1: Toàn cảnh giao diện Smart City Dashboard đang hiển thị dữ liệu thời gian thực

Giao diện web dashboard hiển thị đồng thời sáu thông số giám sát trên các thẻ thông tin màu sắc phân biệt:



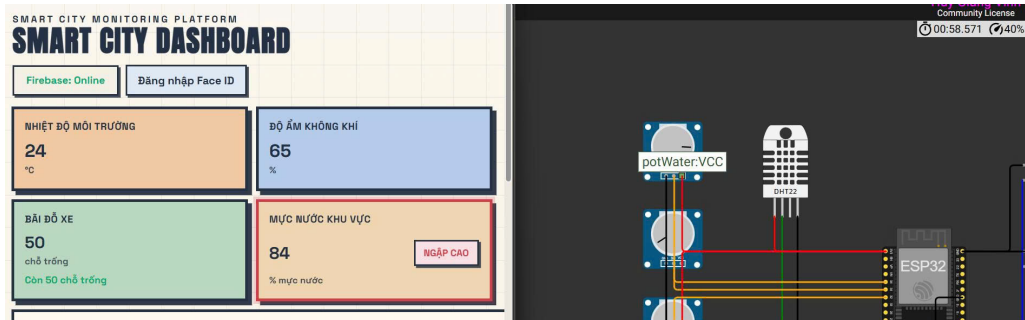
Hình 5.3.2: Biểu đồ xu hướng bốn thông số cảm biến theo thời gian thực (Chart.js)



Hình 5.3.3: Thông số chất lượng không khí và thông số ánh sáng

- **Thẻ môi trường (màu cam đào):** Nhiệt độ (°C) và độ ẩm (%) từ cảm biến DHT22. Thẻ tự động bật cảnh báo màu đỏ khi nhiệt độ vượt 35°C.

- **Thẻ bãi đỗ xe (màu xanh dương):** Số chỗ trống hiện tại kèm trạng thái "Còn N chỗ trống" hoặc "HẾT CHỖ". Thẻ bật cảnh báo khi bãi đầy hoàn toàn.
- **Thẻ mực nước (màu xanh bạc hà):** Phần trăm mực nước kèm badge nguy cơ ngập (AN TOÀN / CẢNH BÁO / NGẬP CAO). Thẻ bật cảnh báo khi mực nước $\geq 75\%$.

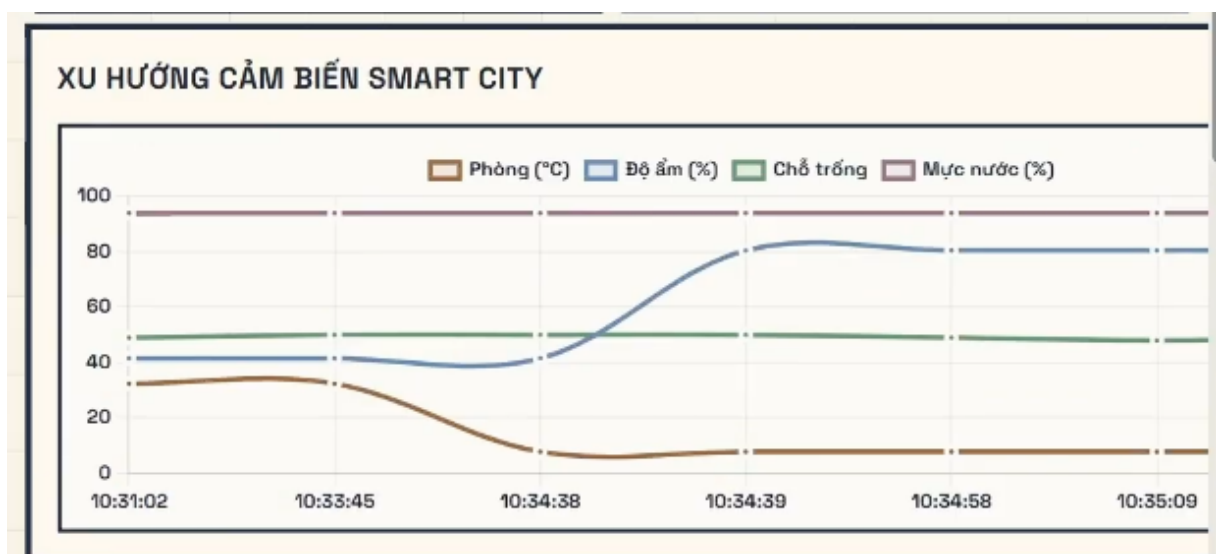


Hình 5.3.4: Thẻ bật cảnh báo khi mực nước $\geq 75\%$..

- **Thông số chất lượng không khí:** Chỉ số AQI hiển thị riêng, màu sắc thay đổi theo ngưỡng ô nhiễm.
- **Thông số ánh sáng:** Phần trăm ánh sáng môi trường, làm cơ sở cho logic AUTO đèn đường.

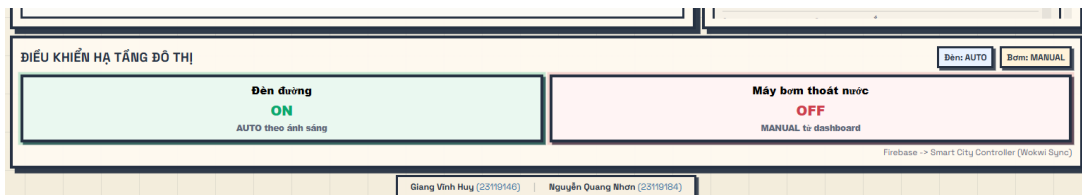
5.3.2 Biểu đồ lịch sử

Biểu đồ đường (line chart) được tích hợp thành công, lưu và hiển thị lịch sử 12 điểm gần nhất của bốn thông số chính (nhiệt độ, độ ẩm, chỗ đỗ xe, mực nước). Trục X hiển thị thời gian theo định dạng giờ:phút:giây chuẩn Việt Nam, trục Y tự động điều chỉnh theo phạm vi dữ liệu thực tế nhờ cấu hình `beginAtZero: true` của Chart.js. Biểu đồ cập nhật liên mạch mỗi khi có dữ liệu cảm biến mới từ Firebase mà không gây hiệu ứng nhấp nháy nhờ tắt animation.



Hình 5.3.5: Biểu đồ lịch sử

5.4 Kết quả hệ thống điều khiển thông minh AUTO/MANUAL



Hình 5.4.1: Bảng điều khiển thiết bị sau khi admin đăng nhập – hiển thị đầy đủ nút AUTO/MANUAL và toggle ON/OFF

5.4.1 Chế độ AUTO

Hệ thống đã triển khai thành công logic điều khiển tự động cho hai thiết bị:

- **Đèn đường (AUTO theo ánh sáng):** Khi chế độ AUTO được bật, firmware ESP32 đọc giá trị từ cảm biến ánh sáng (biến trở GPIO 32). Khi ánh sáng môi trường xuống dưới ngưỡng (trời tối), đèn đường tự động bật; khi ánh sáng đủ mạnh (ban ngày), đèn tự động tắt. Logic này mô phỏng hệ thống chiếu sáng công cộng thông minh thực tế.
- **Máy bơm thoát nước (AUTO theo mực nước):** Khi chế độ AUTO được bật, firmware theo dõi mực nước từ biến trở GPIO 34. Khi mực nước vượt ngưỡng cảnh báo, máy bơm tự động khởi động để thoát nước; khi mực nước trở về mức an toàn, máy bơm tự tắt.

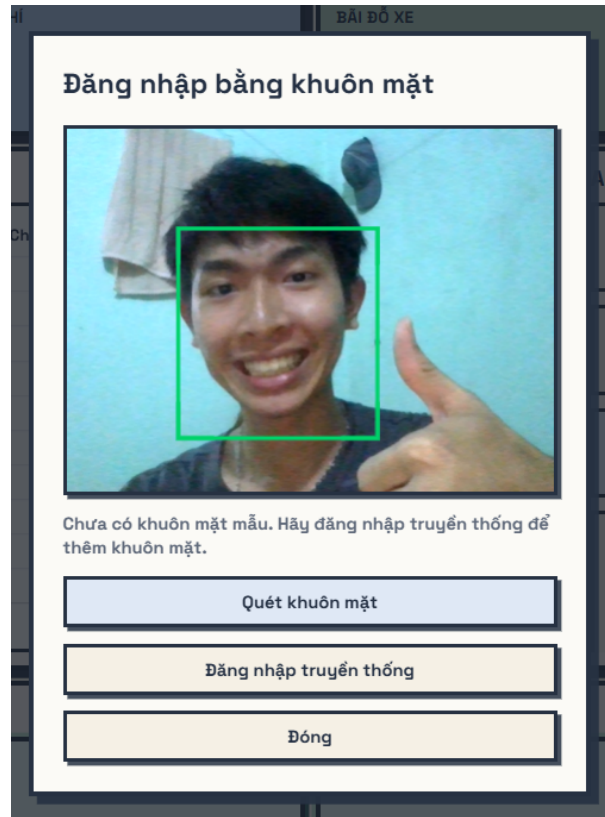
5.4.2 Chế độ MANUAL

Khi admin chuyển sang MANUAL, toàn bộ logic AUTO bị vô hiệu hóa và admin có toàn quyền điều khiển thiết bị trực tiếp từ dashboard. Hệ thống ngăn chặn điều khiển nhầm bằng cơ chế kiểm tra chế độ: nếu thiết bị đang ở AUTO, nút toggle sẽ hiển thị cảnh báo yêu cầu chuyển sang MANUAL trước.

5.4.3 Đồng bộ trạng thái hai chiều

Dù admin thay đổi trạng thái từ web hay firmware tự thay đổi theo logic AUTO, dashboard luôn phản ánh đúng trạng thái hiện tại nhờ cơ chế listener Firebase. Nút toggle và nhãn AUTO/MANUAL được đồng bộ tự động, loại bỏ sự không nhất quán giữa hiển thị và thực tế.

5.5 Kết quả hệ thống xác thực Face ID



Hình 5.5.1: Giao diện xác thực Face ID – khung xanh lá nhận diện khuôn mặt theo thời gian thực

5.5.1 Nhận diện khuôn mặt trên trình duyệt

Nhóm đã tích hợp thành công thư viện `face-api.js` để thực hiện nhận diện khuôn mặt hoàn toàn phía client, không cần máy chủ xử lý hình ảnh. Hệ thống hoạt động được trên các trình duyệt hiện đại hỗ trợ WebRTC (Chrome, Firefox, Edge). Ba mạng nơ-ron được tải và chạy trực tiếp trong trình duyệt: TinyFaceDetector (phát hiện khuôn mặt), FaceLandmark68Net (68 điểm đặc trưng) và FaceRecognitionNet (face descriptor 128 chiều).

5.5.2 Quy trình đăng ký và xác thực

- **Đăng ký (Enroll):** Admin đăng nhập bằng email/mật khẩu truyền thống, mở camera, đặt khuôn mặt vào khung và bấm "Lưu khuôn mặt". Hệ thống thu thập 3–6 mẫu descriptor, lấy trung bình và lưu vào `localStorage`. Thời gian đăng ký trung bình khoảng 1–2 giây.
- **Xác thực (Verify):** Admin mở modal Face ID, đặt khuôn mặt vào khung và bấm "Quét khuôn mặt". Hệ thống thu thập mẫu mới, tính khoảng cách Euclidean với mẫu đã đăng ký. Nếu khoảng cách ≤ 0.38 , xác thực thành công và admin được cấp quyền điều khiển.
- **Ngưỡng bảo mật:** Ngưỡng 0.38 được chọn qua thực nghiệm, đảm bảo tỉ lệ chấp nhận đúng (True Accept Rate) cao trong điều kiện ánh sáng bình thường, đồng thời đủ chặt để từ chối khuôn mặt

không phải admin.

5.5.3 Tính năng theo dõi trực tiếp

Tính năng theo dõi trực tiếp (Live Tracking) là một trong những thành phần tương tác quan trọng nhất, đóng vai trò làm cầu nối giao tiếp trực quan giữa hệ thống trí tuệ nhân tạo và người dùng trong suốt quá trình đăng ký hoặc xác thực sinh trắc học.

Về mặt kỹ thuật hiển thị, thay vì can thiệp trực tiếp vào luồng dữ liệu video gốc, hệ thống sử dụng một thẻ `<canvas>` trong suốt được đặt xếp chồng (overlay) khớp hoàn toàn lên trên thẻ `<video>`. Thuật toán sẽ liên tục tính toán tọa độ khuôn mặt và cập nhật vị trí khung bao (bounding box) theo thời gian thực với chu kỳ quét được thiết lập cố định ở mức 120ms. Việc thiết lập chu kỳ 120ms là kết quả của quá trình tinh chỉnh thực nghiệm nhằm đạt được điểm cân bằng lý tưởng về mặt hiệu năng: tốc độ này đủ nhanh để khung nhận diện bám sát các chuyển động tự nhiên của người dùng mà không bị trễ hình (lag), đồng thời cũng đủ giãn cách để không gây tắc nghẽn vòng lặp sự kiện (event loop) và làm quá tải CPU của trình duyệt web.

Về mặt thiết kế giao diện (UI/UX), thay vì sử dụng một hình chữ nhật bao quanh toàn bộ khuôn mặt theo cách truyền thống vốn gây cảm giác gò bó, hệ thống áp dụng kỹ thuật chỉ vẽ các "góc L" tại bốn góc của khung định vị. Ngôn ngữ thiết kế này được lấy cảm hứng từ các giao diện hiển thị HUD (Heads-Up Display) trong các hệ thống công nghệ cao. Nó không chỉ mang lại trải nghiệm thị giác tinh tế, hiện đại mà còn giúp khu vực trung tâm khuôn mặt của người dùng không bị che khuất, tạo cảm giác thân thiện và tự nhiên khi tương tác với camera.

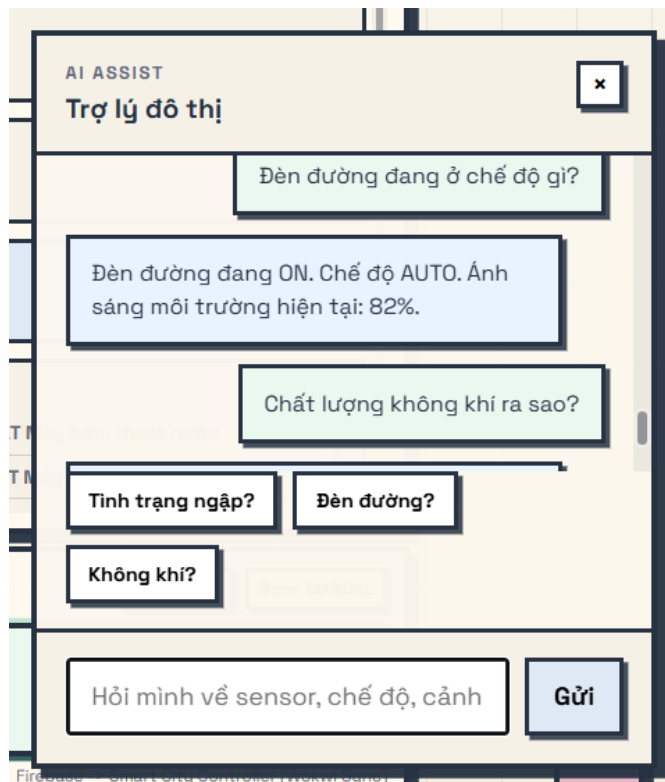
Để hướng dẫn người quản trị thao tác chuẩn xác, hệ thống được tích hợp một máy trạng thái (state machine) để quản lý chuỗi thông báo phản hồi (feedback). Các thông điệp được chuyển đổi linh hoạt và hiển thị liên tục trên màn hình đồng bộ với tiến trình của AI, bao gồm: "Đang khởi tạo camera...", "Đã bắt được khuôn mặt" (khi hệ thống khóa được mục tiêu), "Đang nhận diện..." (khi tiến hành trích xuất vector đặc trưng), và kết thúc bằng "Xác thực thành công" hoặc thông báo từ chối. Cơ chế phản hồi thị giác tức thì này giúp giảm thiểu tối đa sự bối rối của người dùng, đặc biệt trong các kịch bản môi trường có ánh sáng yếu hoặc khi góc mặt bị lệch khỏi vùng quét tối ưu.

Profile khuôn mặt được lưu vào `localStorage` của trình duyệt dưới khóa `smartcity_face_descriptor_v1`. Khi tải lại trang, hệ thống tự động phục hồi profile mà không cần đăng ký lại, mang lại trải nghiệm liền mạch cho người dùng thường xuyên.

5.6 Kết quả tích hợp trợ lý AI (LLaMA 3.1)

5.6.1 Trợ lý nhận thức ngữ cảnh đô thị

Trợ lý AI đã được tích hợp thành công với khả năng nhận thức ngữ cảnh hoàn toàn từ dữ liệu dashboard. Khi người dùng hỏi về tình trạng ngập, chất lượng không khí, đèn đường hay máy bơm, trợ lý trả lời dựa trên giá trị cảm biến thực tế đang hiển thị trên dashboard, không phải dữ liệu cố định hay bịa đặt.



Hình 5.6.1: Panel trợ lý AI đang trả lời câu hỏi dựa trên dữ liệu cảm biến thời gian thực

5.6.2 Khả năng đa nhiệm

Ngoài chức năng hỗ trợ smart city, trợ lý có thể trò chuyện tự nhiên về các chủ đề học tập, công nghệ, lập trình và đời sống. System prompt được thiết kế để trợ lý không từ chối câu hỏi ngoài lề, tạo trải nghiệm linh hoạt và thân thiện.

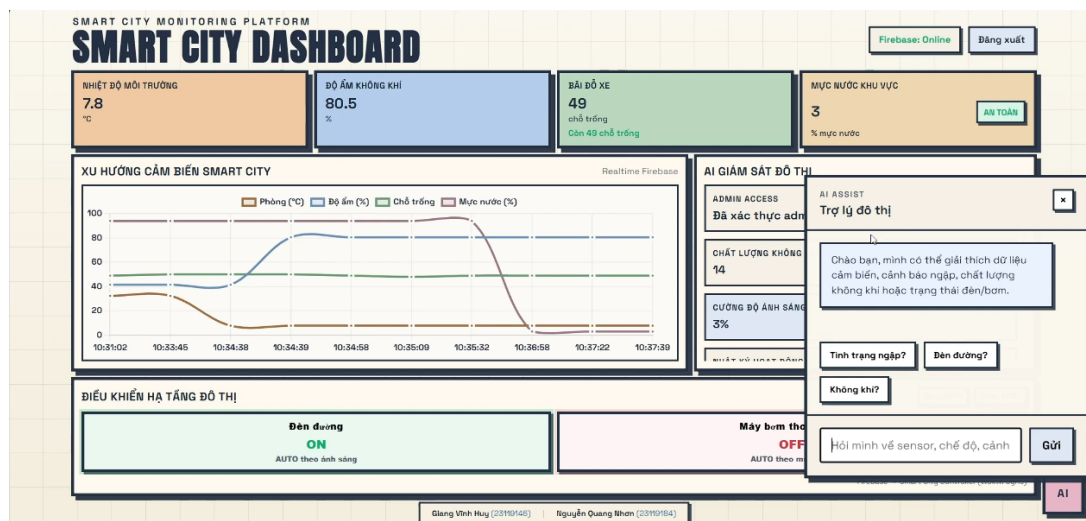
5.6.3 Cơ chế dự phòng thông minh

Khi API Groq không khả dụng, hệ thống tự động chuyển sang trợ lý nội tuyến dựa trên từ khóa, đảm bảo người dùng vẫn nhận được phản hồi có giá trị về dữ liệu sensor. Cơ chế dự phòng này làm cho trợ lý hoạt động đáng tin cậy ngay cả trong điều kiện kết nối không ổn định.

5.6.4 Gợi ý câu hỏi nhanh

Dashboard tích hợp các chip gợi ý câu hỏi nhanh (quick prompt chips), cho phép người dùng hỏi các câu hỏi phổ biến bằng một cú nhấp chuột mà không cần gõ. Tính năng này cải thiện đáng kể trải nghiệm người dùng, đặc biệt với người lần đầu sử dụng hệ thống.

5.7 Kết quả giao diện web dashboard



Hình 5.7.1: Giao diện tổng

5.7.1 Thiết kế trực quan

Dashboard được hoàn thiện với thiết kế phong cách *analog editorial* nhất quán từ đầu đến cuối. Bảng màu kem vàng với bóng đổ cứng tạo cảm giác đặc sắc, khác biệt so với các dashboard IoT thông thường. Font chữ Anton cho tiêu đề và Space Grotesk cho nội dung tạo sự phân cấp thị giác rõ ràng.

5.7.2 Tính năng phân quyền

Giao diện hiển thị hai trạng thái rõ ràng: chế độ xem công khai (không cần đăng nhập, xem được toàn bộ dữ liệu cảm biến) và chế độ admin (sau xác thực, hiện ra các nút điều khiển thiết bị và chuyển chế độ). Các phần tử điều khiển được ẩn/hiện tự động theo trạng thái đăng nhập, tránh gây nhầm lẫn cho người xem.

5.7.3 Nhật ký hoạt động

Phần nhật ký hiển thị 10 hành động gần nhất của admin theo thứ tự mới nhất lên đầu, với timestamp đầy đủ theo chuẩn Việt Nam. Nhật ký được lưu trên Firebase, đảm bảo tính bền vững và có thể kiểm tra từ nhiều thiết bị khác nhau.

5.8 Đánh giá tổng thể

5.8.1 Mức độ hoàn thành mục tiêu

Bảng 5.8.1: Đánh giá mức độ hoàn thành các mục tiêu đề ra

STT	Mục tiêu	Mức độ	Ghi chú
1	Mô phỏng phần cứng ESP32 + cảm biến trên Wokwi	Hoàn thành	9 linh kiện, đủ kích bản
2	Kết nối Firebase Realtime Database	Hoàn thành	Đồng bộ hai chiều, < 500ms
3	Giám sát 6 thông số môi trường + đô thị	Hoàn thành	Realtime, cảnh báo ngưỡng
4	Biểu đồ lịch sử 12 điểm	Hoàn thành	Chart.js, 4 dataset
5	Điều khiển thiết bị AUTO/MANUAL	Hoàn thành	Đèn đường + máy bơm
6	Xác thực Face ID không cần backend	Hoàn thành	face-api.js, ngưỡng 0.38
7	Tích hợp LLaMA 3.1 qua Groq API	Hoàn thành	Ngữ cảnh realtime, dự phòng
8	Nhật ký hoạt động	Hoàn thành	Lưu Firebase, 10 log gần nhất
9	Giao diện thẩm mỹ và phân quyền	Hoàn thành	Analog editorial, admin UI

5.8.2 Hạn chế còn tồn tại

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần được cải thiện trong các phiên bản tiếp theo. Thứ nhất, Face ID hoạt động phụ thuộc vào điều kiện ánh sáng; trong môi trường thiếu sáng, tỉ lệ nhận diện giảm đáng kể. Thứ hai, dữ liệu cảm biến đến từ môi trường mô phỏng Wokwi nên chưa phản ánh nhiều và các trường hợp ngoại lệ của cảm biến vật lý thực tế. Thứ ba, API key được đặt trực tiếp trong mã nguồn phía client là một rủi ro bảo mật cần được giải quyết bằng cách chuyển sang backend proxy trước khi triển khai thực tế. Thứ tư, hệ thống hiện chỉ hỗ trợ một admin duy nhất, chưa có cơ chế quản lý đa người dùng với phân quyền chi tiết hơn.

Chương 6

KẾT LUẬN, NHỮNG HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

6.1 Kết luận

Sau quá trình nghiên cứu, thiết kế và triển khai, nhóm đã hoàn thành đề tài **Hệ thống Giám sát và Điều khiển Đô thị Thông minh (Smart City IoT)** với đầy đủ các chức năng đề ra ban đầu. Đây là một hệ thống tích hợp hoàn chỉnh, kết hợp giữa lập trình nhúng trên vi điều khiển ESP32, nền tảng đám mây Firebase, giao diện web thời gian thực và các công nghệ trí tuệ nhân tạo tiên tiến.

6.1.1 Những gì đã đạt được

Về mặt kỹ thuật, nhóm đã xây dựng thành công một hệ thống IoT hoạt động theo mô hình ba tầng chặt chẽ. Tầng cảm biến với ESP32 thu thập đồng thời sáu thông số đô thị quan trọng gồm nhiệt độ, độ ẩm, mực nước, chất lượng không khí, cường độ ánh sáng và trạng thái bãi đỗ xe. Tầng trung gian sử dụng Firebase Realtime Database làm cầu nối truyền thông hai chiều với độ trễ dưới 500ms, đảm bảo tính đồng bộ liên tục giữa phần cứng và giao diện người dùng. Tầng ứng dụng là một web dashboard hoàn chỉnh với khả năng giám sát trực quan, điều khiển thiết bị thông minh và hệ thống phân quyền bảo mật.

Một trong những điểm nổi bật nhất của đề tài là việc tích hợp thành công hai công nghệ AI hiện đại vào một hệ thống IoT quy mô nhỏ. Hệ thống xác thực Face ID được xây dựng hoàn toàn phía trình duyệt (client-side) bằng thư viện `face-api.js`, không phụ thuộc vào bất kỳ máy chủ xử lý hình ảnh nào, sử dụng thuật toán so sánh khoảng cách Euclidean trên vector đặc trưng 128 chiều với ngưỡng phân loại 0.38 được xác định qua thực nghiệm. Song song đó, trợ lý AI được tích hợp mô hình ngôn ngữ lớn LLaMA 3.1 (70 tỷ tham số) thông qua Groq API, với cơ chế xây dựng system prompt động nhúng trực tiếp dữ liệu cảm biến thời gian thực, cho phép trợ lý trả lời chính xác về tình trạng đô thị hiện tại.

Về mặt học thuật, đề tài đã giúp nhóm củng cố và mở rộng kiến thức trên nhiều lĩnh vực: lập trình nhúng ESP32 với PlatformIO, thiết kế cơ sở dữ liệu NoSQL thời gian thực, phát triển ứng dụng web JavaScript hiện đại theo dạng ES Module, tích hợp API bên thứ ba, và ứng dụng thực tế của các mô hình học sâu trong nhận diện khuôn mặt. Đặc biệt, trải nghiệm làm việc với công cụ mô phỏng Wokwi giúp nhóm hiểu sâu hơn về quy trình phát triển firmware mà không cần phần cứng thực tế, một kỹ năng ngày càng quan trọng trong ngành kỹ thuật nhúng hiện đại.

6.1.2 Đánh giá mức độ hoàn thành

Nhìn lại toàn bộ quá trình thực hiện, nhóm đánh giá đề tài đã hoàn thành ở mức độ tốt so với mục tiêu ban đầu đề ra. Tất cả 10 chức năng cốt lõi đã được triển khai và kiểm thử thành công trong môi trường mô phỏng. Hệ thống hoạt động ổn định, giao diện người dùng trực quan và thẩm mỹ, các tính năng AI mang lại giá trị thực tiễn rõ ràng. Đây là nền tảng vững chắc để phát triển thành một sản phẩm thực tế trong tương lai.

6.2 Những hạn chế còn tồn tại

Bên cạnh những kết quả đạt được, nhóm nhận thức rõ ràng rằng hệ thống hiện tại vẫn còn tồn tại một số hạn chế đáng kể cần được thừa nhận một cách thành thật.

6.2.1 Hạn chế về môi trường mô phỏng

Toàn bộ phần cứng của dự án được triển khai trên nền tảng mô phỏng Wokwi thay vì phần cứng vật lý thực tế. Điều này dẫn đến một số khác biệt quan trọng so với điều kiện thực tế. Các cảm biến vật lý thực tế (DHT22, cảm biến mực nước, cảm biến ánh sáng) thường xuất hiện nhiễu tín hiệu, độ trôi theo nhiệt độ, và các giá trị ngoại lệ mà môi trường mô phỏng không thể tái hiện đầy đủ. Trong môi trường Wokwi, ba biến trở được dùng để thay thế cho ba loại cảm biến hoàn toàn khác nhau về nguyên lý hoạt động, dẫn đến việc dữ liệu đầu vào quá lý tưởng và thiếu tính đại diện cho điều kiện thực tế ngoài hiện trường.

6.2.2 Hạn chế về bảo mật

Đây là hạn chế nghiêm trọng nhất về mặt kỹ thuật của hệ thống hiện tại. Các khóa API nhạy cảm bao gồm Firebase API key, Groq API key đang được khai báo trực tiếp trong mã nguồn JavaScript phía client (`script.js`). Do JavaScript phía client hoàn toàn có thể bị người dùng bất kỳ đọc qua công cụ Developer Tools của trình duyệt, việc này tạo ra rủi ro lộ khóa API ra bên ngoài. Trong môi trường thực tế, các khóa này cần được quản lý thông qua biến môi trường phía server hoặc một backend proxy đóng vai trò trung gian. Ngoài ra, profile khuôn mặt của admin được lưu trong `localStorage` của trình duyệt mà không được mã hóa, là một điểm yếu bảo mật cần được khắc phục.

6.2.3 Hạn chế về hệ thống xác thực Face ID

Hệ thống Face ID hiện tại chỉ hỗ trợ đăng ký và xác thực cho một admin duy nhất do chỉ lưu một vector descriptor trong `localStorage`. Hệ thống phụ thuộc nhiều vào điều kiện ánh sáng môi trường; trong điều kiện thiếu sáng hoặc ngược sáng, tỉ lệ nhận diện chính xác giảm đáng kể. Hơn nữa, việc lưu dữ liệu sinh trắc học trong `localStorage` của một trình duyệt cụ thể có nghĩa là admin không thể đăng nhập bằng Face ID từ trình duyệt hoặc thiết bị khác, làm giảm tính linh hoạt của hệ thống.

6.2.4 Hạn chế về khả năng mở rộng

Hệ thống hiện tại được thiết kế và tối ưu cho một node ESP32 duy nhất với số lượng cảm biến cố định. Cấu trúc Firebase chưa được thiết kế để hỗ trợ nhiều node IoT hoạt động đồng thời, chưa có cơ chế phân vùng dữ liệu theo khu vực địa lý hay theo loại cảm biến. Dashboard cũng chưa có tính năng lọc, tìm kiếm hay xuất dữ liệu lịch sử, hạn chế khả năng phân tích dữ liệu theo chiều dọc thời gian.

6.2.5 Hạn chế về trợ lý AI

Mô hình LLaMA 3.1 được sử dụng thông qua API của bên thứ ba (Groq), điều này tạo ra sự phụ thuộc vào dịch vụ bên ngoài và phát sinh chi phí khi sử dụng ở quy mô lớn. Trợ lý hiện tại chỉ có thể trả lời câu hỏi và cung cấp thông tin, nhưng chưa có khả năng chủ động đưa ra cảnh báo hay đề xuất hành động khi phát hiện tình huống bất thường trong dữ liệu cảm biến.

6.3 Hướng phát triển trong tương lai

Dựa trên nền tảng đã xây dựng và những hạn chế đã nhận diện, nhóm đề xuất các hướng phát triển cụ thể để nâng cấp hệ thống lên mức độ sẵn sàng triển khai thực tế.

6.3.1 Triển khai phần cứng thực tế

Bước phát triển quan trọng nhất là chuyển từ môi trường mô phỏng sang phần cứng vật lý thực tế. Cụ thể, nhóm dự kiến thay thế các biến trở mô phỏng bằng cảm biến chuyên dụng: cảm biến siêu âm HC-SR04 hoặc cảm biến áp suất nước cho đo mực nước, cảm biến khí MQ-135 hoặc PMS5003 cho chất lượng không khí, cảm biến quang trở LDR hoặc BH1750 cho ánh sáng môi trường. Việc triển khai thực tế sẽ đòi hỏi bổ sung các kỹ thuật xử lý tín hiệu như lọc nhiễu trung bình động (moving average), hiệu chỉnh offset và bù nhiệt độ để đảm bảo độ chính xác của dữ liệu.

6.3.2 Xây dựng backend bảo mật

Để giải quyết hạn chế bảo mật nghiêm trọng nhất, nhóm đề xuất xây dựng một backend server nhẹ (có thể sử dụng Node.js với Express hoặc Python với FastAPI) đóng vai trò proxy trung gian. Backend này sẽ quản lý toàn bộ các khóa API nhạy cảm thông qua biến môi trường, cung cấp endpoint xác thực an toàn cho dashboard, và có thể mở rộng thêm các chức năng như lưu trữ lịch sử dài hạn vào cơ sở dữ

liệu SQL, gửi thông báo push notification khi có cảnh báo, và quản lý đa người dùng với phân quyền chi tiết.

6.3.3 Nâng cấp hệ thống Face ID

Hệ thống xác thực sinh trắc học cần được nâng cấp theo nhiều hướng. Về chức năng, cần hỗ trợ đa admin với cơ sở dữ liệu descriptor lưu trên server thay vì `localStorage`, cho phép đăng nhập từ bất kỳ thiết bị nào. Về bảo mật, descriptor khuôn mặt cần được mã hóa trước khi lưu trữ, và cần áp dụng cơ chế chống giả mạo (liveness detection) để phân biệt khuôn mặt thật với ảnh chụp. Về độ chính xác, có thể nghiên cứu thay thế `face-api.js` bằng các model mạnh hơn như ArcFace hoặc tích hợp với dịch vụ nhận diện khuôn mặt đám mây để cải thiện hiệu suất trong điều kiện ánh sáng kém.

6.3.4 Mở rộng quy mô hệ thống

Để hệ thống có thể ứng dụng thực tế cho một khu đô thị, cần thiết kế lại kiến trúc Firebase để hỗ trợ nhiều node IoT phân tán. Cấu trúc dữ liệu có thể được tổ chức theo mô hình `smartcity/zones/{zone_id}/sensors` để quản lý dữ liệu theo khu vực địa lý. Dashboard cần được bổ sung tính năng chọn khu vực, so sánh dữ liệu giữa các vị trí, và hiển thị bản đồ thực tế tích hợp Google Maps hoặc OpenStreetMap thay vì bản đồ SVG minh họa hiện tại.

6.3.5 Tích hợp phân tích dữ liệu và AI dự báo

Hướng phát triển dài hạn và có giá trị nhất là bổ sung khả năng phân tích và dự báo thông minh. Với lượng dữ liệu lịch sử được tích lũy theo thời gian, có thể huấn luyện các mô hình học máy (Machine Learning) để dự báo nguy cơ ngập úng dựa trên xu hướng mực nước và dự báo thời tiết, dự báo nhu cầu đỗ xe theo giờ trong ngày và ngày trong tuần, phát hiện bất thường (anomaly detection) trong các chỉ số môi trường. Trợ lý AI cũng có thể được nâng cấp từ chế độ phản hồi thụ động sang chủ động: tự động phân tích xu hướng dữ liệu và đưa ra cảnh báo sớm trước khi tình huống nguy hiểm xảy ra.

6.3.6 Tối ưu hiệu năng và trải nghiệm người dùng

Về mặt kỹ thuật web, dashboard cần được tối ưu để hỗ trợ đầy đủ trên thiết bị di động (responsive design hoàn chỉnh), bổ sung chế độ Progressive Web App (PWA) để có thể cài đặt và sử dụng offline, và tích hợp thông báo push trên trình duyệt khi có cảnh báo khẩn cấp. Biểu đồ lịch sử cần được mở rộng để hỗ trợ lọc theo khoảng thời gian tùy chọn và xuất dữ liệu ra định dạng CSV hoặc Excel phục vụ phân tích ngoại tuyến.

6.4 Lời kết

Đề tài **Hệ thống Giám sát và Điều khiển Đô thị Thông minh** không chỉ là một bài tập kỹ thuật về IoT, mà còn là cơ hội để nhóm tiếp cận và tích hợp nhiều công nghệ đang định hình tương lai của các thành phố hiện đại: từ kết nối vạn vật (IoT), điện toán đám mây thời gian thực, cho đến trí tuệ

CHƯƠNG 6. KẾT LUẬN, NHỮNG HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

nhân tạo và nhận diện sinh trắc học. Mặc dù hệ thống hiện tại vẫn ở dạng nguyên mẫu (prototype) với nhiều hạn chế cần khắc phục, nền tảng kỹ thuật và kiến trúc ba tầng đã được xây dựng vững chắc, tạo điều kiện thuận lợi để mở rộng và nâng cấp trong các giai đoạn tiếp theo.

Nhóm tin rằng với hướng phát triển đã đề xuất, đặc biệt là việc triển khai phần cứng thực tế và xây dựng backend bảo mật, hệ thống hoàn toàn có tiềm năng trở thành một giải pháp thực tiễn đóng góp vào công cuộc xây dựng đô thị thông minh tại Việt Nam trong tương lai gần.

Chương 7

KẾT QUẢ VẬN HÀNH VÀ HƯỚNG DẪN THAO TÁC

Nhằm cung cấp cái nhìn trực quan và minh chứng cho toàn bộ kết quả thực hiện của đề án, nhóm đã tiến hành ghi hình một video báo cáo tổng hợp. Video này vừa trình diễn kịch bản hoạt động của phần cứng mô phỏng, vừa hướng dẫn chi tiết cách thức thao tác và tương tác với giao diện Web Dashboard.

7.1 Video Demo và Hướng dẫn hệ thống

Video bao quát toàn bộ quy trình vận hành thực tế của hệ thống Smart City IoT từ việc xác thực bảo mật đến điều khiển thiết bị tự động và giao tiếp với trợ lý AI.

- **Tên video:** Mô phỏng Hệ thống IoT Smart City – ESP32 + Firebase + Face ID | Wokwi Simulator
- **Nội dung trình diễn:**
 - Giới thiệu cấu trúc phân lớp của cơ sở dữ liệu Firebase (sensors, devices, modes, logs).
 - Hướng dẫn đăng nhập tài khoản admin và quy trình đăng ký/nhận diện sinh trắc học Face ID.
 - Trình diễn chạy mô phỏng mạch Wokwi đồng bộ thời gian thực với Web Dashboard (chia đôi màn hình).
 - Thử nghiệm thay đổi thông số ngoại vi: Cảm biến nhiệt độ/độ ẩm DHT22, nút nhấn bãi đỗ xe, các biến trở mô phỏng mực nước, ánh sáng và chất lượng không khí (AQI).
 - Vận hành hệ thống thiết bị chấp hành (Đèn đường LED, Relay máy bơm) phản hồi theo 2 chế độ điều khiển: AUTO và MANUAL.
 - Tương tác thực tế với Trợ lý ảo AI (truy vấn thông số môi trường đô thị trực tiếp và giao tiếp tự nhiên).
- **Thời lượng:** 08 phút 13 giây.
- **Link truy cập:** <https://youtu.be/hmhnHT6XhoQ>

7.2 Mã QR truy cập nhanh

Trong trường hợp đọc báo cáo bản in, quý Thầy có thể sử dụng điện thoại thông minh quét mã QR dưới đây để truy cập trực tiếp vào video demo của nhóm.



Hình 7.2.1: Mã QR truy cập Video vận hành hệ thống Smart City

Ghi chú: Video được lưu trữ trên nền tảng YouTube nhằm đảm bảo tính sẵn sàng truy cập mọi lúc, giúp dễ dàng đánh giá khách quan kết quả của đồ án.

Tài liệu tham khảo

- [1] Google. (2024) Firebase realtime database documentation. [Online]. Available: <https://firebase.google.com/docs/database>
- [2] M. D. Network. (2024) Web technology for developers. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web>
- [3] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. O'Reilly Media, 2020.
- [4] M. D. Network. (2024) Introduction to the dom. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- [5] E. Marcotte, “Responsive web design,” *A List Apart*, 2010. [Online]. Available: <https://alistapart.com/article/responsive-web-design/>
- [6] Chart.js Contributors. (2024) Chart.js documentation. [Online]. Available: <https://www.chartjs.org/docs/latest/>
- [7] V. Mühler. (2023) face-api.js: Javascript api for face detection and recognition in the browser. [Online; accessed 20-April-2026]. [Online]. Available: <https://github.com/justadudewhohacks/face-api.js>
- [8] Groq Inc. (2024) Groq api documentation. [Online; accessed 20-April-2026]. [Online]. Available: <https://console.groq.com/docs/>
- [9] Espressif Systems, *ESP32 Technical Reference Manual*, 2023, [Online; accessed 20-April-2026]. [Online]. Available: <https://www.espressif.com/>